学校代码：10284
分 类 号：TP316.8
密　　级：公开
U D C：004.75
学　　号：502022330070



南京大学

硕 士 学 位 论 文

论 文 题 目	面向 AI 模型推理的流处理资源优化关键技术研究
作 者 姓 名	张 晓 铮
专 业 名 称	计 算 机 科 学 与 技 术
研 究 方 向	大 数 据 与 云 计 算
导 师 姓 名	戴海鹏 副教授、顾荣 副教授

2025 年　5 月 30 日

答辩委员会主席	吴楠 副教授
评　　阅　　人	吴楠 副教授
	韩磊　　教授

论文答辩日期 2025 年　5 月 19 日

研究生签名：

导师签名：



南京大学申请硕士学位论文

面向 AI 模型推理的流处理资源优化关键技术研究

作者： 张晓铮

专业： 计算机技术

研究方向： 大数据与云计算

指导教师： 戴海鹏 副教授、顾荣 副教授

南京大学计算机科学与技术系

2025 年 5 月



Research on Key Technology of Stream Processing Resource Optimization for AI Model Inference

Presented By

Xiaozheng Zhang

Supervised by

Haipeng Dai Associate Professor

Rong Gu Associate Professor

A DISSERTATION
FOR THE APPLICATION OF MASTER DEGREE
SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE
AND TECHONOLOGY OF NANJING UNIVERSITY

2025 May

南京大学学位论文原创性声明

本人郑重声明，所提交的学位论文是本人在导师指导下独立进行科学研究工作所取得的成果。除本论文中已经注明引用的内容外，本论文不包含其他个人或集体已经发表或撰写过的研究成果，也不包含为获得南京大学或其他教育机构的学位证书而使用过的材料。对本文的研究做出重要贡献的个人和集体,均已在论文的致谢部分明确标明。

本人郑重声明愿承担本声明的法律责任。

签名：

日期：

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目：面向AI模型推理的流处理资源优化关键技术研究

计算机科学与技术 专业 2022 级硕士生姓名：张晓铮

指导教师（姓名、职称）：戴海鹏 副教授、顾荣 副教授

随着深度学习、大模型为代表的新一代人工智能（AI）技术飞速发展，越来越多的企业和组织开始利用人工智能赋能其业务。AI 模型推理是实际应用中实现人工智能技术价值的“最后一公里”。近年来，AI模型推理需求呈现爆发式增长以及负载动态变化的特点。因此，业界和学界都在关注研究基于分布式流处理系统构建高效分布式AI模型推理服务的关键技术。分布式流处理系统给在线动态AI推理负载请求下的AI模型服务性能提升带来了重要发展机遇。然而，传统分布式流处理系统并非原生为支撑AI模型推理场景所设计，在实际运行流式AI推理服务时仍然面临诸多难点挑战。

首先，为提升流式AI模型推理作业的数据预处理阶段性能，系统通常会使用缓存，然而上游数据负载动态多变，为其在云平台分配合适缓存容量困难。其次，对于需要在线不间断服务的流式AI模型推理作业（称为在线流式AI模型推理作业），数据速率动态变化导致决策作业资源快速、准确扩缩容策略困难。最后，对于离线定期化运行的流式AI模型推理作业（称为离线流式AI模型推理作业），如何对作业端到端处理算子进行全局最优资源配置，以提升系统总体吞吐性能成为关键问题。

综上所述，本文围绕AI模型推理场景下的流处理资源优化问题展开。首先，在流式AI模型推理作业的数据预处理阶段，本文通过优化其缓存资源，从而奠定作业的计算性能基础。其次针对在线AI模型推理的动态负载特性，设计了弹性扩缩容策略，以实现实时资源适配，同时针对离线AI模型推理作业，建立了全局最优资源配置求解方法，从而达成资源最大化利用。最后，本文基于轻量化分布式计算框架 Ray 设计实现了相应的流式 AI 模型推理作业的自动优化系统。具体来说，本文主要研究工作如下：

（1）针对流式AI模型推理数据预处理阶段的缓存配额难确定问题，本文提出了一种基于处理语义的缓存优化方法。该方法使用基于处理语义信息的流状态

缓存容量调整策略，并结合基于近似数据结构的流状态感知方法，对数据预处理所需缓存容量进行动态调整，在满足性能要求的同时减少了缓存资源浪费。

（2）针对在线流式 AI 模型推理作业资源扩缩容难决策的问题，本文提出了一种基于性能建模的作业资源扩缩容策略。该策略首先对流式 AI 模型推理作业进行性能建模，并结合一种基于动态规划的配置求解算法对作业所需的资源配置进行快速求解，使得作业可以应对输入数据动态速率变化的场景，避免了作业性能下降以及端到端时延增加。

（3）针对离线流式 AI 模型推理作业资源配置最优化问题，本文提出了一种全局化的资源最优化策略。该策略分别针对成本最优与吞吐最优两类常见场景提出了配置求解算法。算法结合作业性能模型以及作业拓扑结构，能够为作业整体链路求解全局最优资源配置方案，从而降低资源成本、提高作业性能。

（4）基于以上研究，本文基于轻量化分布式计算框架 Ray 设计实现了一个流式 AI 模型推理作业的自动优化系统。实验结果表明，在输入数据动态变化的场景，对比静态缓存容量方法，本文的数据预处理缓存优化方法可以提升约 20%作业吞吐。其次，对比前沿的扩缩容策略，本文的扩缩容策略可以在满足性能以及端到端时延的前提下节省约 25%的 GPU 资源。最后，对于离线流式 AI 模型推理作业的资源最优化方法，对比基于主流调参经验的自动调参方法，可以节省约 30%的 GPU 资源并提高约 45%的作业性能。最后，本文系统在某互联网社交媒体头部公司进行了测试应用，并取得了较好效果。

关键词：AI 模型推理；分布式流式处理；资源弹性扩缩容；参数自动化配置

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Research on Key Technology of Stream Processing Resource Optimization for AI Model Inference

SPECIALIZATION: Computer Science and Technology

POSTGRADUATE: Xiaozheng Zhang

MENTOR: Haipeng Dai Associate Professor, Rong Gu Associate Professor

With the rapid development of next-generation artificial intelligence (AI) technologies, represented by deep learning and large-scale models, an increasing number of enterprises and organizations are leveraging AI to empower their businesses. AI model inference serves as the "last mile" in realizing the practical value of AI technologies. In recent years, the demand for AI model inference has experienced explosive growth, accompanied by dynamic and fluctuating workloads. Consequently, both industry and academia have focused on researching key technologies for building efficient distributed AI model inference services based on distributed stream processing systems. Distributed stream processing systems present significant opportunities for enhancing the performance of AI model services under dynamic online inference workloads. However, traditional distributed stream processing systems were not originally designed to support AI model inference scenarios, leading to numerous challenges in deploying streaming AI inference services in practice.

First, to improve the performance of the data preprocessing stage in streaming AI model inference tasks, systems often employ caching. However, the dynamic and variable nature of upstream data loads makes it difficult to allocate appropriate cache capacity in cloud platforms. Second, for streaming AI model inference tasks requiring uninterrupted online services (referred to as online streaming AI model inference tasks), the dynamic variation in data rates complicates the rapid and accurate decision-making for resource scaling strategies. Finally, for periodically executed offline streaming AI model inference tasks (referred to as offline streaming AI model inference tasks), achieving globally optimal resource allocation for end-to-end processing operators to enhance overall system throughput remains a critical challenge.

In summary, this paper focuses on optimizing stream processing resources for AI

model inference scenarios. First, for the data preprocessing stage of streaming AI model inference tasks, we optimize cache resource allocation to establish a computational performance foundation. Second, to address the dynamic workload characteristics of online AI model inference, we design an elastic scaling strategy to enable real-time resource adaptation. Simultaneously, for offline AI model inference tasks, we develop a global optimal resource allocation method to maximize resource utilization. Finally, we implement an automated optimization system for streaming AI model inference tasks based on the lightweight distributed computing framework Ray. Specifically, the main contributions of this paper are as follows:

(1) State-Aware Cache Optimization for Data Preprocessing: To address the challenge of determining cache quotas in the data preprocessing stage of streaming AI model inference, we propose a state-aware cache optimization method. This approach employs approximate data structures for stream state awareness and combines it with a cache capacity adjustment strategy based on processing semantics. This dynamically adjusts the required cache capacity for data preprocessing, reducing resource waste while meeting performance requirements.

(2) Performance-Model-Based Resource Scaling for Online Tasks: To tackle the difficulty of resource scaling decisions for online streaming AI model inference tasks, we propose a performance-model-based scaling strategy. This strategy first constructs a performance model for streaming AI model inference tasks and then employs a dynamic programming-based configuration solver to rapidly determine optimal resource allocations. This enables tasks to adapt to dynamically changing input data rates, preventing performance degradation and increased end-to-end latency.

(3) Global Resource Optimization for Offline Tasks: For the resource optimization problem in offline streaming AI model inference tasks, we propose a global optimization strategy. This strategy introduces configuration-solving algorithms for two common scenarios: cost-optimal and throughput-optimal configurations. By integrating task performance models and topology structures, the algorithm derives globally optimal resource allocation schemes for the entire task pipeline, thereby reducing resource costs and improving performance.

(4) Implementation and Evaluation: Based on the above research, we design and

implement an automated optimization system for streaming AI model inference tasks using the lightweight distributed computing framework Ray. Experimental results demonstrate that, under dynamically changing input data, our cache optimization method for data preprocessing improves task throughput by approximately 20% compared to static cache allocation methods. Additionally, compared to state-of-the-art scaling strategies, our approach saves about 25% of GPU resources while maintaining performance and end-to-end latency requirements. For offline streaming AI model inference tasks, our global optimization method reduces GPU resource usage by approximately 30% and improves task performance by about 45% compared to auto-tuning methods based on mainstream parameter-tuning experience. Finally, our system has been tested and applied in a leading internet social media company, achieving favorable results.

Keywords: AI model inference; distributed stream processing; elastic resource scaling; automated parameter configuration

目　录

第一章 绪论.....1

1.1 研究背景.....1
1.2 现有相关工作及不足.....3
1.2.1 云平台环境下流处理系统优化.....3
1.2.2 流式处理作业扩缩容策略.....4
1.2.3 AI 模型推理服务系统以及参数自动优化.....5
1.3 研究难点问题.....6
1.4 本文主要研究工作.....7
1.5 本文的组织结构.....8

第二章 背景知识.....9

2.1 分布式流式处理系统.....9
2.2 流式 AI 模型推理作业.....10

第三章 云平台环境下流式推理数据预处理缓存优化.....13

3.1 问题背景与分析.....13
3.2 基于近似数据结构的流状态分布高效感知方法.....15
3.2.1 问题定义与描述.....15
3.2.2 基于近似数据结构的频率感知方法.....16
3.3 基于处理语义信息的流状态缓存容量调整策略.....18
3.3.1 容量调整策略概述.....19
3.3.2 基于处理语义信息的流状态缓存容量模型.....21
3.4 实验评估与分析.....23
3.4.1 实验环境与配置.....23
3.4.2 流状态分布感知方法有效性评估.....24
3.4.3 流状态缓存容量调整策略评估.....27
3.5 本章小结.....29

第四章 面向在线流式 AI 模型推理的资源扩缩容策略.....31

4.1 问题背景与分析.....31
4.2 轻量化多阶段的流式推理作业算子性能建模方法.....34
4.2.1 性能模型构建过程概述.....34
4.2.2 混合性能模型构建方法.....35
4.3 基于算子性能模型的流式推理作业并行优化策略.....39
4.3.1 流式推理作业并行优化问题形式化描述.....40
4.3.2 基于动态规划的流式推理作业并行优化算法.....42
4.4 实验评估与分析.....48
4.4.1 实验环境与配置.....48
4.4.2 轻量化多阶段的流处理算子性能建模方法评估.....50
4.4.3 流式推理并行优化策略有效性评估.....52
4.5 本章小结.....56

第五章 面向离线流式 AI 模型推理的资源最优化策略 57

5.1 问题背景与分析.....57
5.2 面向成本最优的流式推理作业资源配置方法.....59
5.2.1 问题形式化描述.....59
5.2.2 基于算子性能模型的资源配置方法.....60
5.3 面向吞吐最优的流式推理作业资源配置方法.....62
5.3.1 问题形式化描述.....62
5.3.2 基于瓶颈点分析的资源配置方法.....63
5.4 实验评估与分析.....64
5.4.1 实验环境与配置.....65
5.4.2 面向成本最优的资源配置方法有效性评估.....65
5.4.3 面向吞吐最优的资源配置方法有效性评估.....66
5.5 本章小结.....67

第六章 系统架构与实现.....69

6.1 系统架构.....69
6.2 组件实现与运行流程.....70
6.2.1 作业先验信息获取方法.....70
6.2.2 作业算子性能建模方法.....71

6.2.3 作业配置求解模块.....72
6.2.4 作业数据预处理缓存优化策略.....73
6.3 系统使用流程.....73
6.4 本章小结.....75
第七章 总结与展望.....77
7.1 本文工作总结.....77
7.2 进一步工作.....78
参考文献.....79
致谢.....83
附录.....85

插图清单

图 1.1　AI 模型推理服务架构示意图 1
图 2.1 流式处理作业示意图. 9
图 2.2 流式处理作业运行时状态示意图 9
图 3.1 本地状态存储与远端状态存储作业吞吐对比. 14
图 3.2 不同数据分布情况下作业吞吐随缓存大小的变化情况 15
图 3.3 聚合算子处理逻辑示意图 21
图 3.4 不同目标缓存命中率下分布感知方法所能达到的实际缓存命中率 25
图 3.5 不同实验配置下本文方法缓存键值数量与真实值对比 26
图 3.6 作业吞吐以及缓存键值数量随时间变化情况 28
图 4.1 输入数据速率动态变化时流式 AI 模型推理作业端到端时延变化情况. 31
图 4.2 某互联网头部社交媒体平台某流式推理作业 24 小时 GPU 利用率情况 32
图 4.3 ViT Tiny 模型推理算子吞吐 38
图 4.4 流式推理作业示意图 43
图 4.5 不同 Batch Size 以及不同并行度下算子性能模型预测误差率 51
图 4.6 不同 Batch Size 步幅下本文方法与 Triton Model Analyzer 的 GPU 资源开销 52
图 4.7 评估过程所使用作业的结构 53
图 4.8 作业吞吐随时间变化情况 53
图 4.9 作业使用 GPU 数量随时间变化情况 53
图 4.10 作业 AI 模型推理算子 Batch Size 随时间变化情况 54
图 4.11 作业端到端延迟 CDF 图 54
图 4.12 对比本文策略与 DS2 作业吞吐随时间变化情况 55
图 4.13 对比本文策略与 DS2 作业使用 GPU 数量随时间变化情况 55
图 4.14 对比本文策略与 DS2 作业 Batch Size 随时间变化情况 55
图 4.15 本文策略与 DS2 作业端到端延迟 CDF 图 56
图 5.1 使用相同配置在不同 GPU 上运行不同模型时的吞吐. 58
图 5.2 示例流式推理作业 59
图 5.3 不同数据集与目标完成时间下作业 GPU 资源开销对比 66
图 5.4 不同数据集与资源量下作业吞吐对比 67
图 6.1 流式 AI 模型推理作业自动优化系统整体架构. 69

图 6.2 作业先验信息获取方法运行流程.....70
图 6.3 作业算子性能建模方法运行流程.....71
图 6.4 配置求解模块运行流程.....72
图 6.5 作业数据预处理缓存优化策略运行流程.....73
图 6.6 用户编写的流式 AI 模型推理示例 74
图 6.7 一键式 API 使用示例 74
图 6.8 系统返回配置方案示例.....74

表格清单

表 3.1 流状态缓存容量调整策略的 Overhead.....29

第一章 绪论

1.1 研究背景

近年来，人工智能（**A**rtificial **I**ntelligence, AI）理论及其相关技术取得了迅猛发展。其中，以深度学习（**D**eep **L**earning, DL）为代表性理论技术（例如大语言模型[1–7]、扩散模型[8]等），结合硬件设施算力与数据资源的飞速增长[9–11]，推动了 AI 技术快速走向产业应用。当前，AI 技术已经成为推动全球经济社会持续发展的关键动力[12]。

在这一趋势下，越来越多的企业开始选择使用 AI 技术，并助力其业务发展，例如使用深度推荐模型对用户进行商品推荐[13]、使用多模态模型对用户发布内容进行分析与审核等[14–16]。而在这一过程中，AI 模型推理是人工智能模型赋能领域应用的“最后一公里”。随着 AI 技术在实际场景应用呈现爆发式增长，AI 模型推理服务系统的性能也面临着新的挑战。如何在资源共享的云计算环境中构建与优化 AI 模型推理系统，使其能够高效、便捷、低成本的对 AI 模型进行推理，具有较深的研究意义与应用价值。

当前一种常见的 AI 模型推理系统架构为将 AI 模型推理部署为单独的推理服务[17–22]，其总体架构如图所示。在图 1.1 中，AI 模型推理被部署为一个单独的服务，供外部用户或其他应用进行请求调用。该服务包含多个服务节点，每个节点使用如NVIDIA Triton Inference Server[23]的组件对外提供AI模型推理服务。在各个服务节点内部，可以根据推理请求使用如 Pytorch[24]、Tensorflow[25]、TensorRT[26]等多种 AI 模型推理框架对某个模型进行推理，并返回结果。

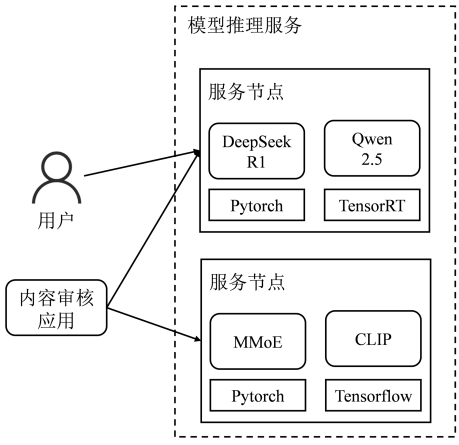


图 1.1　AI 模型推理服务架构示意图

上述方案虽然直观、易用，但在处理复杂 AI 模型推理场景时逐渐暴露出局限性。例如，广告推荐业务流程往往需要串联多种不同的处理步骤（例如用户特征提取、用户画像补全、使用推荐模型推理等），然而采用 AI 模型推理服务的方案时，会导致以下两方面问题：（1）推理计算的前后处理任务导致框架对接复杂：数据预处理、后处理等过程需要使用其他引擎处理；（2）推理请求负载动态变化导致性能不稳定：当多个业务流程同时使用 AI 模型推理服务或者业务负载动态变化时，调用 AI 模型推理服务的耗时不稳定。

针对上述问题，以 Ray[27]、Flink[56]为代表的全链路推理框架开始受到学术界和工业界的广泛关注。这类系统为一种分布式流式处理系统，允许用户将预处理、AI 模型推理、后处理等环节编写为统一的作业，并由系统本身对其中的各个环境进行分布式并行处理。通过上述方法，可以更好的控制业务流程的端到端延迟，并提高硬件利用率与 AI 模型推理性能[28]，从而减低 AI 模型推理成本。

然而，高效集约使用分布式流式处理系统运行 AI 模型推理作业仍然存在着诸多问题。首先，在当前主流的云平台存算分离环境下，数据预处理阶段可能由于额外的网络 IO 开销造成性能下降，从而导致 AI 模型推理作业整体性能下降[29]。其次，某些 AI 模型推理作业需要长时间运行（称为在线流式 AI 模型推理作业），在此期间，外界输入数据的速率往往会动态变化[30–34]，此时作业也需要进行动态扩缩容，这就需要一种策略来准确、快速的进行动态扩缩容决策。最后，对于一部分无需长时间运行的 AI 模型推理作业（称为离线流式 AI 模型推理作业），待处理数据通常为有限的数据集，无需动态扩缩容，然而不合理的资源配置方法可能会导致作业性能低下、资源浪费等问题[35,36]，需要一种方法来进行作业整体链路资源的全局最优化配置。

在上述背景下，本文围绕 AI 模型推理场景下的流处理资源优化问题展开。首先，在流式 AI 模型推理作业的数据预处理阶段，通常使用缓存进行加速，本文提出一种基于处理语义的缓存优化方法，从而奠定作业的计算性能基础。其次针对在线流式 AI 模型推理的动态负载特性，本文设计了一种基于算子性能模型的扩缩容策略，以实现实时资源适配，同时针对离线流式 AI 模型推理作业，本文提出了一种全局最优资源配置求解方法，从而达成资源最大化利用。最后，本文基于轻量化分布式计算框架 Ray 设计实现了相应的流式 AI 模型推理作业的自动优化系统。

1.2 现有相关工作及不足

针对使用分布式流式处理系统运行 AI 模型推理作业所面临的一系列问题，当前存在多方面的相关研究工作。本节将整理现有相关工作，并对这些工作的特点与不足进行分析与阐述。

1.2.1 云平台环境下流处理系统优化

为了优化云平台环境下的流式处理系统所面临的问题，例如负载动态变化、多租户资源共享、大规模状态管理等，当前学术界与工业界已经开展了一系列研究工作。这些工作聚焦于云平台环境下流式处理系统的动态扩缩容机制与策略、大规模流式处理系统管理等，旨在为云平台环境实现易扩展、易维护以及高性能的分布式流处理系统。

ChronoStream [37]是一种分布式流式处理系统，旨在解决云平台环境下的有状态流计算所面临的挑战，包括大状态维护、工作负载波动和多租户资源共享。它引入了一种新颖的状态管理模式，可实现透明的弹性（纵向和横向扩展）和容错，而不会牺牲性能或影响共用租户。该工作考虑了进行算子扩缩容时的状态一致性，但没有考虑负载动态变化时预估缓存容量的问题。

Sponge[38]提出了一种云平台环境下的流式处理系统扩缩容机制。该机制利用 Serverless 函数快速扩缩容的特性，将短时大幅增加的流量交给快速扩容出的 Serverless 函数进行处理，并在此期间针对底层使用的弹性计算实例进行弹性扩缩容，从而有效减低云平台环境下流式处理系统扩缩容时的作业处理延迟。但该工作主要考虑的是负载流量动态变化的场景下，如何对算子实例进行无感扩缩容，没有考虑到状态缓存的扩缩容。

StreamOps [39]是一个云平台环境下流式处理系统的运行时管理系统，旨在解决大规模流式处理系统在可扩展性、高效性等方面的难题。StreamOps 引入了独立的控制平面架构，将策略和机制分离开来，通过轻量级实例和全局状态存储实现可扩展的流式处理系统集群级别管理。该工作主要针对大规模云平台流处理系统的高效管理，例如作业异常情况感知等，但没有单独考虑流状态缓存的优化。

RisingWave [40]是一个云平台环境下的流数据库，旨在提供简单、经济和易用的流式处理服务。该系统通过异步检查点确保快照一致性，并集成了状态存储系统 Hummock，其采用类似于日志结构合并树（Log Structured Merge Tree[41]）

的存储结构，并将远端云存储（如 AWS S3）与本地缓存相结合，从而提供低延迟与大容量的状态键值服务。RisingWave 的状态存储系统 Hummock 在云平台上使用了缓存进行加速，但并没有考虑对缓存进行扩缩容的需求。

Dirigo[42]是一个实现在云平台环境下的分布式流式处理系统，其通过使用 Serverless 架构实现资源与性能的隔离。同时该系统引入了双模式（dual-mode）Actor，其可以在顺序执行与并行执行模型之间动态切换，从而确保了自动扩缩容期间的有序处理。该工作针对 Serverless 环境提出了一种新颖的流处理系统，但对于状态缓存的高效利用缺乏考虑。

上述工作[37–40,42]虽然均在云平台环境下对流式处理系统进行了优化，包括弹性扩缩容机制与策略、流状态管理等，但却没有考虑到对流式处理系统状态缓存进行动态扩缩容的场景，因此无法对状态缓存容量进行动态准确预估。

1.2.2 流式处理作业扩缩容策略

为了有效应对流式处理作业长时间运行期间负载动态变化的情况，近年来学术界已经开展了一系列关于流式处理作业扩缩容策略相关的研究。这些策略旨在快速、准确地为流式处理作业决策出负载变化后所需的新资源配置方案，从而提升流式处理系统的性能、稳定性等。

Dhalion[43]是一种基于规则的流式处理系统扩缩容策略。Dhalion 采用三阶段调整框架（症状检测、诊断、解决），以实现流式处理系统的自动扩缩容决策。其进行扩缩容决策时，主要基于算子的反压（Backpressure）信号以及预设好的诊断规则进行决策，可以简单、快速的应对负载动态变化的场景。基于规则的扩缩容策略简单高效，但难以适应影响因素众多的 AI 模型推理场景。

DS2[44]是一种基于算子当前处理速率以及忙闲比的扩缩容策略。DS2 利用算子忙闲比计算当前算子的实际处理速率，并结合线性模型预测进行扩缩容后的算子处理性能。同时 DS2 能够结合作业的拓扑结构，感知当前算子扩缩容操作对下游算子的影响，从而在一次调整决策中对作业内所有算子进行调整，大幅提高了算法调整效率。DS2 算法简单易用，但其线形模型难以准确覆盖复杂的 AI 模型推理场景。

Turbine[45] 是 Facebook 的流式处理系统服务管理平台，旨在应对大规模流式处理的挑战。它通过将资源预测与历史信息相结合的方式，从而实现主动的扩缩容决策，以处理负载的动态波动，但该工作没有考虑到使用异构资源的 AI

模型推理场景。

StreamSwitch [46]是一种控制平面，其能够在负载动态变化以及负载不均衡的场景下，对流式处理系统的有状态操作进行调整决策。StreamSwitch 通过 “流交换机 ”概念抽象统一了动态扩展和负载均衡，将反应性指标（瞬时延迟）和预测性指标（预计稳态延迟）结合，以诊断系统健康状况并进行最佳资源调整。该工作虽然针对动态扩展和负载均衡问题进行了统一，但同样没有考虑使用 GPU 时对作业性能带来的更多影响因素。

ContTune[47]采用两阶段的方式对流式处理系统进行动态扩缩容决策。其将扩缩容决策过程分为大小两个阶段：在大阶段，算法采用激进的策略将每个算子优先调整至不会反压的状态，接着在小阶段利用贝叶斯优化方法并行对每个算子决策新的并行度。ConTune 通过两阶段算法，可以快速、准确的进行扩缩容决策，但大阶段的激进调整策略，在 AI 模型推理场景下会导致大量 GPU 资源的浪费。

上述工作[43–47]尽管采用多种方法对流式处理系统进行扩缩容决策，但均没有考虑到流式 AI 模型推理场景，忽略了使用 GPU 后带来的新问题，例如 GPU 积攒一批数据后同时执行会取得更高吞吐。这导致了这些策略无法最大化利用 GPU，进而导致资源浪费。

1.2.3 AI 模型推理服务系统以及参数自动优化

在 AI 模型进行推理时，影响其性能的因素多种多样，如何配置相关参数，从而达到 AI 模型推理的最优性能成为重要问题。当前学术界与工业界已经针对该方面问题展开了一系列研究与开发工作。

Optuna [48]是下一代超参数优化框架，其主要创新包括 “运行定义”应用程序接口（define-by-run API），它通过命令式代码动态构建参数搜索空间，从而实现灵活的模块化超参数定义。它集成了高效采样算法（如 TPE[49]、CMA-ES[50]）和异步连续减半剪枝机制，以加速优化。该框架支持从轻量级实验到分布式计算等多种部署方案，同时保持了简便的设置和可扩展性。该参数优化框架较为通用，针对较大的参数搜索空间，需要大量实际运行，需要消耗大量资源。

Morphling [51]是一个自动配置云原生 AI 模型推理服务的框架。Morphling 利用与模型无关的元学习（Model-Agnostic Meta-Learning）来离线训练元模型，从而捕捉不同模型的通用性能趋势。对于新服务，它使用最少的样本调整元模型，以预测最佳配置。该工作仅考虑了单一的 AI 模型推理服务，没有考虑 AI 模型推

理全流程的优化。

Triton Model Analyzer[52]是英伟达推出的一款模型分析工具，其主要与其 AI 模型推理服务组件 Triton Inference Server 配套使用。通过用户自定义待分析模型、参数空间、所需资源等信息，利用 Triton Inference Server 来部署模型，并使用不同的参数配置实际运行模型，从而收集模型的性能表现并产出模型性能报告。该组件同样基于实际运行的方法对 AI 模型推理服务进行优化，在参数搜索空间较大时，同样会消耗大量资源。

VIDUR[53]是一个针对大语言模型（Large Language Model, LLM）的性能模拟器。其通过对 LLM 中包含的计算操作进行单独建模，并结合 LLM 的结构来预测 LLM 在不同配置下的性能，同时 VIDUR 还提供了参数配置求解工具，可以工具 LLM 的性能预测搜索符合用户要求的最优参数配置。该工作仅考虑了大语言模型，没有考虑通用场景的 AI 模型推理服务参数优化。

上述工作[48,51–53]采用多种方法对 AI 模型推理进行参数自动化配置，例如基于实际运行的方法、基于性能模型预测的方法等。但这些工作都仅针对 AI 模型推理，没有考虑全链路 AI 模型推理作业整体的参数自动化配置。对于全链路 AI 模型推理作业来说，需要一种对作业中各个流程进行同步调优的方法。

1.3 研究难点问题

本文提出了一种流式 AI 模型推理作业的自动优化系统，旨在提高流式 AI 模型推理作业的性能与易用性，并降低所需资源成本开销。该系统在设计时存在以下三方面的难点问题：

（1） 流式 AI 模型推理数据预处理阶段缓存容量预估难

在当前主流的云平台环境下，通常使用存算分离架构，这会导致有状态数据预处理操作的性能下降，当前通常使用缓存来进行加速。然而在作业长时间运行期间，输入数据的分布可能会发生变化，从而导致状态分布情况的变化，可能引发状态缓存容量不足，进而造成作业性能下降。为了解决这一问题，需要一种能够在作业运行期间捕获状态的分布情况并预估所需的缓存容量的策略。这要求该策略一方面需要足够轻量，避免影响作业正常运行，另一方面也需要足够准确，从而避免作业性能下降以及缓存资源的浪费。

（2） 在线流式 AI 模型推理作业高效资源扩缩容决策难

对于需要长时间运行的在线流式 AI 模型推理作业，在作业运行期间其外界

输入数据速率通常会动态变化。这些变化可能来自突发社交媒体事件等，通常难以预测，这就要求作业能够根据外界输入数据速率进行动态资源扩缩容决策。然而，对于在线流式 AI 模型推理作业，由于其使用了异构资源（CPU 与 GPU），导致其在进行资源扩缩容时需要考虑更多影响因素，例如如何权衡 GPU 利用率与作业端到端时延。同时，由于在线作业的外界输入数据速率可能会短时间内激增，因此资源扩缩容策略需具备高效、快速的决策能力。

（3） 离线流式 AI 模型推理作业资源最优化配置难

对于无需长时间运行的离线流式 AI 模型推理作业，其输入数据通常为固定大小数据集，无需考虑负载动态变化的场景以及作业端到端时延要求，用户往往追求资源的最优化利用。然而，在对离线流式 AI 模型推理作业进行最优化资源配置时，会面临两方面问题：一方面为 AI 模型推理由于需要使用 GPU，其计算过程较为复杂，对其资源参数配置进行优化较为困难，另一方面为需要对作业整体链路进行全局资源配置优化，避免资源分配不合理导致作业整体性能下降。

1.4 本文主要研究工作

为解决在使用与运行流式 AI 模型推理作业时遇到的诸多问题，本文提出了流式 AI 模型推理作业的自动优化系统。该系统的主要研究工作如下：

（1）针对流式 AI 模型推理数据预处理阶段缓存容量预估难的问题，本文提出了一种基于处理语义的缓存优化方法。具体来说，该方法使用一种基于近似数据结构的流状态分布高效感知方法，可以轻量高效的对流状态分布情况进行实时感知。同时，该方法使用一种基于处理语义信息的流状态缓存容量调整策略，并结合流状态分布情况，可以做到对流状态缓存容量进行准确预估。

（2）针对在线流式 AI 模型推理作业高效资源扩缩容难的问题，本文提出一种基于性能建模的资源扩缩容策略。具体来说，该策略采用一种轻量化多阶段建模的方式，对流式 AI 模型推理作业进行性能建模。同时，本策略也对资源扩缩容问题进行理论分析与建模，并使用一种基于动态规划的配置求解算法，结合作业性能模型，可以做到对作业进行高效、快速的资源扩缩容决策。

（3）针对离线流式 AI 模型推理作业资源最优化配置难的问题，本文针对成本最优与吞吐最优两类场景，分别提出了一种全局化的资源最优化方法。这两种方法可以根据作业整体的拓扑结构以及性能模型，自动化求解使得作业整体成本最低或性能最优的全局资源配置方案。

（4）基于以上研究，本文设计实现了一个流式 AI 模型推理作业自动优化系统。实验结果表明，在输入数据动态变化的场景下，对比使用静态缓存容量，本文的数据预处理缓存优化方法可以通过动态调整缓存容量，从而提升约 20%作业吞吐。同时，本文的资源扩缩容策略，对比 DS2[44]算法，可以在满足性能以及端到端时延的前提下节省约 25%的 GPU 资源开销。最后，对于离线流式 AI 模型推理作业的资源最优化方法，对比基于主流调优经验的自动化调参方法，本文方法可以降低约 30%的 GPU 资源并提高约 45%的作业性能。

1.5 本文的组织结构

本文由七个章节组成，各章节主要内容如下：

第一章，绪论：该章节主要阐述本文的研究背景，并介绍当前相关研究工作的特点与不足，接着分析本文研究工作所面临的问题与难点，并介绍了本文主要的研究工作，最后针对本文的组织结构进行了介绍。

第二章，背景知识：该章节针对与本文研究相关的概念与背景知识进行了介绍，包括分布式流式处理系统与流式 AI 模型推理作业。

第三章，基于处理语义的流式推理数据预处理缓存优化方法：该章节中首先对问题背景进行了介绍并对问题进行了分析，其次分别介绍了基于近似数据结构的流状态分布高效感知方法以及基于处理语义信息的流状态缓存容量调整策略。最后对本章节提出的缓存优化方法进行了实验评估。

第四章，面向在线流式 AI 模型推理的资源扩缩容策略：该章节首先分析问题提出的背景，接着介绍了轻量化多阶段的流式推理作业算子性能建模方法，然后介绍了基于算子性能模型的并行优化策略，最后对本文提出的资源扩缩容策略进行了实验评估与分析。

第五章，面向离线流式 AI 模型推理的资源最优化策略：该章节首先对问题背景进行了介绍与分析，其次分别针对成本最优与吞吐最优两类场景介绍了本文提出的资源配置方法，最后对本文资源配置方法进行了实验评估与分析

第六章，系统架构与实现：本章介绍了流式 AI 模型推理作业自动优化系统的整体架构，并对其中的关键组件与模块进行了详细介绍。

第七章，总结与展望：本章首先对本文的研究工作进行了总结，接着对下一步可能的工作开展方向进行了介绍。

第二章 背景知识

本章将介绍与本文提出的方法和系统相关的背景知识。2.1 小节将介绍分布式流式处理系统的相关概念与背景知识。2.2 小节将介绍流式 AI 模型推理作业的相关概念与背景知识。

2.1 分布式流式处理系统

对于用户给定的数据处理逻辑，流式处理作业通常会将其表示为一个有向无环图（Directed Acyclic Graph，DAG），例如图 2.1 展示了一个简单的流式处理作业对应的 DAG。其中图中的顶点表示用户特定的数据处理逻辑，也被称为**算子（Operator）**，本例中包含五个算子，分别表示读取数据的算子（即 Source）、对数据进行预处理的前处理算子、使用 AI 模型进行推理的算子、对推理结果进行处理的后处理算子以及将结果写入外部的算子（即 Sink）。DAG 中的有向边表示两个算子之间存在数据传输。对于这类仅表达作业处理整体逻辑的 DAG，被称为流式处理作业的逻辑视图。

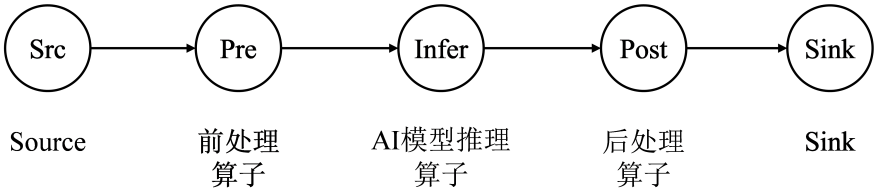


图 2.1 流式处理作业示意图

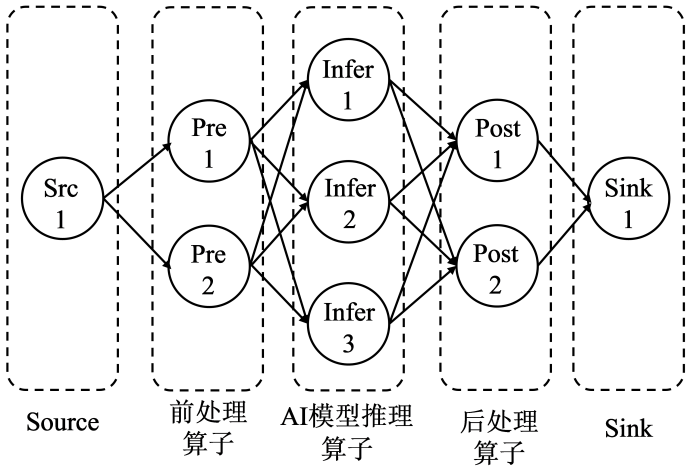


图 2.2 流式处理作业运行时状态示意图

在流处理作业实际运行的过程中，为提升作业的性能，会采用分布式的方式运行作业。分布式运行的方法为：针对每个算子，会使用多个实例并行执行。其中每个算子并行执行的实例数量被称为算子的**并行度（Parallelism）**。例如图 2.1 中所示的作业，在实际运行时的状态可能会如图 2.2 所示。其中，各个算子并行运行的实例数量分别为 1、2、3、2、1，对应它们的并行度也分别为 1、2、3、2、1。对于展示算子实际运行情况的 DAG，也被称为流式处理作业的物理视图。

同时，分布式流式处理作业在运行期间通常不能一次性获取所有的数据，因此对于一些计算逻辑，例如统计某个网页的点击次数，则需要使用额外的变量来保存中间结果，这些中间结果也被称为流式处理作业的**状态（State）**。

此外，随着云技术的发展，分布式流式处理系统的架构也发生了变化，其中最为显著的变化之一为存算分离架构的引入。存算分离架构指的是计算任务在需要写入存储设施时，不再写入本地的持久化存储，而是通过网络写入远端的统一存储系统，如 AWS S3[54]、阿里云 OSS[55]等。使用存算分离架构后，计算任务无需与某个特定的节点绑定，可以更方便的进行计算任务的调度，便于进行资源弹性扩缩容、任务恢复等操作。

对于包含状态的分布式流式处理作业，传统架构下通常储存在算子本地。导致了：（1）当作业中包含状态的算子需要扩缩容时，需要考虑复杂的状态同步机制；（2）当恢复作业时，需要花费大量时间在本地恢复算子状态。在引入存算分离架构后，算子的状态会被统一存储到远端存储，从而解决了上述问题。但使用存算分离架构后，由于读取写入状态时引入了额外的网络 IO 开销，也导致了作业性能的大幅下降。

2.2 流式 AI 模型推理作业

流式 AI 模型推理作业与一般的流式处理作业相同，也使用有向无环图（Directed Acyclic Graph，DAG）来表示作业，且在执行时也会使用多个实例并行执行来增加作业性能。

但由于流式 AI 模型推理作业会包含使用 AI 模型进行推理的过程，对比传统流式处理作业，其通常会使用 GPU 来进行推理加速。GPU 是一种专为大规模数据并行计算设计的计算加速器，其架构基于单指令多线程（Single Instruction, MultipleTthreads, SIMT）执行模型。GPU 核心架构由多个流多处理器（Streaming Multiprocessor, SM）构成，其中每个 SM 内又包含若干个算术逻辑单元。

对于一个计算任务，GPU 将其划分为多个线程块（Thread Block），在执行时会将线程块分配给多个 SM。在 SM 内，线程块又会被进一步划分为多个线程束（Warp），并最终调度到计算核心上进行并行执行。因此，对于一个计算任务，若其涉及到的计算量过少，可能无法充分发挥 GPU 的计算能力。简单举例说明，若一个计算任务仅能使用 GPU 中 40%的计算核心进行并行运算，当计算量增加一倍时，由于 GPU 中还包含多余的计算核心可以用于计算，则可以使用 80%的计算核心来并行计算，从而将任务的吞吐提高一倍。

上述例子仅用于说明原理，在实际使用过程中的情况会更加复杂。对于 AI 模型推理来说，可以调整一次推理输入的数据条数来达到提升吞吐的效果，其中一次 AI 模型推理同时输入的数据条数也被称为 **Batch Size**。举例说明，对于传统流式处理作业，当数据到达时即可开始处理，但当考虑 Batch Size 后，例如 AI 模型推理算子所需的 Batch Size 为 1024，若数据输入速率为 100 条/秒，则该 AI 模型推理算子在数据到达时不能立即处理，而是需要等待约 10 秒，积攒到 1024 条数据后才可以开始进行推理。

同样，与传统流式处理作业类似，存在一类流式 AI 模型推理作业同样需要长时间运行，本文称其为**在线流式 AI 模型推理作业**。但相较于传统流式处作业，由于 AI 模型推理的计算开销较大、耗时较长，用户对于流式 AI 模型推理作业的实时性要求（秒级别到分钟级别）往往低于传统流式处理作业（毫秒级别）。对于在线场景下的流式 AI 模型推理作业，由于放宽了延迟要求，则可以通过适当调整 Batch Size 来提高吞吐，从而降低 GPU 资源的使用量。

此外，流式 AI 模型推理作业并非仅能用于在线场景，也可以被用于离线场景，即待处理的数据集为有限数据集，作业无需长时间运行。这是由于流式 AI 模型推理作业需要同时使用 CPU 与 GPU 两种异构硬件，流式执行的方式可以将 CPU 的计算过程与 GPU 的计算过程以流水线的方式串联执行，即前序的 CPU 计算过程处理完部分数据后即可开始后续的 GPU 计算过程，从而大大减少 GPU 的等待时间，提高 GPU 的利用率以及作业的性能。本文称这一类无需长时间运行的作业为**离线流式 AI 模型推理作业**。

第三章 云平台环境下流式推理数据预处理缓存优化

在流式推理的数据处理链路中往往包含数据预处理步骤（如统计某网页点击次数等），在当前主流的云平台存算分离架构下，通常会使用缓存对需要使用状态的预处理算子进行性能优化。然而，在流式推理作业长时间运行期间，输入数据的特性并非不变，也会导致所需缓存大小的变化，使用过大或过小的缓存会带来资源浪费、性能下降等问题。为解决上述问题，本章提出了一种基于处理语义的流式推理数据预处理缓存优化方法。3.1 小节中介绍了问题提出的背景并对问题进行了分析，3.2 小节中介绍了基于近似数据结构的流状态分布高效感知方法，3.3 小节介绍了基于处理语义信息的流状态缓存容量调整策略，3.4 小节对本章提出的方法和策略进行了实验评估与分析。

3.1 问题背景与分析

随着云平台环境的普及，当前越来越多的应用或引擎选择适配云平台环境，例如 Flink[56]、Ray[27]等引擎都针对 Kubernetes[57]推出了相应的 Kubernetes Operator[58,59]。云平台环境的一个特点为存算分离架构，即将任务的计算与存储进行解耦，某个任务不必由于数据存储强制绑定到某个节点，而是可以调度到任意节点并通过网络访问远端的存储。

在流式处理中，由于面向无界数据流，某些算子需要存储中间结果并用于后续的计算，这些中间结果即状态。在流式推理的数据处理链路中，也需要对数据进行预处理，也可能会涉及到有状态算子，例如原始数据为用户点击网页的日志，需要提取用户点击次数作为 AI 模型推理的特征之一，则需要存储用户当前总点击次数这一状态。

流式处理引擎当前也在向存算分离架构转变，例如 Flink 2.0 版本推出了使用远端存储的状态管理机制[60]。然而使用远端状态存储后，每次数据处理访问状态都需要进行网络 IO，会带来较大的性能下降。如图 3.1 展示了使用本地状态存储的作业吞吐以及使用 HDFS 作为远端状态存储的作业吞吐，分析可知，对比使用本地状态存储，使用 HDFS 作为远端状态存储导致作业吞吐下降了 85%。

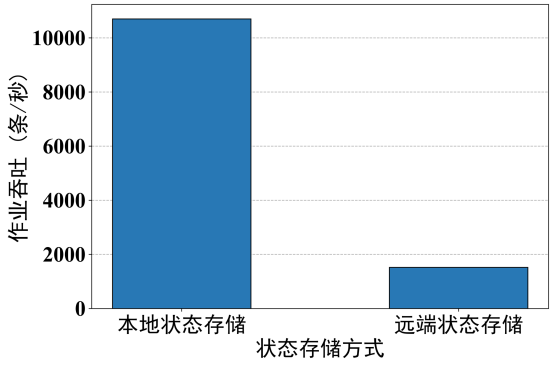


图 3.1 本地状态存储与远端状态存储作业吞吐对比

对于上述问题，当前常见的优化方法为使用缓存进行加速。然而，由于流处理作业的长时间运行特性，会带来如下特点：（1）短时间内的数据通常具有局部性；（2）长时间内的数据特性通常会发生变化。上述两个特点会导致：一方面并不需要缓存所有数据，另一方面在长时间运行的过程中，需要缓存的数据量会发生变化。如果使用静态缓存大小，若缓存空间过大，则会导致内存资源浪费，若缓存空间过小，则会导致性能下降。

云平台环境下，可以方便的进行资源的弹性扩缩容，这就允许了状态缓存随着数据特性的变化进行动态调整。然而在动态调整的过程中，会面临如下问题：

(1) 如何感知流状态的分布情况？

在作业长时间运行期间内，输入数据在短期内会具有局部性，但在长期内这种局部性会发生变化。例如，某一批用户经常在上午访问网站，而另一批用户经常在晚上访问，如果使用流处理作业处理用户的访问日志，则仅需要缓存一部分用户对应的流状态即可，然而在不同时间段内，需要缓存的用户流状态分布情况却是不同的。

这就要求在作业运行过程中能够感知流状态的分布情况，从而做出状态缓存调整的决策。一种简单的方法为记录所有出现过的数据，并根据全量数据分析分布情况。然而该方案会带来两方面问题：（1）存储全量数据会带来过高的内存开销；（2）根据全量数据进行统计的操作耗时较长。在感知流状态分布时，一方面要避免占用过多内存或计算资源，导致影响作业正常运行，另一方面当分布发生变化时，需要能够较快做出响应。这就要求流状态感知方法需要轻量且高效。

(2) 如何调整流状态的缓存容量？

在动态调整的过程中，同样需要决定作业使用的状态缓存容量。图 3.2 展示了聚合算子在不同数据分布情况下（图中使用了齐普夫分布，参数分别取 0.8 与 1.1），使用不同缓存大小时算子的吞吐变化情况。可以看到，一方面在缓存大小

超过约 100MB 时，吞吐不在变化，表明使用过大的缓存会带来资源浪费，另一方面，对于不同的数据分布情况，达到同一吞吐时所需要的缓存容量并不相同，表明当数据分布情况发生变化时，需要一种方法来计算所需的缓存容量大小。

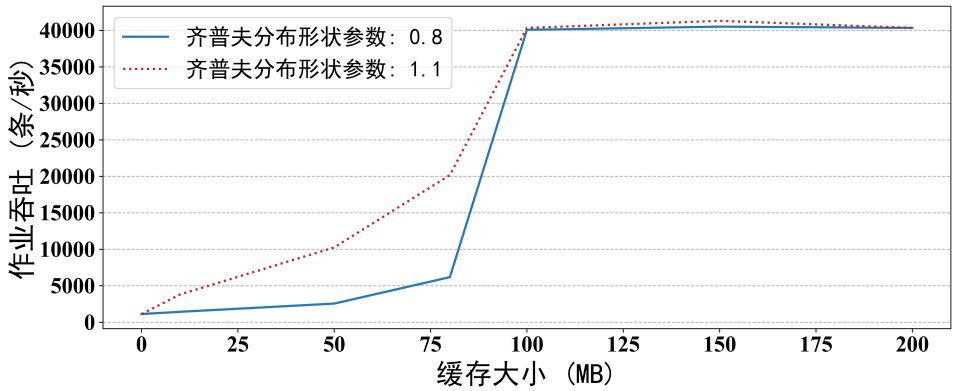


图 3.2 不同数据分布情况下作业吞吐随缓存大小的变化情况

针对上述问题，本章提出了一种云平台环境下流式推理数据预处理缓存优化方法。具体来说，为解决流状态的分布感知问题，3.2 小节提出了一种基于近似数据结构的流状态分布高效感知方法，为解决流状态缓存的容量确定问题，3.3 小节提出了一种基于处理语义信息的流状态缓存容量调整策略。

3.2 基于近似数据结构的流状态分布高效感知方法

为了能够感知流状态的分布情况，同时避免占用过多内存与计算资源，本小节提出一种基于近似数据结构的流状态分布高效感知方法。由于基于近似数据结构，本方法无需占用过多资源，达到了轻量、高效的效果。下文将详细介绍该方法，3.2.1 小节将对问题进行定义与描述，3.2.2 小节将介绍一种基于近似数据结构的频率感知方法。

3.2.1 问题定义与描述

流状态在存储时会根据数据的键值进行存储，对于同一个键值，则认为是同一个流状态。在动态调整流状态缓存的过程中，需要能够动态感知流状态的分布情况，这里的流状态分布指的即为状态键值的分布情况。

感知流状态分布情况的目标并非探测分布的具体函数，例如正态分布等，而是在后续调整缓存容量过程中，根据捕获到的分布情况预测缓存命中率。为达到

这一目标，本文采用的方法为根据数据出现的频率预测。例如共有五个不同的键值 Key1、Key2、Key3、Key4、Key5，若共有 1000 条数据，其中这五个键值出现的次数分别为 500 次、300 次、100 次、50 次、50 次，则若希望达到命中率为 80%，则根据频率统计，需要缓存 Key1 与 Key2 两个键值对应的状态，因为其出现频率之和为$\frac{500}{1000}+\frac{300}{1000}=80\%$。

因此，流状态分布感知方法的目标为处理一定数据后，能够根据给定的命中率目标，依据频率给出需要缓存不同键值的数量。具体来说，对于给定的数据集D，数据集大小为D_s，其中出现的不同键值的集合为$K_s=\{k_1,\dots,k_m\}$，每种键值出现的次数为$K_c=\{k_1^c,\dots,k_m^c\}$，则对于给定的命中率目标$h$，要求流状态分布感知方法给出需要缓存的键值数量$n$满足：

$$\min n \tag{3.1}$$

$$s.t.\ \exists\ R \subseteq K_s \tag{3.2}$$

$$|R| = n \tag{3.3}$$

$$\sum_{k_i \in R} \frac{k_i^c}{D_s} \geq h \tag{3.4}$$

即需要找到一个最小的键值集合R，该集合大小为n（约束 3.3）且该集合为K_s的子集（约束 3.2），使得集合中的键值出现频率之和大于等于用户要求的命中率（约束 3.4）。

3.2.2 基于近似数据结构的频率感知方法

为解决 3.2.1 小节中提出的问题，一种方案为记录每个键值的出现次数以及当前的总数据条数。当需要对问题进行求解时，对每个键值对出现次数进行排序，并从大到小计算出现频率之和直到超过要求的命中率即可。但这种方案由于需要存储所有出现过的键值，会带来高额的内存资源开销。同时排序的过程也会消耗较多的计算资源，会影响作业的正常运行。

由于分布感知方法是利用过去一段时间内的数据分布情况来预测未来的情况，本身具有一定的不准确性，则在感知流状态分布时，无需要求完全准确，因此可以牺牲一定的准确度来换取方法的轻量性与高效性。本小节中提出一种基于近似数据结构的频率感知方法。

该数据结构采用了类似直方图的思想。直方图将数据分为一定数量的组数，并统计每组数据出现的频率总和。可以用于观察数据的分布情况。由于并不需要保存所有出现过的键值，而是通过分组的方式，可以有效控制占用内存的大小。但直方图并不能直接应用于本小节涉及到的问题，这是由于：（1）直方图丢失了每个分组中不同键值数量的信息；（2）对于频率相同的直方图分组，其含义可能并不相同。针对问题 2，举例说明：若直方图的两个分组对应的频率相同，但其中一个分组中仅包含 2 个不同的键值，而另一个分组中包含 10 个不同的键值，则在求解时应该使用前者。

针对上述两个问题，本小节提出的方法采用如下方案来进行优化：

(1) 对于每个分组，使用额外的变量记录其包含的不同键值数量。在判断某个键值是否出现过时，同样需要采用轻量高效的方案。本文使用了布隆过滤器这一近似数据结构来判断某个键值是否出现过。

(2) 对于每个分组，若总出现次数为T，不同的键值数量为C，则定义该组的分值$S = T/C$，通过该分值来评价各个分组的贡献。其中，出现次数越多且键值数量越少，表明该组中的键值实际出现频率越高。

综上，本小节定义了一种近似数据结构$FrequencyCapture$，该数据结构同样将数据分为多个分组，其包含如下属性：分组数G_n、分组数组G，其中的元素为该分组数据出现的次数以及不同键值的数量、布隆过滤器$filter$以及当前的数据总数N。该数据结构包含的方法如下：

算法 3.1 $FrequencyCapture$的初始化方法

输入：待初始化的$FrequencyCapture$对象F，分组数量G_n
输出：初始化后的$FrequencyCapture$对象F

```
1. func init(F, G_n) {
2.     F.G_n = G_n
3.     F.G = Initialize array with size G_n and element (0,0)
4.     F.filter = Initialize a bloom filter
5.     F.N = 0
6. }
```

初始化方法中会为对象中的属性进行赋值，包括用于后续统计的数组G（第 3 行）以及用于判断数据是否出现过的布隆过滤器$filter$（第 4 行）。

算法 3.2 $FrequencyCapture$的添加数据方法

输入：初始化后的$FrequencyCapture$对象F，待添加数据的键值k

```
1. func add(F, k) {
2.     F.N = F.N + 1
```

```
3.      index = hash(k) % F.G_n
4.      F.G_n[index][0] += 1
5.      if k not in F.filter then
6.          F.filter.add(k)
7.          F.G_n[index][1] += 1
8. }
```

在向$FrequencyCapture$中添加数据时，首先会将数据总数自增（第 2 行）。由于数据的键值k的数据类型并不固定，所以需要通过哈希值来计算其所属的分组（第 3 行）。在确定分组后，需要将该分组的数据数量自增（第 4 行），同时根据布隆过滤器来判断是否出现过该数据，若没有出现过，则需要增加对应分组中不同键值的数量（第 5-7 行）。

算法 3.3 $FrequencyCapture$的求解方法

输入：初始化后的$FrequencyCapture$对象F，目标命中率h
输出：满足目标命中率的键值数量c

```
1.  func calculate(F, h) {
2.      G_sorted = sort F.G in descending order according to g[0]/g[1] for every g in F.G
3.      s = 0
4.      c = 0
5.      target = ⌈F.N · h⌉
6.      for g in G_sorted do
7.          s += g[0]
8.          c += g[1]
9.          if s ≥ target then
10.             break
11.     return c
12. }
```

在求解满足目标命中率的所需的键值数量时，会首先对分组数据G按照前文提到的分值进行降序排序（第 2 行）。对于目标命中率h，其要求的出现次数为$F.N \cdot h$（第 5 行），即需要找出出现次数超过该值的键值数量，寻找方法为对排序后的分组数组进行遍历并累加数据出现次数，直至超过目标值，此时键值数量即为求解结果（第 6-11 行）。

3.3 基于处理语义信息的流状态缓存容量调整策略

当输入数据的分布发生变化时，所需的流状态缓存容量也需要变化，过大或过小的缓存容量会导致资源浪费或者性能下降。为解决该问题，本小节提出了一

种基于处理语义信息的流状态缓存容量调整策略。3.3.1 小节对容量调整策略进行了概述，3.3.2 小节介绍了基于处理语义信息的流状态缓存容量模型。

3.3.1 容量调整策略概述

在流式处理作业长时间运行期间，由于输入数据特性会随着时间变化，为了避免流状态缓存资源浪费或者作业性能下降，需要对流状态缓存容量进行动态调整。针对这一问题，本文提出了一种基于处理语义信息的流状态缓存容量调整策略。该策略主要针对如下两个问题：

(1) 如何确定流状态缓存容量调整时机？

当输入数据的分布发生变化时，对应的流状态缓存分布也会发生变化，这会导致流状态缓存容量也需要调整。例如过去一段时间内出现频率超过 90%的数据键值为 Key1 与 Key2，则缓存容量为缓存 Key1 与 Key2 对应的状态所需的大小，当数据分布发生变化时，例如出现频率超过 90%的数据键值变为 Key1、Key2、Key3，若希望维持作业性能，则需要将缓存容量变为缓存 Key1、Key2 与 Key3 对应的状态所需的大小。

为了感知流状态分布的变化，3.2 小节提出了一种基于近似数据结构的流状态分布高效感知方法。此外，由于流式处理作业通常无法一次性获取全部数据，则在感知流状态分布的过程中，本文采用了基于时间窗口的方法，即在固定的时间窗口W内采集流状态的分布情况，并作为流状态的整体分布情况。

3.2 小节中提出的感知方法会根据给定的命中率给出所需的键值数量，本文将其视为流状态的分布情况。当在某个时间窗口内感知到的该值变化幅度超过某个阈值后，则认为需要对流状态缓存容量做出调整。

(2) 如何确定当前流状态缓存所需容量？

当决策判断出需要对流状态缓存容量做出调整后，仍需要解决当前流状态缓存所需容量的问题。会对缓存容量产生影响的因素包括：（1）用户对作业的性能要求；（2）流状态的分布情况；（3）作业如何使用流状态。其中，对于因素 1，例如当用户对作业的性能要求提高后，可能导致需要缓存更多的流状态，进而导致流状态容量的增加，然而用户对于作业的性能要求往往是确定的，可以认为是已知量。对于因素 2，在上文中已经进行过举例说明，同样可以使用 3.2 小节提出的方法来对分布情况进行感知。

对于因素 3，下面举例说明作业使用流状态的方式不同如何影响缓存容量。若作业 A 为聚合不同用户对某广告的点击次数，则该作业使用流状态存储用户 ID 以及对应的点击次数这一信息。作业 B 为统计五分钟内用户某行为统计指标的中位数，则该作业使用流状态存储过去五分钟内用户的行为日志，用于每五分钟触发进行一次统计。作业 A 与作业 B 的用户的分布情况可能相同，但由于其使用流状态的方式不同：作业 A 的流状态包括用户 ID 与点击次数，而作业 B 的流状态包括用户 ID 以及完整的行为日志，导致流状态的大小也不相同。

对于影响因素 3，与数据仓库或数据库的即席查询每次的查询逻辑不固定不同，流式处理作业的处理逻辑通常较为固定且作业会存在较长时间，可以在用户提交作业时进行获取，并可以在后续作业长时间运行的过程中使用。本文将作业的处理逻辑称为作业的处理语义信息，基于作业的处理语义信息，可以构建流状态的缓存容量模型，从而计算所需的缓存容量大小，3.3.2 小节中将对该模型进行详细阐述。综上，本策略的总体算法流程如下：

算法 3.4 容量调整策略算法

输入：用户对作业吞吐的要求 T，基于用户作业处理语义信息的缓存容量模型 M，时间窗口的大小 W，决策调整缓存容量时分布的变化阈值 t

1. $d = 0$
2. Initialize state cache size *S*
3. Initialize FrequencyCapture *F*
4. $time_{start} =$ current time
5. **while** job is running **then**
6. $time_{current} =$ current time
7. e is the data to be processed
8. add(*F, e.k*)
9. **if** $time_{current} - time_{start} < W$ **then**
10. **continue**
11. $time_{start} = time_{current}$
12. $h =$ calculate hit ratio according to *M* and *T*
13. $d_{new} =$ calculate(*F*, *h*)
14. **if** $|d_{new} - d|/d \leq t$ **then**
15. **continue**
16. $d = d_{new}$
17. $F =$ initialize a new FrequencyCapture
18. $S =$ calculate state cache size according to *M* and *d*
19. apply new stat cache size *S*

首先，本策略在运行前会进行一系列初始化操作，包括初始的流状态分布情况、初始流状态缓存大小、当前时间窗口的起始时间以及 3.2 小节中介绍的用于

流状态分布感知的数据结构 FrequencyCapture（第 1 至 4 行）。接着在作业运行期间，每接受到一个输入数据后，首先会将其加入到流状态分布感知数据结构中（第 8 行），并判断当前时间窗口是否超过了给定的时间窗口大小W，若没有超过，则继续处理（第 9 至 10 行），若超过，则探测当前的流状态分布情况并判断是否超过阈值（第 12 至 15 行），其中第 12 行涉及到利用基于处理语义的缓存容量模型来计算目标命中率，该过程将在第 3.3.2 小节中详细介绍。当流状态分布变化情况超过给定阈值，则利用缓存容量模型重新计算缓存大小（第 18 行），同样该过程将在第 3.3.2 小节中详细介绍。

3.3.2 基于处理语义信息的流状态缓存容量模型

本小节介绍基于处理语义信息的流状态缓存容量模型。在 3.3.1 小节中提到流式处理作业使用流状态的方式不同也会导致缓存容量的不同，但流式处理作业在长时间运行期间处理逻辑通常固定不变。本小节提出的缓存容量模型可以利用作业的处理逻辑，也即作业的处理语义信息来进行建模。

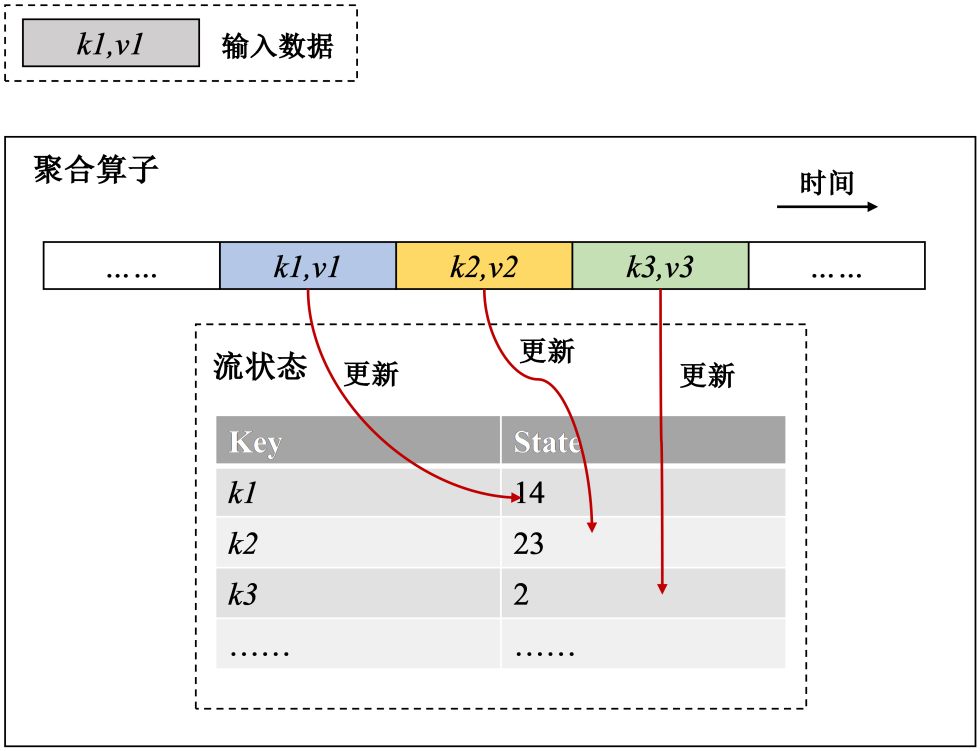


图 3.3 聚合算子处理逻辑示意图

具体来说，本小节针对常用的聚合算子进行了流状态缓存容量建模。聚合算子的处理逻辑如图 3.3 所示，当数据到达时，聚合算子会根据数据的键值访问对应的流状态，在图例中数据的键值为 *k1*、*k2* 与 *k3*，对应的流状态为该键值出现的次数。在取得对应的流状态后，会根据数据的内容对流状态进行更新，在图例中则是对出现次数进行加一。

因此，根据聚合算子使用流状态的方式，可以建模计算得出聚合算子的吞吐。若聚合算子使用了状态缓存来加速流状态的存取，其中命中缓存时，处理一条数据的时间为t_h，未命中缓存时，处理一条数据的时间为t_m。若已处理了a条数据，其中命中缓存的比例为h，则聚合算子的吞吐T为：

$$T = \frac{a}{a \cdot h \cdot t_h + a \cdot (1-h) \cdot t_m} = \frac{1}{h \cdot t_h + (1-h) \cdot t_m} \tag{3.5}$$

其中，算子在命中与未命中时处理一条数据的时间认为是已知的且$t_m > t_h$，则在用户给定目标吞吐率T_{target} $(T_{target} > 0)$ 后，可以计算出需要达到的命中率h_{target}为：

$$h_{target} = \frac{t_m - \frac{1}{T_{target}}}{t_m - t_h} \tag{3.6}$$

计算得到目标命中率后，可以利用 3.2 小节中介绍的流状态分布感知方法获取需要的键值数量d，若数据的键值大小为S_{key}，对应的流状态大小为S_{state}，则所需的状态缓存大小S为：

$$S = d \cdot \left(S_{key} + S_{state}\right) \tag{3.7}$$

综上，对于聚合算子，可以在作业运行前预先收集其处理语义信息，包括待处理数据键值的大小、对应流状态的大小以及命中缓存与未命中缓存时处理一条数据的时间，从而根据上述公式构建基于处理语义信息的流状态缓存容量模型M。在流状态缓存容量调整策略运行期间，可以利用该缓存容量模型M来计算调整时所需要的信息。在 3.3.1 小节中介绍了容量调整策略运行的整体流程，在策略运行期间，需要根据用户给定目标吞吐来计算目标命中率，则此时可以根据公式 3.6 进行计算，当策略获取到流状态分布情况后，则可以根据公式 3.7 来计算所需流状态缓存容量。

3.4 实验评估与分析

本小节将对本章提出的云平台环境下流式推理数据预处理缓存优化方法进行实验评估。首先，3.4.1 小节将介绍实验环境与配置，包括软硬件环境、工作负责以及评估指标。其次，3.4.2 小节将对本章提出的基于近似数据结构的流状态分布感知方法进行有效性评估。最后，3.4.3 小节将对本章提出的基于处理语义信息的流状态缓存容量调整策略进行评估。

3.4.1 实验环境与配置

3.4.1.1 软硬件环境

在本节实验过程中，使用了一个由 3 个物理节点组成的集群。硬件方面，每个物理节点都配置了两个 Intel(R) Xeon(R) Silver 4314 CPU @ 2.40GHz，该 CPU 具有 16 个物理核心和 64 个逻辑核心。同时，每个节点配有 256GB 内存，且节点之间使用 10Gbps 的以太网进行连接。

软件方面，每个物理节点使用了 CentOS 8.5.2111 版本的操作系统，且 Linux 内核版本为 v4.18.0，编程语言方面则使用了 Java 8，其对应的 JDK 版本为 1.8.0_202。同时，由于云平台存算分离架构通常需要一套统一的分布式远端存储系统，在本实验中则使用了 HDFS 作为该存储系统，其版本为 3.4.0。同时，对于流处理引擎，则使用了 Flink 存算分离版本，该版本基于 Flink 1.19 版本进行了存算分离改造，并使用了 ForSt[61]作为状态存储管理系统，该存储管理系统会使用 HDFS 作为状态的远端存储。

3.4.1.2 工作负载

工作负载分为两方面介绍：

(1) 实验数据方面，实验中使用了随机生成的数据，数据生成的过程符合齐普夫分布（Zipf Distribution），在生成过程中，通过控制齐普夫分布的形状参数来调整数据的分布情况。同时，实验中使用了一个嵌入到作业源算子的齐普夫数据生成器，可以通过给定形状参数来模拟生成不同分布情况的数据，同时该生成器还可以模拟形状参数动态变化的场景。

(2) 实验作业方面，模拟了实时统计用户点击某网页次数的场景。该作业的两个关键算子为源算子与聚合算子。源算子使用了上文提到的齐普夫数

据生成器来生成不同用户 ID 的访问记录，聚合算子则统计不同用户 ID 的访问次数，并进行累加计算当前的总访问次数并输出。

3.4.1.3 评估指标

为了对本章提出的云平台环境下流式推理数据预处理缓存优化方法进行评估，本文主要采用了以下评估指标：

(1) 缓存命中率：该指标表示处理一批数据后，命中缓存数据的比例。用于评估流状态分布感知方法给出的缓存键值数量能否满足要求的缓存命中率，反映了流状态分布感知方法的有效性。

(2) 缓存键值数量：该指标表示缓存中最多能保留的键值数量，每个键值都对应一个流状态，可以反映流状态缓存的容量。

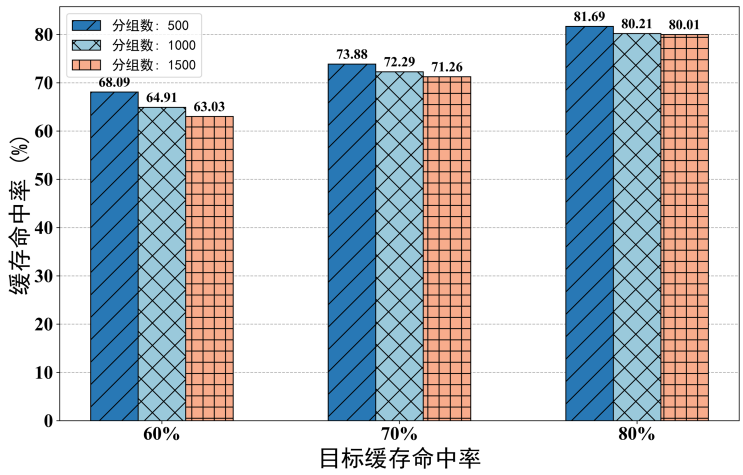
(3) 作业吞吐：该指标表示作业每秒处理的数据条数，用于评估流状态缓存容量对作业性能的影响。

此外，在实验过程中还对本文提出的方法进行了 Overhead 评估，主要评估指标为运行策略带来的额外时间以及内存开销。

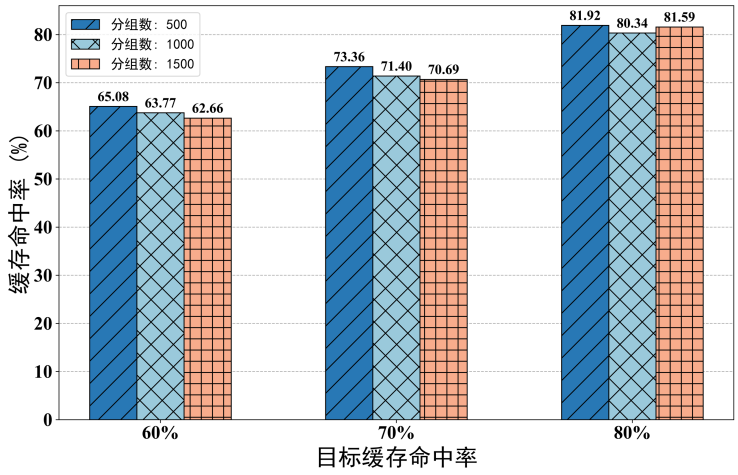
3.4.2 流状态分布感知方法有效性评估

本小节对流状态分布感知方法的有效性进行评估，评估内容分为两部分：（1）在给定不同目标缓存命中率下，本章提出的流状态分布感知方法给出的缓存键值数量能否满足目标缓存命中率？（2）在给定目标缓存命中率后，本章提出的流状态分布感知方法给出的缓存键值数量，与实际满足目标缓存命中率的最小缓存键值数量之间的差距是多少？

首先，针对第一部分，实验过程中分别使用齐普夫分布形状参数为 1.1 与 1.05 来生成随机数据，生成过程中数据键值的取值空间为 10000。同时，本章提出的流状态分布感知方法可以指定的分组数量，分组数越多，计算开销以及内存开销相应会越大，但精度会更高，在实验过程中，分组数分别取 500、1000 以及 1500。对于给定的目标缓存命中率，在实验过程中分别取 60%、70%以及 80%。



(a) 测试数据齐普夫分布形状参数为 1.1



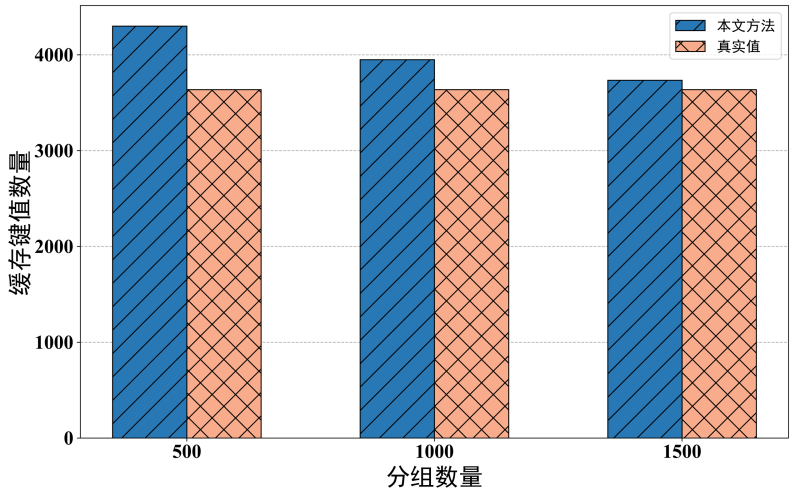
(b) 测试数据齐普夫分布形状参数为 1.05

图 3.4 不同目标缓存命中率下分布感知方法所能达到的实际缓存命中率

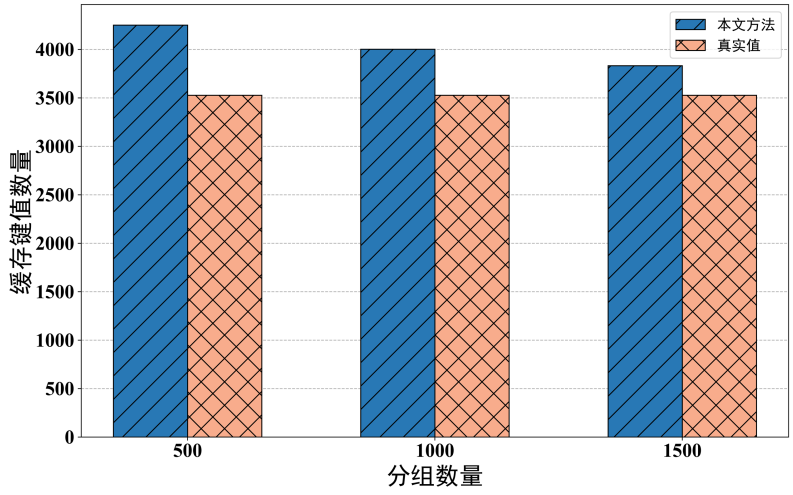
实验结果图 3.4 所示，首先对于不同目标缓存命中率，使用本章提出的流状态分布感知方法给出的缓存键值数量后，实际缓存命中率均能达到目标缓存命中率要求。表明本方法在不同数据分布情况下，均可以使用较小的额外数据（分组数为 500 至 1500），来有效感知数据的分布情况，并给出能满足目标缓存命中率的缓存键值数量。

同时，不同分组数量下，实际达到的缓存命中率基本呈现分组数 500＞ 分组数 1000＞ 分组数 1500 的趋势。这是由于使用更少的分组数时，会增大数据键值哈希碰撞的概率，导致每个分组中的键值数量增加，进而导致最终结果偏大。这也表明在实际使用本方法时，可以通过调整分组数来控制决策的缓存键值数量。例如，若用户希望优先保证缓存命中率，可以浪费一定缓存容量，则可以使用较小的分组数。

针对第二部分，分别在齐普夫分布形状参数为 1.1、目标缓存命中率为 70%以及形状参数为 1.05、目标缓存命中率为 60%两种实验配置下进行了实验。在随机生成实验数据时，同样控制键值的取值空间为 10000。此外，实验过程中使用了 500、1000、1500 三种分组数量，并且对比了本文方法给出的缓存键值数量，以及通过尝试得出的满足给定目标缓存命中率所需的最小缓存键值数量（在本文中称为真实值）。



(a) 齐普夫形状参数为 1.1、目标缓存命中率为 70%



(b) 齐普夫形状参数为 1.05、目标缓存命中率为 60%

图 3.5 不同实验配置下本文方法缓存键值数量与真实值对比

实验结果如图 3.5 所示，分析可知本文提出的流状态分布感知方法给出的缓存键值数量均会大于实际值，这是由于：对于本文方法，每个分组中可能存在实际频率较低，但由于哈希碰撞被分组到同一组的键值，导致更倾向于选择缓存更

多的键值。其中，在图 3.5 (a) 和 (b) 中，本文方法给出的缓存键值数量最多超出真实值 20.5%。

此外，分析图 3.5 (a) 和 (b) 可知，随着分组数量的增加，本文方法给出的缓存键值数量与真实值的差距会逐渐缩小，表明本文方法在更大的分组数量下会给出更加准确的缓存键值数量。其中，本文方法给出的缓存键值数量最小超出真实值 2.8%。

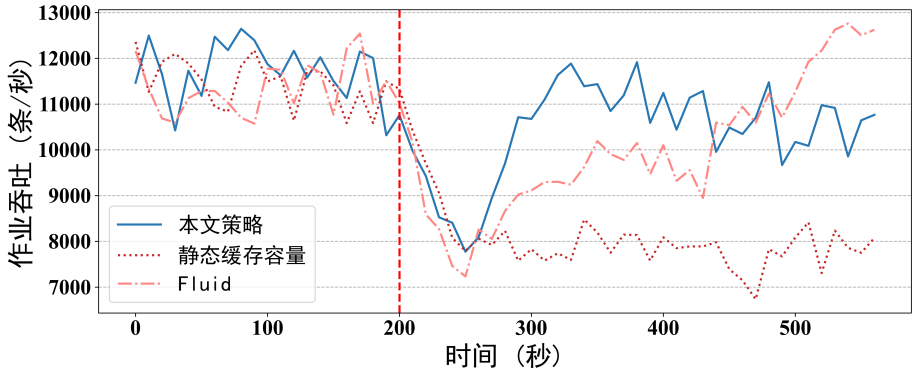
3.4.3 流状态缓存容量调整策略评估

本小节对本文提出的流状态缓存容量调整策略进行评估。评估内容分为两部分：（1）在数据分布动态变化时，本文策略能否有效调整流状态缓存容量（3.4.3.1 小节）？（2）本文方法对作业正常运行带来的 Overhead 有多少（3.4.3.2 小节）？

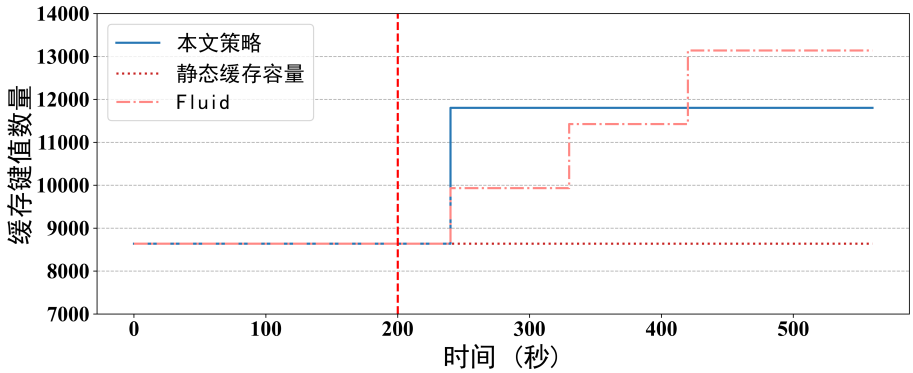
3.4.3.1 端到端实验评估

本小节对本文策略进行端到端实验评估，其中评估使用的作业为 3.4.1.2 小节中提到的模拟统计用户访问次数作业，性能目标设定为吞吐达到 10000 条/秒，数据方面使用了模拟生成的符合齐普夫分布的数据，生成时键值的取值空间为 20000，且分布的形状参数会在作业运行的第 200 秒从 1.1 动态变化至 1，从而模拟输入数据分布动态变化的场景。

实验对比了本文策略、Fluid 缓存扩缩容策略[72]（图中称为 Fluid）以及静态缓存容量。其中 Fluid 缓存扩缩容策略是一种基于规则的策略，其会监测过去一个时间窗口的数据处理速率，并按照一定比例进行缓存扩缩容。实验过程中本文策略使用的分组数量为 500、时间窗口为 30 秒，初始缓存容量大小由本文策略进行决策，并配置给本文策略以及静态缓存容量。Fluid 方面则将每次扩缩容的比例设置为 15%，且时间窗口同样设置为 30 秒。



(a) 使用本文策略与静态缓存时作业吞吐随时间的变化情况



(b) 缓存键值数量随时间变化情况

图 3.6 作业吞吐以及缓存键值数量随时间变化情况

图 3.6 展示了实验过程中作业吞吐以及缓存键值数量随时间的变化情况。首先，在作业运行的初始阶段，本文策略决策出的缓存键值数量为 8639，此时本文策略以及静态缓存容量均能将作业吞吐维持在 10000 条/秒，表明了本文策略决策的缓存容量能够满足目标吞吐要求。

随后，在第 200 秒时（即图中红色虚线指示位置），作业的数据分布情况发生变动，此时本文策略以及静态缓存容量的作用吞吐均出现了下降，这是由于数据分布发生了变动，导致原有的流状态缓存已经不能满足达到目标吞吐所需的命中率，进而导致作用吞吐的下降。但在第 240 秒左右，本文策略进行了一次决策，动态将缓存键值数量扩充到 11805，而静态缓存容量则维持不变。此后，本文策略的作业吞吐开始逐渐上升，并重新维持在 10000 条/秒左右，表明本文新决策的缓存容量同样能够达到目标吞吐要求，而静态缓存容量的作业吞吐则维持在 8000 条/秒左右，表明不进行缓存容量的动态调整导致了约 20%的性能下降。

Fluid 缓存扩缩容策略在数据分布变化后，同样进行了一次缓存扩容，即将缓存容量上调 15%，但此时缓存容量不足以达到目标吞吐，因此一段时间后 Fluid 缓存扩缩容策略继续上调了一次缓存容量。最终为了稳定达到目标吞吐，Fluid 缓存扩缩容策略共调整了 3 次，且最后一次调整由于同样按照固定比例调整，导致了缓存容量浪费，即此时作业的吞吐已经超过所需的目标吞吐。对比 Fluid 基于规则的缓存扩缩容策略，本文方法可以结合作业处理语义信息，进行准确的扩缩容，从而大大减少调整所需时间并且降低了缓存浪费。

3.4.3.2 Overhead 评估

本文提出的流状态缓存容量调整策略会嵌入到用户作业中，与用户作业一同运行，由此会对用户作业带来额外开销。本小节将对引入的开销进行评估。

具体而言，本文策略要求用户作业在运行时，每接受到一条数据都要将其键值加入到本文提出的流状态分布感知方法中，且每隔一段时间会进行一次决策计算，由此引入了运行时间开销。同时，由于本文方法需要存储键值数量以及使用布隆过滤器等原因，也引入了额外的内存开销，本小节将对本文策略带来的时间以及内存开销进行评估。评估过程中，本文方法的分组数量取值为 1000。表 3.1 展示了添加数据、决策计算的运行时间开销以及内存开销。

表 3.1 流状态缓存容量调整策略的 Overhead

资源类型	Overhead
时间	添加数据：0.002ms
	决策计算：0.21ms
内存	47KB

分析表 3.1 可知，本文提出的流状态缓存容量调整策略对作业计算耗时影响极为有限，同时内存占用仅为 47KB，引入较小的 Overhead。

3.5 本章小结

流式推理数据预处理过程可能会涉及到使用流状态，在云平台存算分离环境下，通常会使用流状态缓存进行性能优化。然而在流式推理数据预处理作业运行过程中，输入数据的特性可能会随着时间变化，进而导致对流状态缓存容量需求的变化，需要一种方法对流状态缓存进行优化。针对上述问题，本章介绍了一种流状态缓存优化方法。首先，本章介绍了一种基于近似数据结构的流状态分布高效感知方法，其可以对流状态的分布情况进行高效感知。其次，本章提出了一种基于处理语义信息的流状态缓存容量调整策略，该策略可以及时感知流状态分布的变化，并基于作业的处理语义信息对流状态缓存容量进行调整。最后，本章对流状态分布感知方法以及缓存容量调整策略进行了实验评估与分析，验证了本章提出的方法可以准确、高效感知流状态分布情况并调整缓存容量以提升作业性能。

第四章 面向在线流式 AI 模型推理的资源扩缩容策略

在线场景下，流式推理作业需要长时间运行，期间输入数据速率通常会动态变化，这要求其性能应该始终能够满足输入数据的速率。为满足这一要求，本章提出了一种流式推理作业资源动态扩缩容策略。4.1 小节介绍了问题提出的背景并对问题进行分析，4.2 小节介绍了一种轻量化多阶段的流处理算子性能建模方法，4.3 小节介绍了基于算子性能模型的流推理并行优化策略，4.4 小节对本章提出的方法与策略进行了实验评估与分析。

4.1 问题背景与分析

与传统流处理作业类似，在线场景下的流式推理作业也面临着输入数据速率动态变化的问题。当输入数据速率上升时，若流式推理作业的处理性能不能满足满足输入数据速率，会导致作业的端到端处理时延大幅增加，进而对用户或者下游业务造成影响。图 4.1 展示了一个流式推理作业端到端时延在输入数据速率动态变化时的情况。在第 300 秒时，输入数据速率从 200 条/秒变为 1000 条/秒，此时流式推理作业的配置保持不变，可以看到作业的端到端时延大幅增加。

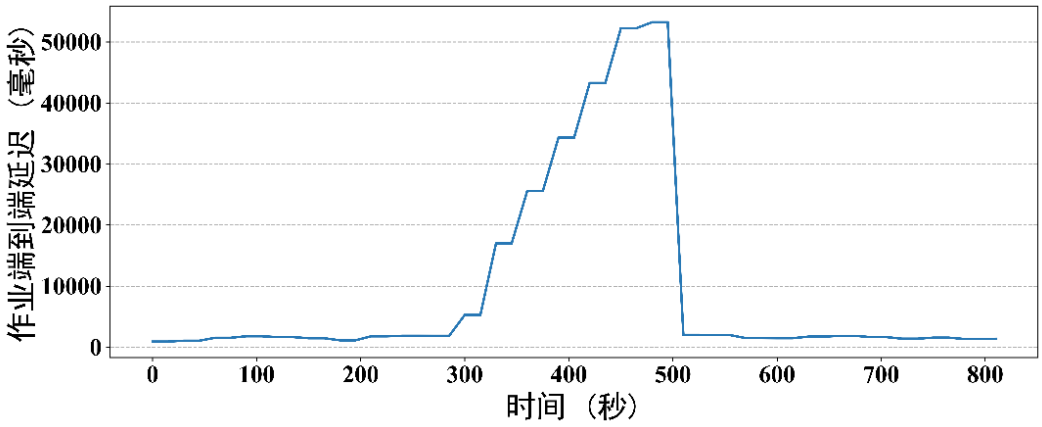


图 4.1 输入数据速率动态变化时流式 AI 模型推理作业端到端时延变化情况

针对上述问题，一种简单的解决方案为根据高峰流量值，预先为流式推理作业分配足够的资源。但在流量低峰期间，预先分配的额外资源无法被充分利用，导致资源浪费、成本增加。同时，流式推理作业与传统流处理作业不同，其通常需要使用 GPU 进行 AI 模型推理加速，而 GPU 的使用成本显著高于 CPU（以阿里云为例，同规格的云服务器，额外增加一张 V100 GPU 后，每小时的使用成本

增加了约 6 倍[62]），这导致了预先分配资源的方案会进一步增加资源的使用成本。图 4.2 展示了某互联网头部社交媒体平台在使用预先分配资源方案的过程中，某流式推理作业的 24 小时 GPU 利用率情况，可以看到在业务低峰期时 GPU 利用率大幅下降，造成了大量的资源浪费。

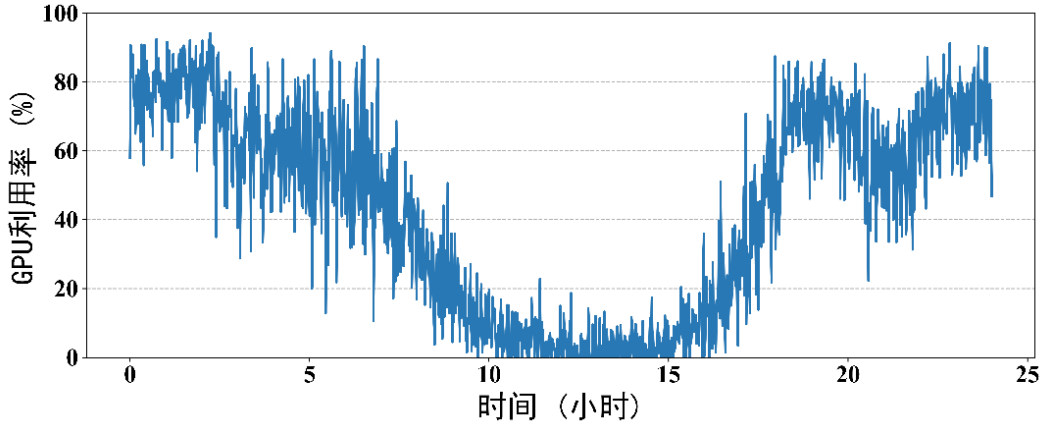


图 4.2 某互联网头部社交媒体平台某流式推理作业 24 小时 GPU 利用率情况

在该背景下，流式推理作业需要能够根据输入数据速率的变化进行动态的资源扩缩容。具体来说，假设流式推理作业的算子集合为$S_O = \{O_i \mid 0 \le i \le n\}$，当输入数据速率变化为$r$时，需要确定各个算子$O_i$的最小并行资源实例数量（即$O_i$的并行度），使$O_i$的处理速率能够满足上游数据的输入速率。针对流处理作业算子并行度自动调整策略，当前已经存在多种方法：如 DS2[44]方法根据算子处理的忙闲比例来确定算子的并行度、ConTune[47]方法使用两阶段并行度调整方法以及贝叶斯优化方法来确定算子的并行度，但这些方法都提出的背景均面向传统的流处理作业，并没有考虑到流式推理作业中需要使用 GPU 的特点，而这一特点也为并行度调整引入了新的挑战：

（1）流式推理作业算子性能预测难

在进行流式推理作业并行度调整时，通常需要获取算子在新并行度下的性能表现，从而判断新并行度是否能够满足用户的性能要求。针对该问题，通常使用两种方案：一种方案为直接使用新并行度配置运行作业，从而收集算子的性能表现；另一种方案则为预测算子在新并行度下的性能表现。

针对第一种方案，其优点为获取的算子性能表现准确度高，但缺点也十分明显，即每次调整并行度都需要多次运行作业，一方面需要消耗大量的资源，另一方面也会导致并行度调整策略决策过慢，导致作业无法满足用户要求。而第二种

方案为预测算子的性能表现，从而避免了重复运行作业导致的资源开销大的缺点，但该方案也面临着需要准确预测算子性能表现的挑战。

在传统流处理作业场景下，存在一些预测算子性能的解决方案，如基于算子历史性能表现以及线形模型进行预测[44]。但由于流式推理作业中的 AI 模型推理算子通常需要使用 GPU，从而带来了新的硬件特性，即在使用 GPU 进行 AI 模型推理时，积攒一批数据（这一批数据的数量通常被称为 Batch Size）之后同时进行计算会带来更高的吞吐。这就导致了当 AI 模型推理算子使用不同的 Batch Size 时会有不同的性能表现。

基于该背景，在预测流式推理作业算子性能时，还需要考虑不同 Batch Size 带来的影响。例如当增加某个 AI 模型推理算子的 Batch Size 后，可以增加该算子的吞吐，从而降低 GPU 资源的使用量。

（2）作业整体吞吐与端到端时延权衡难

在在线场景下，用户对流式推理作业的要求除了作业的整体吞吐外，还对作业的端到端时延有要求。在传统流处理作业中，作业的端到端时延通常仅和作业本身的复杂程度有关，即当作业的整体吞吐能够满足输入数据速率时，作业本身的算子越多，则端到端时延越长。所以传统的流处理作业并行度调整方法通常仅考虑作业的整体吞吐，忽略了端到端延迟。

然而，由于上文提到的 GPU 可以积攒一批数据同时计算从而提高吞吐的特性，在流式推理作业运行过程中还需要考虑积攒数据带来的等待时间。例如，当增大 AI 模型推理算子的 Batch Size 后，单个算子获得了更大吞吐，但积攒一批数据的时间也相应增加，导致作业的端到端时延增加，同样当减小 Batch Size 后，单个算子吞吐下降，积攒一批数据的时间以及作业的端到端时延也会相应下降。

所以在调整流式推理作业的并行度时，需要权衡作业的整体吞吐与端到端时延，即需要找到一个合适的 Batch Size 与并行度，使得作业可以利用更少的资源达到用户的吞吐与端到端时延要求。

针对上述问题，本文提出了一种面向在线流式推理场景的资源扩缩容策略。该策略首先对流式推理作业中的算子进行性能建模。再得到算子性能模型后，再依据该性能模型，以及用户作业拓扑结构，求解满足用户吞吐以及端到端时延要求的并行度。下文将从两个方面对该策略展开叙述：

（1）如何对流式推理作业的算子进行性能建模？（4.2 小节）

（2）如何确定流式推理作业满足用户要求所需的并行度？（4.3 小节）

4.2 轻量化多阶段的流式推理作业算子性能建模方法

为避免在进行并行度调整时频繁重复运行作业，需要对流式推理作业算子进行性能建模。构建算子性能模型的方法可以粗略分为两类：一类为对硬件以及算子运算过程进行理论分析并构建理论性能模型，另一类为实际运行算子并根据算子的实际运行情况来构建性能模型。

使用理论分析构建性能模型的方法不需要消耗额外的计算资源，但由于流式推理作业通常需要使用 GPU，而针对 GPU 以及 GPU 上的运算过程构建准确的理论性能模型较为困难。与之相对，使用实际运行的方法构建性能模型相对较为简单，但需要消耗额外的计算资源。同时，由于 GPU 的使用成本更高，在使用实际运行的方法构建性能模型时更需要降低资源开销。为解决该问题，本文提出了一种轻量化多阶段的流式推理算子性能建模方法，其核心思想为：通过分阶段实际运行与模拟运行相结合，采集多维度性能数据，并利用机器学习方法混合建模，从而构建高精度、低开销的流式推理作业算子性能模型。

本节将在 4.2.1 小节中对构建性能模型的整体过程进行介绍，并在 4.2.2 小节中针对构建过程中的关键步骤进行详细介绍。

4.2.1 性能模型构建过程概述

在构建性能模型时，需要明确模型要预测的性能指标以及与之相关的自变量集。性能指标方面本文方法关注算子的吞吐。自变量方面，对于传统流式处理作业来说，对算子吞吐影响最大的参数为算子并行度。对于流式推理作业来说，算子并行度同样重要，同时由于其中某些算子需要使用 GPU，所以也需要考虑 Batch Size 对算子吞吐的影响。综上，构建算子O_i性能模型的关键为构建一个关于算子并行度p_i、Batch Size b_i与吞吐T_i之间的关系函数$Perf$：

$$T_i = Perf(p_i, b_i) \tag{4.1}$$

总体上，本文提出的性能模型构建方法属于基于实际运行的方法，即需要以不同的算子并行度、Batch Size 运行并收集算子吞吐数据，并利用收集到的数据构建算子性能模型。在基于吞吐数据构建算子性能模型时，本文使用了一种混合建模方法。具体来说，本文方法将构建函数$Perf$的过程拆解为两个阶段：

（1） 构建单点性能模型P：单点性能模型反映了算子O_i在并行度为 1 时，算子吞吐与 Batch Size 之间的关系。在构建算子性能模型的过程中会先构建各个算子的单点性能模型。由于这一步骤中各个算子的并行度均为 1，所以使用少量资源即可完成（例如一张 GPU）。

（2） 构建扩展性能模型S：扩展性能模型反映了算子O_i的吞吐与并行度之间的关系。在构建完成各个算子的单点性能模型后，会继续构建各个算子的扩展性能模型。此外，在这一步骤中使用了模拟运行的方法，从而避免了使用大量 GPU 资源，降低了资源开销。

综上，本文使用了一种混合性能模型，即结合单点性能模型P与扩展性能模型S来共同预测算子的吞吐：

$$T_i = Perf(p_i, b_i) = S\big(p_i, P(b_i)\big) \tag{4.2}$$

同时，在流式推理作业中，并非所有算子都需要积攒一批数据再进行计算。对于这类算子，性能模型仅使用扩展性能模型即可，即退化为$Perf(p_i) = S(p_i)$。

此外，对于用户给定的任意流式推理作业，在构建算子性能模型前，需要获取部分关于该作业的信息，例如每个算子并行度的上限、Batch Size 的上限等，这一部分信息称为先验信息。这一步的目的是为后续构建性能模型提供必要的信息，例如在构建单点性能模型时，需要根据 Batch Size 的上限，确定采集哪些 Batch Size 配置下的性能数据。

综上所述，构建用户给定流式推理作业算子性能模型的整体流程为：

（1） 获取用户给定流式推理作业的先验信息，为后续构建算子性能模型提供必要的支撑。（4.2.2.1 小节）

（2） 针对作业中的各个算子，收集不同 Batch Size 下的算子吞吐数据，并构建单点性能模型。（4.2.2.2 小节）

（3） 最后，收集作业中各个算子在不同并行度下的吞吐表现，并构建扩展性能模型。（4.2.2.3 小节）

4.2.2 混合性能模型构建方法

在上一小节中概述了本文提出的混合性能模型$Perf(p_i, b_i) = S\big(p_i, P(b_i)\big)$，并简要介绍了构建流式推理作业算子性能模型的整体流程，在本小节中将对构建过程中的关键步骤进行详细介绍，包括作业先验信息获取方法、单点性能模型构

建方法以及扩展性能模型构建方法。

4.2.2.1 作业先验信息获取方法

在构建用户给定流式推理作业算子性能模型前，需要获取关于该作业必要的先验信息。本步骤中需要获取的先验信息包括：

（1） 算子 Batch Size 上限：构建算子单点性能模型时，需要在不同 Batch Size 下运行算子并收集吞吐数据。在此过程中，需要均匀的收集算子 Batch Size 范围内的多个数据点，所以在本步骤中需要获取算子 Batch Size 的上限。

（2） 算子并行度上限：同理构建单点性能模型的过程，在构建扩展性能模型时，也需要均匀的收集算子并行度范围内的多个数据点，所以同样需要获取算子并行度的上限。

（3） 算子输出输入比：某些算子在处理一定量的输入数据后，并不会输出相同数量的数据。例如某些推理过程会针对一条输入文本生成多个输出向量、过滤算子会排除部分输入数据导致输出数量小于输入数量。当算子输入i条数据后，若输出数据量为o，则算子的输出输入比r定义为:

$$r = \frac{o}{i} \tag{4.3}$$

在后续求解算子并行度时，往往会要求算子的吞吐能够满足上游算子的输出速率，此时就需要根据上游算子输出输入比r来确定当前算子需要满足的吞吐。

在获取上述三种先验信息时，需要依据一定的经验规则，具体获取方法如下：

算子 Batch Size 上限：在获取算子的 Batch Size 上限时，会优先获取算子 Batch Size 的数量级，具体来说会从 Batch Size 为 1 开始尝试运行算子并不断提升 Batch Size 的数量级，直至超出 GPU 显存、算子吞吐变动幅度不大或大幅下降。额外使用算子吞吐判断是否达到 Batch Size 上限的原因是：某些 AI 模型推理框架（如 Tensorflow）会独占 GPU 的全部显存，而非随着 Batch Size 增长，导致无法依据 GPU 显存来判断是否达到 Batch Size 上限。

当搜索到算子 Batch Size 所在的数量级后，会在该数量级进行二分搜索，进一步查找精确的 Batch Size 上限。判断 Batch Size 是否达到上限的条件同上，且当搜索的区间范围长度小于等于 Batch Size 所在数量级的 10%时，则停止搜索。

例如当二分搜索区间为 2300 至 2400 时，则不再进一步搜索，因为区间范围长度为 100，小于数量级 1000 的 10%即 100。

算子并行度上限：在获取算子并行度上限时，需要由用户给定高峰吞吐目标 T_h，并且在确定算子 Batch Size 上限的过程中，会同时记录过程中算子最低吞吐值 T_l，则该算子的并行度上限为T_h/T_l。此外，对于不需要积攒数据的算子，T_l则为直接运行该算子采集到的吞吐值。

需要注意的是，这里计算得到的并行度上限可能会小于实际的上限，这是因为算子的吞吐并不一定随着并行度线性增长。获取这一信息的主要目的为：在构建扩展性能模型时，可以采集到更接近实际并行度上限的数据值，有助于构建更精确的算子扩展性能模型。

算子输出输入比：获取算子输出输入比的方法较为直观，给定算子一定数量的数据并运行该算子，根据其输出数据的数量即可计算出该算子的输出输入比。在这一过程中，需要避免输入过多的数据导致运行时间过长。解决方法为在搜索算子 Batch Size 上限时同时记录算子的最大吞吐，根据算子的最大吞吐值以及时间限制计算得出输入数据的数量，从而避免运行过长时间。

4.2.2.2 单点性能模型构建方法

构建算子单点性能模型的总体流程为：在算子 Batch Size 范围内，均匀地以不同 Batch Size 运行算子，并采集算子的吞吐数据，再基于机器学习算法拟合算子 Batch Size 与吞吐之间的关系。

在拟合算子 Batch Size 与吞吐之间的关系时，本文选择了随机森林算法，这是由于：一方面需要积攒一批数据同时计算的情况较多，例如积攒一批数据可以降低算子的 IO 开销、积攒一批数据同时计算可以更充分利用 GPU 等，另一方面在采集算子的吞吐数据时，需要考虑资源开销，导致采集的数据量有限，以上两方面决定了需要选取一种所需数据量适中、表达能力较强的机器学习算法。

对于线性回归、多项式回归等方法，其要求等数据量适中，但表达能力较弱，不易体现不同算子 Batch Size 与吞吐之间的关系。而如多层感知机（MLP）等方法表达能力较强，但需要较多数据，会导致采集算子吞吐数据时间开销以及资源开销过大。综上所述，随机森林算法能够平衡所需数据量以及表达能力，在较小的训练数据集上也可以取得较好的拟合效果。

此外，在构建算子的单点性能模型时，也会同时收集算子在不同 Batch Size 的 CPU 资源使用量，并同样适用随机森林算法进行拟合。这一步的目的是为后续确定作业整体资源使用量时提供必要的信息。

4.2.2.3 扩展性能模型构建方法

扩展性能模型会预测算子O_i的吞吐T与并行度p_i之间的关系。一种简单的方法为使用线性模型进行预测，即假设算子O_i在并行度为 1 时的吞吐为t，则其吞吐与并行度之间的关系为：

$$T = p_i \cdot t \tag{4.4}$$

然而在实际应用中，由于多种因素的影响，例如网络带宽的限制等，算子的吞吐可能并非随着并行度线形增长。如图 4.3 所示，该算子使用 ViT Tiny 模型，该模型接受图片作为输入数据，会占用较多的网络带宽，导致带宽成为瓶颈，所以该算子的吞吐与并行度之间并非线形关系。

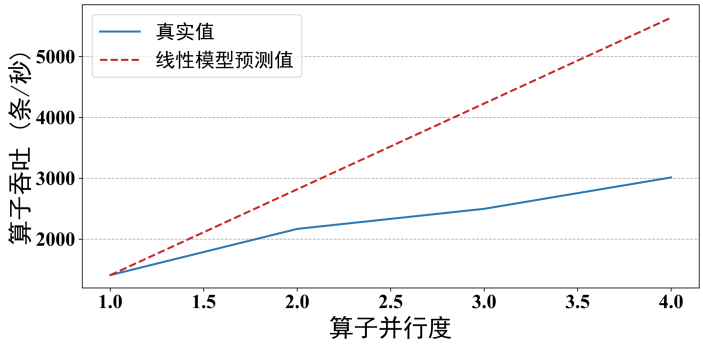


图 4.3 ViT Tiny 模型推理算子吞吐

为解决这一问题，本文提出了衰减因子f的概念，即算子吞吐随着并行度增加而衰减的幅度，其值域为 (0,1] 。结合衰减因子f，可以将算子吞吐与并行度之间的关系修正为：

$$T = p_i \cdot t \cdot f \tag{4.5}$$

在定义衰减因子f时，本文观察到：导致算子吞吐不随着并行度线形增长的因素通常来自算子外部，如网络带宽、计算引擎本身的调度开销、通信机制等。基于此，本文认为算子本身的吞吐越大时，越容易受到外部因素的影响，例如当算子吞吐越大时，每秒需要传输的数据量越多，则更容易导致网络带宽成为瓶颈。因此，本文将衰减因子f定义为：

$$f = e^{-a \cdot t \cdot (p_i - 1)} \tag{4.6}$$

上式具有如下两个特点：（1）当p_i取值为 1 时，f恒定为 1，反映了算子在并行度为 1 时，吞吐不会衰减；（2）当p_i或t增加时，f逐步减小，反映了算子本身吞吐越大时，越容易受到外部因素影响这一观察。

公式 4.6 中的参数a为待拟合参数，需要通过拟合采集到的算子在不同并行度下的吞吐数据得到。在采集算子吞吐数据时，需要区分算子是否可以积攒一批数据同时计算：如果无需积攒数据则算子在并行度为 1 时的吞吐t为定值，如果可以积攒数据，则不同的 Batch Size 对应的t通常不同，则需要采集不同 Batch Size 以及不同并行度下的算子吞吐数据。

但在采集不同并行度下的算子吞吐数据时，若算子需要使用 GPU，则需要占用多张 GPU，导致高额的资源成本开销。为解决这一问题，本文提出了基于模拟运行的解决方案。具体来说，在算子使用 GPU 进行 AI 模型推理时，模拟运行机制并不会真正使用 GPU 进行推理，而是等待与推理该批次数据相同的时间。

该解决方案的依据同样基于上文提到的观察，即导致算子吞吐非线性增长的原因往往来自外部，所以将使用 GPU 推理的过程替换为等待相应的时间不会对外部因素产生影响，采集到的吞吐数据同样能够反映吞吐非线性增长的特性。通过模拟运行的方法，避免了使用多张 GPU，从而大大降低了算子性能建模的资源成本开销。

4.3 基于算子性能模型的流式推理作业并行优化策略

在流式推理作业长时间运行期间，当输入数据的速率变化时，需要及时进行资源扩缩容，从而动态调整算子的并行度以适应输入数据速率。在在线场景下，用户对于流式推理作业通常具有整体吞吐与端到端时延两方面的要求。然而，由于流式推理作业中的部分算子可以积攒一批数据同时计算从而提高吞吐，所以在调整算子并行度时需要额外考虑算子的 Batch Size 配置。提高 Batch Size 可以提高单个算子的吞吐，降低资源使用量，但同时也会增加等待时间，因此在调整算子并行度时需要在作业整体吞吐与端到端时延之间进行权衡。

尽管传统流式处理作业的并行度调整算法可以应用到流式推理作业中，但由于没有考虑到算子可以积累一批数据同时计算来提高吞吐这一特点，可能会导致资源浪费、端到端时延无法满足等问题。针对上述问题，本文提出了一种基于算

子性能模型的流式推理作业并行优化策略，其中算子性能模型为 4.2 小节中介绍的算子性能模型。

本节将在第 4.3.1 小节中针对流式推理作业并行优化问题进行形式化建模。针对该问题，在第 4.3.2 小节中将介绍本文提出的基于动态规划的流式推理作业并行优化算法。

4.3.1 流式推理作业并行优化问题形式化描述

流式推理作业并行优化问题可以概述为：当用户给定流式推理作业、作业目标吞吐以及目标端到端时延后，确定给定作业能满足用户目标所需的最小资源量。

具体来说，对于用户给定的流式推理作业J，共包含n个算子，其算子集合为$O_s = \{O_1, \dots, O_n\}$，其中算子集合中的算子顺序为作业$J$执行图的拓扑排序。其中，第$i$个算子$O_i$的并行度为$p_i$、Batch Size 为$b_i$（若为需要积攒数据的算子）、输出输入比为$f_i$，其在该配置下可以达到的吞吐为$T_i$、延迟为$L_i$，其所需的 CPU 资源量为$C_i$、GPU 资源量为$G_i$，作业的数据输入速率为$r$，用户的端到端时延要求为$t_{slo}$，则流式推理作业的并行优化问题可以表示为最小化资源使用量：

$$\min \sum_{i=1}^{n} (C_i + \alpha \cdot G_i) \tag{4.7}$$

同时需要满足用户要求，即需要在如下约束内求解上式：

$$T_i \geq r_{i-1}, \qquad \forall\ i \in [1, \dots, n] \tag{4.8}$$

$$\sum_{i=1}^{n} L_i \leq t_{slo} \tag{4.9}$$

上述两个约束表示：（1）每个算子都要能够满足上游算子的输出速率（公式 4.8）；（2）作业中每个算子时延的和（即作业整体的端到端时延）能够满足用户要求的时延要求，即t_{slo}（公式 4.9）。下文将对该形式化描述进一步介绍。

首先，在问题目标中本文引入了系数α，这是因为使用相同数量的 CPU 与 GPU 往往具有不同的现实意义，例如同样使用 1 核 CPU 与一张 V100 GPU，其使用成本、算力等都具有很大差异。因此公式 4.7 中的系数α的作用即为将 CPU 资源使用量与 GPU 资源使用量进行统一。例如，假设单张 GPU 的使用成本为 1

核 CPU 的 10 倍，则$\alpha = 10$，或者假设单张 GPU 的算子等价于 100 核 CPU，则$\alpha = 100$。通过引入系数α，可以统一作业的 CPU 与 GPU 资源使用量。

同时，也需要确定算子O_i所需的计算资源C_i与G_i的计算方式。设算子O_i的单个实例在满负载运行时所消耗的 CPU 资源与 GPU 资源分别为c_i与g_i，则$C_i = p_i \cdot c_i$，$G_i = p_i \cdot g_i$。其中，对于c_i，若算子无需积攒数据，则为定值，若算子需要积攒数据，则和算子设定的 Batch Size 值b_i有关。对于g_i，本文设定$g_i \in \{0,1\}$，这和 GPU 的复用方式有关。GPU 的复用方式可以分为两类：

（1） 时分复用：多个进程在共用 GPU 时，同一时间内仅允许一个进程使用 GPU，不同时间多个 GPU 可以交替使用 GPU。在该模式下，多个进程共用 GPU 通常不会带来较大的性能提升。通常情况下，GPU 的默认复用方式即为时分复用。

（2） 空分复用：由于 GPU 存在多个计算核心，空分复用的方式运行多个进程在同一时间使用不同的计算核心进行计算。此种复用方式可以更加有效的利用 GPU 资源，但需要额外的软件或硬件支持。例如虚拟 GPU 方案（qGPU[63]、cGPU[64]）、MPS 方案[65]等。

在本文中，均默认 GPU 没有使用额外的配置，采用默认的时分复用模式。则在该模式下，使用非整数个 GPU 通常不会带来较大的性能提升。此外，本文认为通常算子不应使用多于 1 个 GPU，这是由于如果算子需要在多张 GPU 上并行处理，则应将该功能交付于底层引擎进行处理（如 Ray），而非由用户在一个算子实例内自行实现。综上，g_i的取值为 0 或 1，对于需要使用 GPU 的算子，则g_i取值为 1，反之则为 0。

其次对于约束 4.8，第i个算子的输出速率r_i表达式如下：

$$r_i = \begin{cases} r, & i = 0 \\ f_i \cdot \min(T_i, r_{i-1}), & i \in [1, \dots, n] \end{cases} \tag{4.10}$$

对于算子O_i，其输出速率为该算子的输出输入比f_i与实际吞吐的乘积。算子的实际吞吐表达式为$\min(T_i, r_{i-1})$，这是由于上游算子O_{i-1}的输出速率r_{i-1}可能小于算子O_i能够达到的吞吐T_i，则算子实际的吞吐为r_{i-1}。此外由于算子本身特性，其可能输出比输入数据量更多或更少的数据，所以算子的输出速率还需要与f_i相乘。对于源算子r_1，由于其没有上游算子，则其上游的输出速率定义为作业本身的数据输入速率，即$r_0 = r$。

最后，对于约束 4.9，算子O_i的时延L_i可以用如下公式来表达：

$$L_i = \begin{cases} t_{exec}^i, & O_i \notin O_B \\ t_{wait}^i + b_i \cdot t_{exec}^i, & O_i \in O_B \end{cases} \tag{4.11}$$

其中O_B为所有可以积攒一批数据同时计算的算子集合。上式表达了两种情况，即算子的延迟需要区分算子是否需要积攒数据：

（1） 若算子O_i不需要积攒一批数据，则处理一条数据的时间t_{exec}^i即为该算子的时延。

（2） 若算子O_i需要积攒一批数据，则该算子的时延分为两个部分：等待积攒一批数据的时间t_{wait}^i与处理这一批数据所需的时间$b_i \cdot t_{exec}^i$。其中等待时间可以表示为：

$$t_{wait}^i = \left\lceil \frac{b_i}{r_{i-1}} \right\rceil \tag{4.12}$$

即积攒该算子 Batch Size 数量的数据所需的时间。对于处理时间，t_{exec}^i表示处理一条数据所需的时间，所以需要与b_i相乘，表示处理一批数据所需的时间。

对于公式 4.11 中的处理一条数据所需时间，本文定义其计算方式为$t_{exec}^i = 1/T_i$，即使用该算子所能达到的吞吐计算。同时，采用该方式计算时，也包含了数据在算子与算子之间传输的时间。

4.3.2 基于动态规划的流式推理作业并行优化算法

在 4.3.1 小节中提到，算子O_i的资源消耗与算子的并行度p_i与 Batch Size b_i有关，则求解该问题的关键在于为每个算子选择最优的并行度与 Batch Size，使得作业消耗的资源最少。在本小节中，将首先对该问题进行进一步分析与建模（4.3.2.1 小节），并介绍一种基于动态规划的并行优化算法对问题进行求解（4.3.2.2 小节）。

4.3.2.1 问题分析与建模

在流式推理作业运行的过程中，每个算子都需要消耗一定时间对输入的数据进行处理，当一条数据流经整个作业图后，其消耗的时间即为作业的端到端时延。对于传统流式处理作业，其作业图的规模往往决定了作业的端到端时延。但对于流式推理作业，其中一些算子需要积攒一批数据再进行计算（如使用 GPU 进行

推理），导致流式推理作业的端到端时延不仅和作业图的规模有关，也和这些算子的 Batch Size 有关。

对于传统的流式处理作业并行优化算法，由于通常不涉及积攒一批数据同时处理的场景，所以作业的端到端延迟已经由作业图决定，不存在调整空间。则在调整并行度时仅需考虑如何确定每个算子的并行度，使得作业能够满足数据的输入速率即可。然而流式推理作业引入了新的问题，这里举例说明：若存在如图 4.4 所示的流式推理作业，其包含四个算子，其中推理算子与正则化算子均需要积攒一批数据执行。

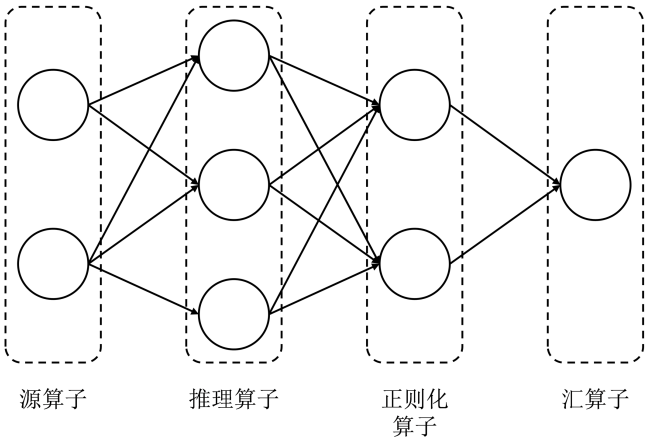


图 4.4 流式推理作业示意图

当上游输入数据速率增加后，需要调整每个算子的并行度。在调整过程中，会面临如下两个问题：

（1） 在调整推理算子和正则化算子的并行度时，需要同时考虑 Batch Size 对这两算子性能的影响。例如，若提升这两算子的 Batch Size 可以相应增加单个算子实例的吞吐，则在调整并行度时可以适当降低算子的并行度，从而降低资源开销。

（2） 推理算子与正则化算子使用不同的并行度与 Batch Size 会带来不同的时延，需要考虑如何利用有限的端到端时延。例如，当上游数据输入速率为 1000 条/秒时，若并行度为 2、Batch Size 为 128，则积攒一批数据的时间为 256 毫秒，若并行度为 4、Batch Size 为 256，则积攒一批数据的时间变为 1024 毫秒，同时算子在不同 Batch Size 下的吞吐可能不同，导致执行时间也并不相同。在该背景下，需要考虑如何利用有限的端到端时延，例如要求端到端时延为 5000 毫秒，源算子与汇算子的时延均为 100 毫秒，则可以要求推理算子的

时延低于 3000 毫秒、正则化算子的时延低于 1800 毫秒。不同的时延要求方案可能会带来不同的资源开销，需要求解最优的方案。

为解决上述问题，本文将流式推理作业的并行优化问题建模为分组背包问题。具体来说，在进行作业并行度调优时，对于无需积攒一批数据的算子，其时延已经固定，将这些算子的时延去除后，剩余的时延即为其余需要积攒一批数据的算子可用的时延。

对于需要积攒一批数据的算子，其时延由上游数据输入速率、算子的并行度与 Batch Size 共同决定。对于上游数据的输入速率，由于最终每个算子都需要满足上游数据的输入速率，则可以根据作业的输入数据速率以及每个算子的输出输入比计算得出。例如算子O_3的输入数据速率，可以根据作业的输入数据速率r以及两个上游算子O_1、O_2的输出输入比f_1、f_2确定，即为$f_2 \cdot f_1 \cdot r$。由此，当用户给定作业与目标输入数据速率后，即可确定每个算子的上游数据输入速率。

当确定各个算子的上游数据输入速率后，给定并行度与 Batch Size 就可以确定该配置下算子的时延与吞吐。在算子可能的并行度与 Batch Size 范围内（即算子的配置空间），去除掉吞吐不能满足上游数据输入速率以及时延超出端到端时延要求的配置组合，剩余的配置即为可能满足要求的配置组合。本文将这些配置组合看作背包问题中的物品，时延看作物品的体积，由该配置计算得到的资源开销看做物品的重量，时延要求看作背包总体积，则该问题可以建模为如何在每个算子对应的物品中挑选一个物品，使得总体积不超过背包的体积且重量最小。

4.3.2.2 基于动态规划的并行优化算法

在上一小节中将流式推理作业并行优化问题建模为了分组背包问题，在本小节中将基于动态规划算法对该问题进行求解。首先，本文定义状态$S_{i,j}$，表示使用前i个需要积攒一批数据的算子在时延要求j下所需的最小资源。注意到，由于时间是连续变化的变量，所以需要进行离散化处理，这里使用毫秒为单位对其进行离散化处理。同时，定义第i个算子的可用配置组合列表为A_i，其中列表中的每个配置项$A_i[k]$都包括两个属性，即时延$A_i[k].latency$与资源开销$A_i[k].cost$。则对于本问题，状态转移方程为：

$$S_{i,j} = \min_{0 \le k < size(A_i)} \left(S_{i-1, j-A_i[k].latency} + A_i[k].cost \right) \tag{4.13}$$

若存在n个需要积攒一批数据的算子，对这些算子的时延要求为t，则$S_{n,t}$、为最终结果。同时在求解过程中需要记录状态的转移路径，用于给出各个算子最终所选择的配置。

在求解过程中，需要获取给定配置后的时延与资源开销等属性。本文基于 4.2 小节中介绍的性能建模方法，对各个算子构建相应的性能模型，并基于算子的性能模型来获取所需要的信息。通过该方法，可以快速获取算子在给定配置下的性能信息，从而可以做到在作业输入数据速率变化后快速响应。

算法总体流程为如下：

算法 4.1 基于动态规划的并行优化算法

输入：用户作业的算子列表$O_s = [O_1, \dots, O_n]$，其中算子按照拓扑排序顺序排列，端到端时延要求t_{slo}，作业的输入数据速率r，GPU 资源与 CPU 资源的比例系数α
输出：更新各个算子的并行度与 Batch Size 配置后的算子列表O_s

```
target_rate = r
time_left = t_slo
batch_op_list = []

for i = 1 to n do
    O_i = O_s[i]
    if O_i is batch operator then
        batch_op_list.append((target_rate, O_i))
    else
        p_i = calculate minimal parallelism of O_i
        O_i.conf = (p_i, null)
        time_left = time_left - O_i.L
    target_rate = target_rate * O_i.f

for i = 1 to size(batch_op_list) do
    curr_target_rate, batch_op = batch_op_list[i]
    batch_op.available_conf=get_available_conf(batch_op,curr_target_rate,
time_left, α)

calculate_conf(batch_op_list, time_left)
return O_s
```

该算法接受用户给定作业的算子列表O_s、端到端时延要求t_{slo}、作业的数输入数据速率r以及 GPU 资源与 CPU 资源的比例系数α。其中，算子列表中的元素为一个对象，其包含多种在求解过程中所需要的属性，这些属性均为使用 4.2 小节中的性能建模方法得到的。

首先，算法 4.1 的第 4 至 12 行对算子列表进行了遍历。在每次遍历过程中，会更新下游算子的输入数据速率，更新方法为将当前算子的输入数据速率与算子的输出输入比相乘（第 12 行）。同时，在遍历过程中，会区分算子是否需要积攒一批数据，若需要，则将其连同输入数据速率进行暂存，用于后续的进一步求解（第 7 行）；若不需要，则该算子的时延为固定值O_i.L，需要在作业整体的端到端时延中减去，用于计算后续需要积攒一批数据的算子的时延要求（第 11 行），并且由于时延为固定值，则该算子资源开销最小的情况即为并行度最小，根据该算子的扩展性能模型进行计算，并保存到该算子对象中即可（第 9、10 行）。

其次，对于所有需要积攒一批数据的算子均需要生成符合要求的配置项（第 16 行），这里调用了 get_available_conf 函数。最后根据生成的配置项，基于动态规划算法进行求解（第 18 行），这里调用了 calculate_conf 函数。下面将针对 get_available_conf 函数与 calculate_conf 函数进行详细介绍。

对于 get_available_conf 函数，其作用为生成算子能够满足上游数据输入速率要求，且时延不超过给定时延要求的所有可能配置。具体算法流程如下：

算法 4.2 获取算子满足要求的所有可用配置项（get_available_conf 函数）

输入：待获取配置项的算子O，上游数据输入速率r_u，时延要求t，GPU 资源与 CPU 资源的比例系数α

输出：算子O的所有可用配置项列表A

1. Initialize configuration list A
2. **for** $b = 1$ to O.batch_size_limit **do**
3. p_m = calculate max parallelism under b and r_u according to O.performance_model
4. **for** $p = 1$ to p_m **do**
5. $latency,\ throughput = O$.performance_model(p, b)
6. $t_{wait} = p \cdot b / r_u$
7. **if** $latency \le t_{wait}$ && $throughput \ge r_u$ && $latency + t_{wait} \le t$ **then**
8. Initialize an element a in A
9. $a.latency = latency + t_{wait}$
10. $a.cost = p \cdot (\alpha \cdot O.$gpu_usage $+ O.$cpu_usage_model$(b))$
11. $a.batch_size = b$
12. $a.parallelism = p$
13. A.add(a)
14. **return** A

该算法会遍历给定算子O的所有可能 Batch Size 取值，其中算子O的 Batch Size 上限可在对算子进行性能建模时得到（第 2 行）。对于每种可能的 Batch Size 取值，算法首先会根据算子的性能模型以上游数据输入速率，计算得到并行度上

限，即在该 Batch Size 取值下，根据算子性能模型计算满足上游数据输入速率所需的并行度（第 3 行）。然后算法会遍历所有可能的并行度取值，并分别计算该算子在给定并行度与 Batch Size 下的处理一批数据的延迟与吞吐（第 5 行），以及等待一批数据所需的时间（第 6 行）。对于一个可用的配置组合，需要满足以下三个条件，分别对应算法第 7 行中的三个判断条件：

（1） 处理一批数据的延迟应小于等于等待时间，这样可以保证数据不会积压，进而导致作业端到端延迟增加。

（2） 算子吞吐应大于等于上游数据输入速率，同样可以保证数据不会积压。

（3） 算子的总体延迟，即处理一批数据的延迟以及等待一批数据的延迟之和，应小于总体延迟要求。

当使用给定配置项能够满足上述条件时，则可以将该配置项加入到可用配置项列表中（第 7 至 13 行）。

当得到所有可能的配置项后，则需要使用 calculate_conf 函数求解最优配置，求解过程如下：

算法 4.3 求解最优配置算法（calculate_conf 函数）

输入：待求解配置的算子列表O_b，时延要求t

1. $dp =$ initialize 2-dimension array with -1
2. $trace =$ initialize 2-dimension array with (-1,-1,-1,-1)
3. **for** $i = 0$ to size(O_b) $- 1$ **do**
4. $O = O_b[i]$
5. **for** $t_c = t$ to 0 **do**
6. $cost_{min} = +\infty$
7. $b_{best} = -1$
8. $p_{best} = -1$
9. $from = (-1, -1)$
10. **for** a in O.available_conf **do**
11. $latency$ = ceil($a.latency$)
12. **if** $latency \le t_c$ && ($i == 0$ || $dp[i-1][t_c - latency] \neq -1$) **then**
13. $cost = a.cost$
14. **if** $i > 0$ **then**
15. $cost = cost + dp[i-1][t_c - latency]$
16. **if** $cost < cost_{min}$ **then**
17. $cost_{min} = cost$
18. $b_{best} = a.batch_size$
19. $p_{best} = a.parallelism$
20. $from = (i-1, t_c - latency)$
21. **if** $b_{best} \neq -1$ && $p_{best} \neq -1$ **then**

22. $dp[i][t_c] = cost_{min}$
23. $trace[i][t_c] = (b_{best}, p_{best}, from[0], from[1])$
24. $index = (\text{size}(O_b) - 1, t)$
25. **while** $index[0] \geq 0$ **do**
26. **assert** $dp[index[0]][index[1]] \neq -1$
27. $b, p, i_{from}, j_{from} = trace[index[0]][index[1]]$
28. $O_b[index[0]].\text{conf} = (p, b)$
29. $index = (i_{from}, j_{from})$

该算法的求解过程为典型的求解分组背包问题的过程。初始时会初始化状态数组（第 1 行），该状态数组中表示的状态与公式 4.13 中描述的状态一致。同时，由于本问题需要求解最终配置，需要额外的数组记录状态转移的过程，即算法中第 2 行对应的数组。算法求解时，会遍历状态数组dp，对于每个被遍历到的状态，需要根据公式 4.13 寻找资源开销最小的配置项，即算法中第 6 行至 23 行，其中由于求解过程中需要对延迟进行离散化处理，则在第 11 行对算子处理延迟进行了向上取整操作。

当求解完成后，需要根据状态转移数组，将最优配置分别赋值给相应的算子（第 24 至 29 行）。同时，可能会出现没有满足要求的配置的情况，需要进行额外判断并抛出异常（第 26 行）。

4.4 实验评估与分析

本节通过一系列实验来对本章提出的算子性能建模方法与流推理并行优化策略进行评估与分析。首先，4.4.1 小节介绍了实验环境与配置，包括实验的软硬件环境、实验使用的工作负载以及评估指标。接着，4.4.2 小节对本章提出的轻量化多阶段的流处理算子性能建模方法进行了评估。最后，4.4.3 小节在某互联网头部社交媒体平台的环境下对本章提出的基于算子性能模型的流推理并行优化策略进行了评估。

4.4.1 实验环境与配置

4.4.1.1 软硬件环境

硬件方面，在 4.4.2 小节中，我们使用了三台物理服务器，每个节点都包含两个 Intel(R) Xeon(R) Silver 4314 CPU @ 2.40GHz 以及两张 NVIDIA A6000 GPU，其显存大小为 48GB。内存方面，每个节点都拥有 256GB 的内存；网络方面，三

节点均通过 10 Gbps 的以太网相互连接。在 4.4.3 小节的某互联网头部社交媒体平台环境下，使用的是该平台提供的云服务器，可自由配置 CPU、GPU 数量以及内存大小，CPU 规格为 AMD EPYC 9K84 CPU @ 2.6GHz，GPU 规格为 NVIDIA L20，显存大小为 48GB，且节点之间均使用 10Gbps 的以太网相互连接。

软件方面，在 4.4.2 小节中，三台物理服务器使用的操作系统均为 CentOS 8.5.2111 版本，且 Linux 内核版本为 v4.18.0。同时，实验使用了 Ray v2.39.0 作为数据处理以及 AI 模型推理底层计算引擎。在 4.4.3 小节中则使用了某互联网头部社交媒体平台提供的 Ray Klein v2.30.，该引擎基于 Ray v2.30.0 的开源版本进行改造，添加了类似 Flink 的持续流式处理功能。

4.4.1.2 工作负载

工作负载方面，在 4.4.2 小节对算子进行性能建模过程中，本文使用了随机生成的数据。具体来说，本文实现了一个 Ray 的自定义数据源，在实验过程中，其可以根据待建模算子需要的输入，生成符合其要求（如 Tensor 大小、字典数据格式）的随机数据。在 4.4.3 小节中，由于在某互联网头部社交媒体平台环境下进行评估，则直接使用了该平台内部真实的线上数据。

此外，在 4.4.3 小节中，为了端到端评估流推理并行优化策略的有效性，本文实现了一个单独的程序来控制注入线上数据的速率，以此来模拟不同的流量。同时在 4.4.3 小节中，本文还设计了一种负载动态变化的场景，具体来说，输入数据速率初始时为 10 条/秒，持续 10 分钟后突增至 500 条/秒，维持十分钟后再回落至 10 条/秒，以此来模拟输入数据速率激增的场景。

4.4.1.3 评估指标

本节主要关注以下几个指标：

（1）算子性能模型预测误差率：指将模型预测的算子在不同并行度与 Batch Size 下的数据处理速率$r_{predict}$ (条/秒) 与实际的算子数据处理速率r_{real} (条/秒) 进行对比，即：$\frac{|r_{predict}-r_{real}|}{r_{real}}$

（2）算子性能建模资源开销：指对流推理作业中的某个算子进行性能建模所需的时间与所需要的资源的乘积，在本小节主要考虑 GPU 资源开销，即使用 GPU 张数 × 占用时间，单位为张 · 秒。

（3）流式推理作业吞吐量：指单位时间内流推理作业能够处理的数据条数，

实现上通过采集一段时间内 Ray 上报的已处理数据条数进行计算。

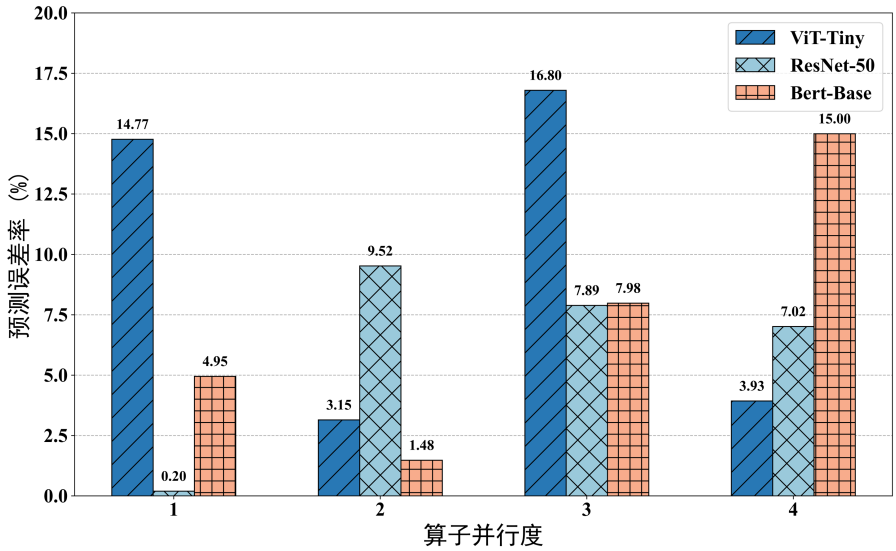
（4）流式推理作业端到端时延：指一条数据从进入系统到被输出的整体延迟。实现上采用和 Flink 记录端到端延迟相同的方法[66]，即每隔一段时间从 Source 算子生成一条特殊的数据（被称为 Barrier），并注入到作业中，记录其生成时间与离开系统的时间，作为作业整体的端到端延迟。

4.4.2 轻量化多阶段的流处理算子性能建模方法评估

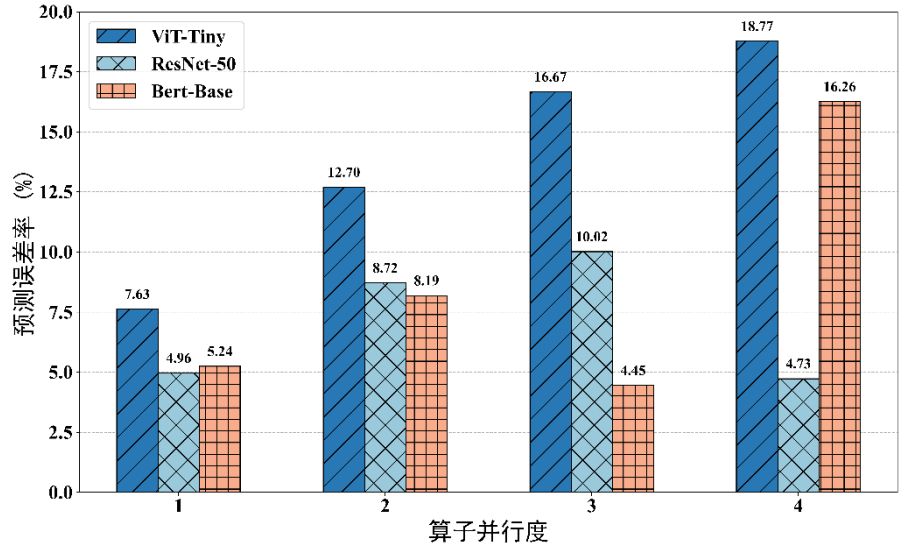
本节将对本章提出的轻量化多阶段的流处理算子性能建模方法进行评估，包括：（1）评估通过本文方法得到的算子性能模型的误差率（4.4.2.1 小节）。（2）评估使用本文方法构建算子性能模型所需的资源开销（4.4.2.2 小节）。

4.4.2.1 算子性能模型误差率评估

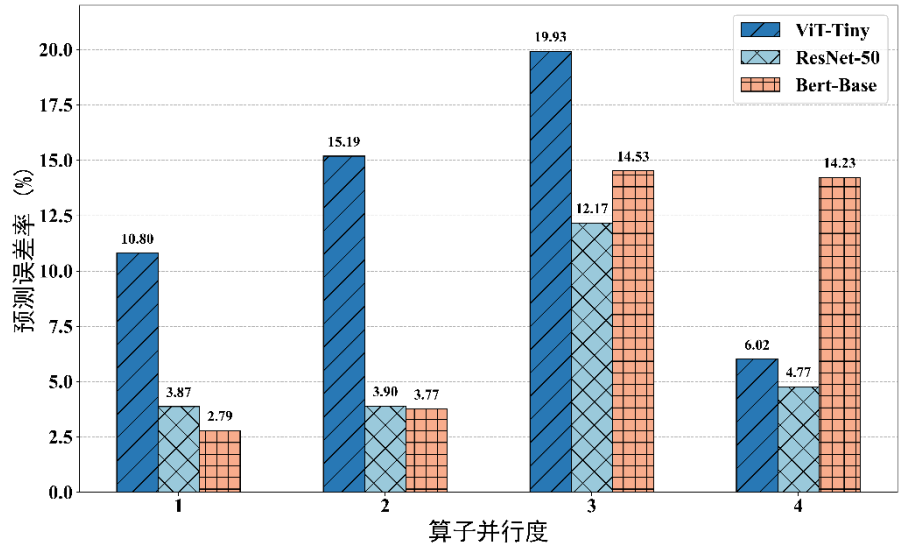
本小节将对构建出的算子性能模型的误差率进行评估。评估过程中对三种 AI 模型推理算子进行了性能建模，三种算子分别使用 ViT-Tiny[67]、ResNet-50[68]以及 Bert-Base[69]进行推理。同时，分别在 Batch Size 取值为 1000、2000、3000 以及并行度为 1、2、3、4 共 12 种配置组合下分别对算子性能模型的误差率进行了评估。同时，为确保评估的有效性，在构建性能模型时并未采集上述 12 种配置组合的性能数据。性能模型预测误差率如图 4.5 所示。



(a) Batch Size 取值为 1000



(b) Batch Size 取值为 2000



(c) Batch Size 取值为 3000

图 4.5 不同 Batch Size 以及不同并行度下算子性能模型预测误差率

分析图 4.5 (a) 到 (c) 可知，使用本文方法针对三种 AI 模型推理算子构建的性能模型，在给定的 12 种配置组合下，预测误差率均不超过 20%，表明了使用本文方法构建的性能模型能够较为准确的对算子性能进行预测。此外，在图中展示的结果种，ViT-Tiny 普遍误差率较高（最高达约 20%），这是由于 ViT-Tiny 模型较小，可以取得较大的 Batch Size，同时 ViT-Tiny 的输入数据为图片，使得处理性能受数据传输影响较大，进而导致性能波动较大。

4.4.2.2 算子性能建模方法资源开销评估

本小节将对算子性能建模方法的资源开销进行评估。评估过程中对使用 Bert-Base 模型的推理算子进行建模，在 2 张 NVIDIA A6000 上，其配置空间为

Batch Size 取值范围为 1 至 7500、并行度取值 1 和 2。同时对比了 Triton Model Analyzer[52]，该工具使用实际运行的方法对模型进行性能建模并能给出模型性能报告。实验中，Triton Model Analyzer 使用了两节点的 Triton Model Server 集群进行 AI 模型推理，同时使用了其 Manual Brute Search 模型。评估过程中，两方法每次运行时间均设置为了 3 分钟。图 4.6 展示了使用不同 Batch Size 步幅的情况下，两方法的 GPU 资源开销。

分析图 4.6 可知，本文方法进行性能建模时的 GPU 资源开销均小于 Triton Model Analyzer，在 Batch Size 步幅为 500 时，本文方法的 GPU 资源开销约为 Triton Model Analyzer 的 50%，这是由于本文方法采用了分阶段的性能建模方式，将性能建模的过程解耦为分别对单点性能模型与扩展性能模型建模，同时利用了模拟运行的方法构建扩展性能模型，有效的降低了 GPU 资源开销。同时，随着 Batch Size 步幅的减小，两方法的 GPU 资源开销均会增大，但本文方法的 GPU 资源开销增大幅度显著小于 Triton Model Analyzer，这同样是由于本文方法使用了多阶段的方法构建性能模型，仅在构建单点性能模型时需要使用 GPU。

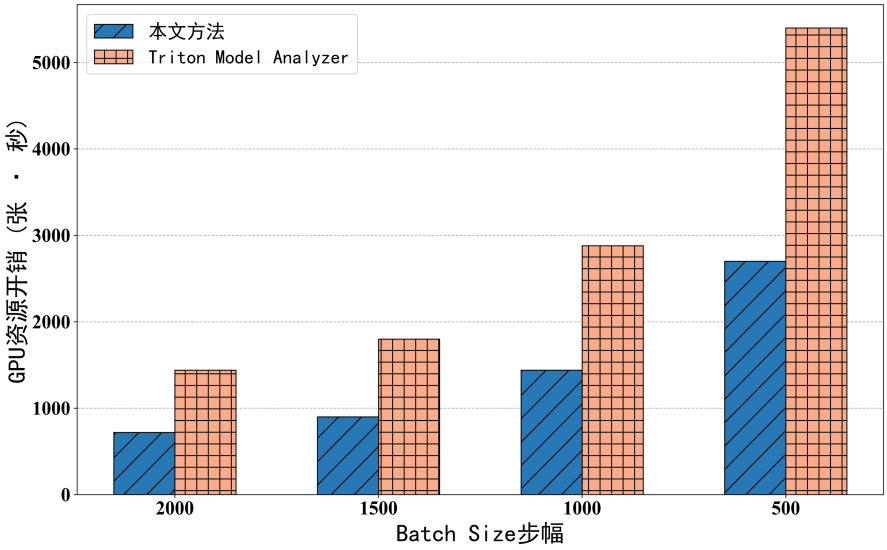


图 4.6 不同 Batch Size 步幅下本文方法与 Triton Model Analyzer 的 GPU 资源开销

4.4.3 流式推理并行优化策略有效性评估

本小节对流式推理并行优化策略进行有效性评估。首先，在 4.4.3.1 小节中，对使用了本文策略的作业进行了端到端性能评估。其次，在 4.4.3.2 小节中，与并行调整策略 DS2[44]进行了对比。

4.4.3.1 端到端作业性能评估

本小节对本文提出的流式推理并行优化策略进行了端到端评估。评估过程中使用的作业结构如图 4.7，Source 算子会从 Kafka 中持续读取数据，在数据预处理算子中会对输入文本进行词嵌入操作并输出。AI 模型推理算子使用了 Bert-Large 模型对嵌入后的文本进行 Embedding 操作，接着后处理算子使用 Numpy 对 Embedding 向量进行归一化操作，最终数据通过 Sink 算子写出。



图 4.7 评估过程所使用作业的结构

此外，在该作业中，AI 模型推理算子与数据后处理算子都需要积攒一批数据后再进行计算。数据方面使用了某互联网头部社交媒体平台真实的线上数据，同时使用了 4.4.1.2 小节中介绍的输入数据速率动态变化的场景来对本文策略进行评估。

在实验过程中，对作业的吞吐要求为满足输入数据速率，延迟要求为低于 2 秒。作业初始配置同样由本文策略给出。图 4.9 到图 4.10 展示了实验结果。

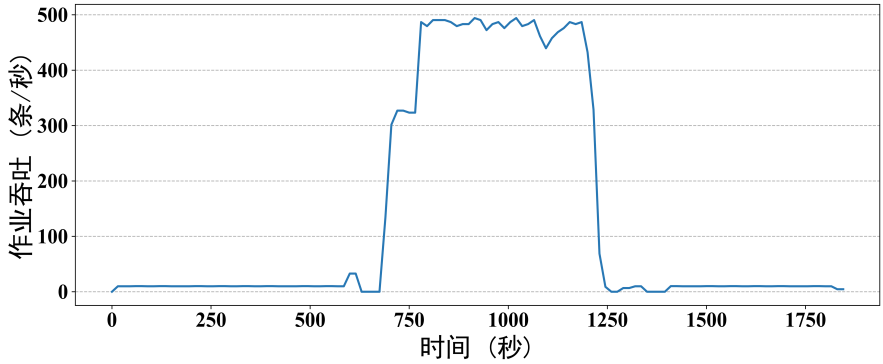


图 4.8 作业吞吐随时间变化情况

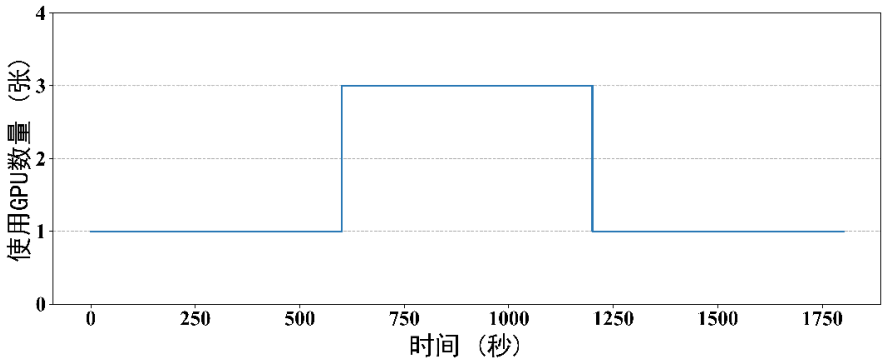


图 4.9 作业使用 GPU 数量随时间变化情况

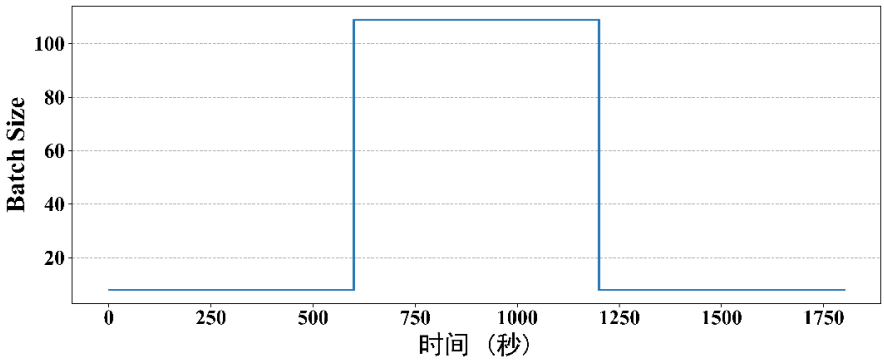


图 4.10 作业 AI 模型推理算子 Batch Size 随时间变化情况

分析图 4.9 到图 4.10 可知，初始时输入数据速率为 10 条/秒，此时作业使用 1 张 GPU（即 AI 模型推理算子并行度为 1）、AI 模型推理算子 Batch Size 为 8，此时作业吞吐能够满足输入数据速率。在 600 秒左右，输入数据速率突增至 500 条/秒，此时本文策略根据新条件重新决策并行配置，此时 AI 模型推理算子的并行度为 3、Batch Size 为 109，重新应用调整后的配置，作业吞吐逐渐上升并基本能够满足输入数据速率。在第 1200 秒左右，输入数据速率重新回落到 10 条/秒，此时本文决策再次调整配置，并将配置重新调整至初始配置，重新应用配置后，作业同样可以满足输入数据速率。

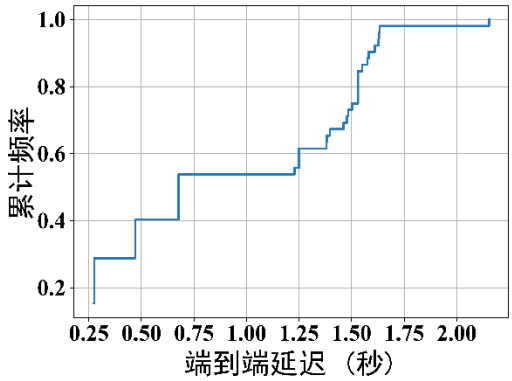


图 4.11 作业端到端延迟 CDF 图

图 4.11 展示了作业运行过程中端到端延迟的累计分布函数（CDF）图，对于给定的 2 秒延迟要求，分析图中结果可知，98%的数据端到端延迟均低于 2 秒，表明本文策略给出的配置可以同样可以满足作业的端到端延迟要求。

4.4.3.2 与其他策略对比

本小节将使用本文策略与并行调整策略 DS2 进行对比。DS2 在进行并行度调整时，会根据算子真正用于处理数据的时间来计算算子所能达到的最大性能，并根据线性模型来预测算子增加并行度后的性能。

在对比过程中，采用了与 4.4.3.1 小节相同的作业、负载场景以及性能要求。初始时两策略的作业配置均由本文策略给出。实验结果如图 4.12 到图 4.14 所示：

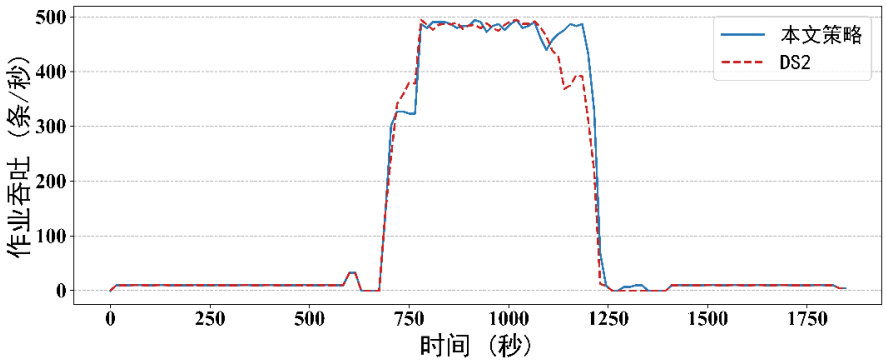


图 4.12 对比本文策略与 DS2 作业吞吐随时间变化情况

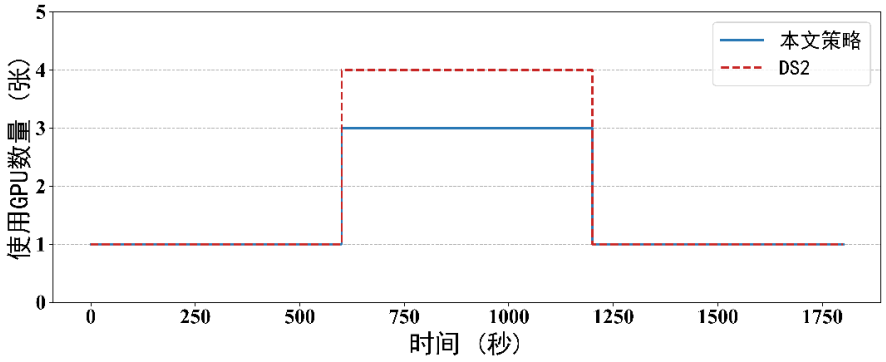


图 4.13 对比本文策略与 DS2 作业使用 GPU 数量随时间变化情况

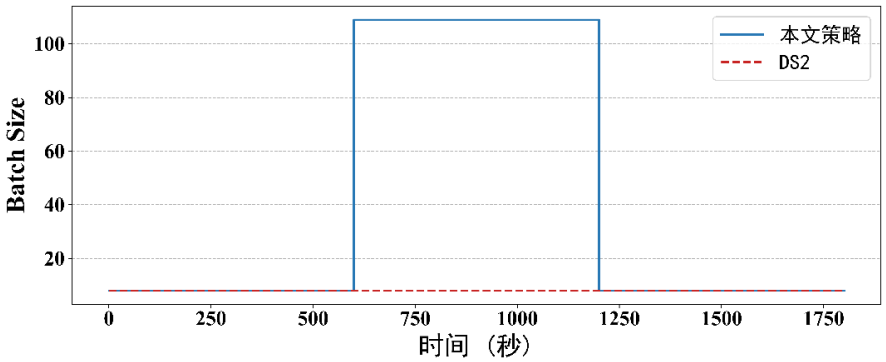


图 4.14 对比本文策略与 DS2 作业 Batch Size 随时间变化情况

分析图 4.12 到图 4.14 可知，在初始时作业配置下，两策略作业吞吐均能满足输入数据速率。在第 600 秒时，输入数据速率突增至 500 条/秒，此时两策略均进行一次配置决策。DS2 策略在进行决策时依据当前算子的最大处理能力，并根据线性模型计算算子并行度，但忽略了 Batch Size 对 AI 模型推理算子性能的影响，因此 DS2 策略给出的配置中，AI 模型推理算子 Batch Size 仍然维持初始值 8、并行度调整至 4（即使用 4 张 GPU）。

相对的，本文策略同时考虑了在延迟与吞吐要求下，适当调整 AI 模型推理算子 Batch Size 从而增加其性能。使用本文策略给出的配置中，AI 模型推理算子的 Batch Size 被调整至 109，此时仅使用 3 张 GPU 即可满足上游输入数据速率，对比 DS2 策略节省了 25%的 GPU 资源。

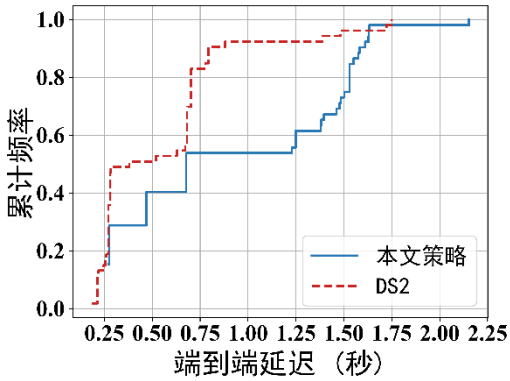


图 4.15 本文策略与 DS2 作业端到端延迟 CDF 图

图 4.15 展示了本文策略与 DS2 策略的作业端到端延迟 CDF 图。分析可知，两策略均能满足作业 2 秒的端到端延迟需求，但 DS2 策略由于没有增加 AI 模型推理算子的 Batch Size，使得等待时间更低，导致其端到端延迟均低于 1.75 秒，而本文策略适当增加了 Batch Size，导致作业端到端延迟对比 DS2 策略更高，但也能够满足 2 秒的端到端延迟需求。

4.5 本章小结

在在线场景下，流式 AI 模型推理作业会面临输入数据速率动态变化的问题，这要求作业能够动态进行资源扩缩容。本章针对这一问题，提出了一种资源扩缩容策略。该策略首先会对作业算子进行轻量化多阶段的性能建模，结合算子性能模型，本章提出了一种基于动态规划的并行优化算法，该算法可以自动化对作业的各个算子决策最优并行配置。最后，本章通过实验验证了所提出性能建模方法与并行优化算法的有效性。

第五章 面向离线流式 AI 模型推理的资源最优化策略

在离线推理场景下，由于通常需要同时使用 CPU 与 GPU，流式执行的模式可以增加资源的利用率。对于离线的流式推理作业，往往不需要考虑作业端到端时延要求，而是追求资源的最优化利用。针对该问题，本章提出了一种面向离线流式 AI 推理场景的资源最优化策略。首先，5.1 小节将对问题背景进行介绍与分析。其次，在 5.2 与 5.3 小节中，将针对成本最优与吞吐最优两类场景介绍相应的资源最优化策略。最后在 5.4 小节中将对本章提出的策略进行实验评估与分析。

5.1 问题背景与分析

在传统的大数据领域中，对于离线作业通常采用批式处理的方式进行计算。但对于包含 AI 模型推理的离线作业，由于通常需要使用 GPU 进行 AI 模型推理加速，即需要同时使用 CPU 与 GPU，使用批式处理的方式可能会导致资源浪费、性能下降等问题。

这是由于在批式执行的过程中，通常采用整体同步并行（**B**ulk **S**ynchronous **P**arallel，BSP）模型[70]。该模型将数据处理分为多个阶段，在每个阶段内由多个实例并行执行，并在各个阶段之间同步等待上一个阶段完成，例如 MapReduce 中的 Map 与 Reduce 阶段，Apache Spark 中的宽依赖概念等。对于使用异构资源的离线推理作业来说，若某个阶段使用 CPU 计算而下个阶段使用 GPU 计算，使用 BSP 模型会导致 CPU 任务与 GPU 任务无法流水线执行，进而导致 GPU 资源浪费以及作业性能的下降。同时，使用 BSP 模型时，对于大规模中间结果数据往往需要写入磁盘等持久化设备，进一步降低了作业的性能。

然而，如果使用流式处理的方式，可以将 CPU 任务与 GPU 任务进行流水线执行，GPU 任务可以无需等待上游 CPU 任务全部完成即可开始处理，有效提高了资源的利用率以及作业的性能。同时，在离线流式推理的场景，往往对数据的时效性没有要求，追求的是对硬件资源的最优化利用。同样，由于流式推理作业通常需要使用 CPU 与 GPU，在配置作业资源时，往往需要面临如下问题：

（1）如何进行合理配置从而达到 GPU 上 AI 模型推理性能的最大化？

GPU 与 CPU 不同，如果在使用 GPU 时，能够积攒一批数据同时处理，往往可以增加 GPU 利用率，提高 GPU 上计算任务的吞吐。如果能够为流式推理作

业选择合适的 Batch Size（即积攒一批数据的数量），从而达到 GPU 的最佳性能，则可以优化资源使用量，降低资源成本开销。

然而，在实际应用中，往往面临着用户使用多种模型以及底层提供多种 GPU 的场景。图 5.1 展示了不同模型在不同 GPU 上使用相同配置时的吞吐。可以看到，对于同一种 GPU 上的不同模型来说，它们的吞吐均不相同，对于同一模型在不同 GPU 上，其吞吐也并不相同。导致该结果的原因是：一方面不同种类的 GPU 具有不同的算力、显存大小与显存带宽等特性，另一方面不同模型本身也具备不同的特性，例如计算密集型模型或访存密集型模型。

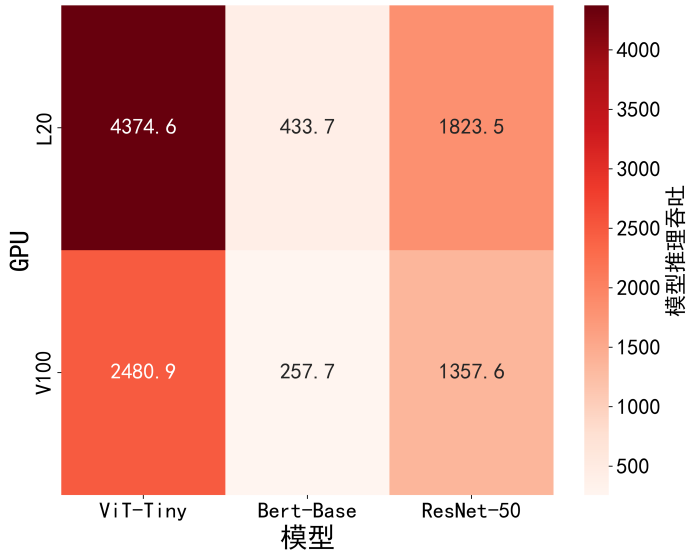


图 5.1 使用相同配置在不同 GPU 上运行不同模型时的吞吐

综上，如何为用户的推理作业选择合适的 Batch Size，从而最优化利用 GPU 成为关键问题。

（2）如何最优化 AI 模型推理作业的分配资源？

流式推理作业使用流水线执行的方式运行，当某个算子的下游算子吞吐远小于当前算子吞吐时，会造成当前算子的吞吐也逐渐下降至下游算子的吞吐（即反压机制）。出现反压后，会导致作业性能下降以及资源浪费。

在为流式推理作业配置资源时，若在总资源量有限的前提下，则需要考虑如何分配资源。例如，假设存在如图 5.2 所示的流式推理作业，若在分配资源时将过多的 CPU 资源分配给了前处理算子，则会导致后处理算子吞吐不足，反压上游推理算子，进而导致 GPU 资源的浪费。

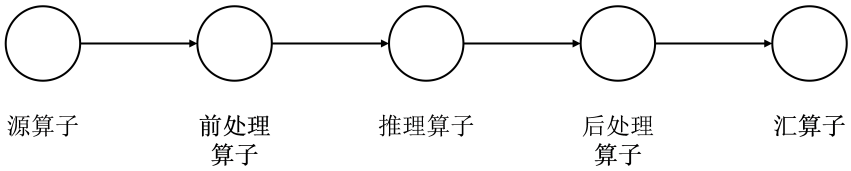


图 5.2 示例流式推理作业

综上，在为流式推理作业配置资源时，也需要考虑如何为各个算子分配资源。不合理的分配方案同样会导致作业吞吐下降以及资源成本增加。

上述两个问题可以与两个常见的用户使用场景相对应：（1）用户给定处理完一批数据的时间限制，要求使用的资源成本最优。（2）用户给定使用资源成本的限制，要求达到最优吞吐。针对这两类场景，本章分别提出了面向成本最优的流式推理作业资源配置方法（5.2 小节）以及面向吞吐最优的流式推理作业资源配置方法（5.3 小节）。

5.2 面向成本最优的流式推理作业资源配置方法

本小节针对用户的使用场景为：给定待处理的数据集以及处理的时间限制，要求使用的资源成本最优。针对该场景，本小节提出了一种面向成本最优的流式推理作业资源配置方法。在 5.2.1 小节中，针对该场景下的问题进行了形式化的描述，并在 5.2.2 小节中介绍了一种基于算子性能模型的资源配置方法。

5.2.1 问题形式化描述

在本小节涉及到的场景中，用户会给定作业J、待处理的数据集大小D_s以及用户要求的时间限制t_{slo}。对于作业J，其算子列表为$O_s = [O_1, \dots, O_n]$，且该列表中的算子按照作业J的拓扑排序顺序排列。对于算子O_i，其并行度为p_i、Batch Size 为b_i、输出输入比为f_i，其在该配置下所能够达到的吞吐为T_i、消耗的 CPU 资源为C_i、消耗的 GPU 资源为G_i，设作业整体可以达到的吞吐为T_J，则该问题可以被形式化描述为：

$$\min \sum_{i=1}^{n} (C_i + \alpha \cdot G_i) \tag{5.1}$$

$$s.t. \quad \frac{D_s}{T_J} \leq t_{slo} \tag{5.2}$$

约束 5.2 表明作业需要能够在用户要求的时间时间内完成处理。其中，作业J整体能够达到的吞吐T_J可以使用如下表达式进行计算：

$$\theta(x) = \begin{cases} 1, & x = 1 \\ \prod_{i=1}^{x} \frac{1}{f_i}, & x > 1 \end{cases} \tag{5.3}$$

$$m = \underset{1 \le i \le n}{\operatorname{argmin}} T_i \cdot \theta(i) \tag{5.4}$$

$$T_J = T_m \tag{5.5}$$

下面对上述计算过程进行解释：由于流式推理作业使用流水线的方式执行，导致作业中性能最低的算子会通过反压机制影响上游算子，并最终影响整体作业的吞吐，所以作业的整体吞吐由性能最低的算子决定。然而，在计算算子性能时，不能仅通过对比算子吞吐进行衡量，这是因为算子可能会对数据进行过滤或扩充，导致每个算子对作业整体的吞吐贡献并不相同。

举例说明，若某个算子的吞吐为 100 条/秒，但其上游算子会过滤 80%的数据，则事实上该算子可以支撑 500 条/秒的上游数据输入速率。相对应的，若某个算子的吞吐为 200 条/秒，但其上游算子不会过滤数据，则其可以支撑的上游数据输入速率为 200 条/秒，小于前者。

为此，公式 5.3 定义了贡献系数函数$\theta(x)$，该函数使用各个算子的输出输入比f_i，计算从源算子到当前第x个算子的吞吐变化系数。当求解算子O_i所能支撑的作业输入数据速率时，即可使用算子所能达到的吞吐T_i与该贡献系数函数$\theta(i)$相乘。公式 5.4 表示所有算子中，能支撑作业输入数据速率最小的算子为O_m，则作业整体的吞吐T_J即为T_m，即公式 5.5。

同时，在优化目标 5.1 中，与 4.3.1 小节中类似，同样引入了系数α，其作用同样为统一 CPU 资源与 GPU 资源。类似的，算子 O_i 的 CPU 资源开销与 GPU 资源开销计算方法为：$C_i = p_i \cdot c_i$以及$G_i = p_i \cdot g_i$。其中c_i与g_i代表算子O_i在并行度为 1 且满负载运行时所需的 CPU 资源与 GPU 资源。对于c_i，如果算子需要积攒一批数据则和其 Batch Size 值b_i有关，反之则为定值。同样，在本小节涉及的场景中，GPU 仍然采用默认的时分复用方式，则$g_i \in \{0,1\}$。对于需要使用 GPU 的算子，则$g_i = 1$，反之则$g_i = 0$。

5.2.2 基于算子性能模型的资源配置方法

在上述约束 5.2 中，可变换得到$T_J \ge D_s / t_{slo}$，即要求作业整体的吞吐大于

D_s/t_{slo}，设该吞吐值为T_{slo}。在面向成本最优的场景下，为流式推理作业进行资源配置的方法总体思路为：针对每个算子O_i，寻找使用资源量最小的配置，使其能达到的吞吐$T_i \geq T_{slo}$。在为每个算子寻找最小资源配置时，需要得知在某种资源配置下算子的吞吐，这里本文同样基于 4.2 小节中介绍的算子性能建模方法，对各个算子进行性能建模，并基于性能模型来进行搜索。其总体流程如下：

算法 5.1 基于算子性能模型的资源配置方法。

输入： 用户给定作业的算子列表$O_s = [O_1, \ldots, O_n]$，其中算子按照拓扑排序顺序排列，要求达到的目标吞吐T_{slo}，GPU 资源与 CPU 资源的比例系数α

输出：更新各个算子的并行度与 Batch Size 配置后的算子列表O_s

```
1.  target_throughput = T_slo
2.  for i = 1 to n do
3.      O_i = O_s[i]
4.      if O_i is batch operator then
5.          O_i.conf = search_minimial_resource(O_i, target_throughput, α)
6.      else
7.          p_i = calculate minimal parallelism of O_i
8.          O_i.conf = (p_i, null)
9.      target_throuhgput = target_throughput * O_i.f
10. return O_s
```

算法 5.1 会按照算子的拓扑排序顺序遍历算子列表，并为每个算子找出满足吞吐要求所需的最小资源配置。在计算每个算子需要满足的吞吐时，会在每次迭代中乘以当前算子的输出输入比（第 9 行），则算子O_i需要满足的吞吐为：

$$T_i \geq T_{slo} \cdot \prod_{k=1}^{i-1} f_k \tag{5.6}$$

等价于结合公式 5.3 计算算子能够支撑的作业输入数据速率。

对于每个算子，需要区分其是否需要积攒数据，若不需要，则仅需根据算子的性能模型计算满足吞吐要求所需的最低并行度即可（第 7 行）。若需要积攒数据，则需要进一步根据性能模型搜索所需资源最小的并行度与 Batch Size（第 5 行）。搜索过程如下：

算法 5.2 搜索算子资源开销最小的配置方案（函数 search_minimial_resource）

输入：待搜索的算子O_i，要求达到的目标吞吐T_{target}，GPU 资源与 CPU 资源的比例系数α

输出：资源开销最小时的并行度p_i与 Batch Size b_i

```
1.  min_resource_usage = +∞
```

2. p_i = none
3. b_i = none
4. **for** bs = 1 to O_i.batch_limit **do**
5. p_c = 1
6. T_c = O_i.perf_model(p_c, bs)
7. T_p = T_c
8. **while** $T_c \geq T_p$ **do**
9. **if** $T_c \geq T_{target}$ **then**
10. resource_usage = O_i.cpu_usage_model(bs) + $\alpha \cdot O_i$.gpu_usage
11. **if** resource_usage < min_resource_usage **then**
12. $p_i = p_c$
13. $b_i = bs$
14. **break**
15. $T_p = T_c$
16. $p_c = p_c + 1$
17. T_c = O_i.perf_model(p_c, bs)
18. **return** (p_i, b_i)

在算法 5.2 中，会在算子O_i的 Batch Size 配置范围内搜索（第 4 行），该信息在对算子进行性能建模时可以获取到。对于每个可能的 Batch Size，会从小到大搜索能够满足目标吞吐的并行度（第 8 至 17 行），当搜索到满足目标吞吐的并行度时，会计算该配置下的资源开销，并更新最优资源配置方案（第 10 至 13 行）。最终该算法会返回在所有可能的配置方案中，能够满足目标吞吐且资源开销最小的配置方案（第 18 行）。

5.3 面向吞吐最优的流式推理作业资源配置方法

本小节针对用户的使用场景为：给定计算资源限制，要求作业在该资源限制下达到最优吞吐。针对该场景，本小节提出了一种面向吞吐最优的流式推理作业资源配置方法。在 5.3.1 小节中，针对该场景下的问题进行了形式化的描述，并在 5.3.2 小节中介绍了一种基于瓶颈点分析的资源配置方法。

5.3.1 问题形式化描述

在本小节涉及到的场景中，用户会给定作业J、CPU 资源限制C_{limit}（CPU 核数）、GPU 资源限制G_{limit}（GPU 张数）。对于作业 J，其算子列表为$O_s = [O_1, \dots, O_n]$，且该列表中的算子按照作业J的拓扑排序顺序排列。对于算子O_i，其并行度为p_i、

Batch Size 为b_i、输出输入比为f_i，其在该配置下能够达到的吞吐为T_i、消耗的 CPU 资源为C_i、消耗的 GPU 资源为G_i，作业整体可以达到的吞吐为T_J，则问题可以形式化描述为：

$$\max\ T_J \tag{5.7}$$

$$s.t.\ \sum_{i=1}^{n} C_i \le C_{limit} \tag{5.8}$$

$$\sum_{i=1}^{n} G_i \le G_{limit} \tag{5.9}$$

其中，优化目标 3.7 表示需要最大化作业的吞吐，其中作业的吞吐T_J计算方式与公式 3.3 到 3.5 一致。对于约束 3.8 与 3.9 中的算子 CPU 资源开销计算方式与 GPU 资源开销计算方式为：$C_i = p_i \cdot c_i$以及$G_i = p_i \cdot g_i$。其中c_i与g_i代表算子O_i在并行度为 1 且满负载运行时所需的 CPU 资源与 GPU 资源。对于c_i，如果算子需要积攒数据则和其 Batch Size 有关，反之则为定值。同样在本小节中，GPU 仍然采用时分复用模式，因此$g_i \in \{0,1\}$。对于需要使用 GPU 的算子则$g_i = 1$，若不需要使用 GPU 则$g_i = 0$。

5.3.2 基于瓶颈点分析的资源配置方法

在本小节涉及到的场景中，用户给定了有限的资源，需要找到一种资源配置方案使得作业能够达到的吞吐最大。作业的整体吞吐计算方式为公式 5.3 至 5.5，计算过程的核心为：作业中性能最低的算子决定了作业的整体吞吐，则在分配资源时，若将资源分配给非性能最低的算子，对于作业整体的吞吐并没有提升。

因此，本小节提出了一种基于瓶颈点分析的资源配置方法，其核心思想为：分析寻找作业中性能最低的算子（即瓶颈点），并向其增加资源直到其不再是瓶颈点，迭代这一过程直至资源耗尽。算法的总体流程如下：

算法 5.3 基于瓶颈点分析的资源配置方法

输入：用户给定作业的算子列表$O_s = [O_1, \dots, O_n]$，其中算子按照拓扑排序顺序排列，CPU 资源限制C_{limit}，GPU 资源限制G_{limit}

输出：更新各个算子的并行度与 Batch Size 配置后的算子列表O_s

1. H_{min} = Initialize an empty min-heap, $\theta = 1$
2. **for** $i = 0$ to size(O_s) **do**
3. $e =$ initialize an element in H_{min}

4. $e.\theta = \theta$
5. $e.O = O_s[i]$
6. $e.position = i$
7. $e.throughput = 0$
8. // compare element e in H_{min} with $e.\theta \cdot e.throughput$ and $e.position$
9. H_{min}.add(e)
10. $\theta = \theta \cdot 1/O_s[i]$.f
11. **while** $C_{limit} > 0$ && $G_{limit} > 0$ **do**
12.
13. $e = H_{min}$.pop()
14. $S, C_{limit}, G_{limit}, t_{new}$ = add_resource($e.O, C_{limit}, G_{limit}$)
15. **if** !S **then**
16. **break**
17. $e.throughput = t_{new}$
18. H_{min}.add(e)
19. **return** O_s

算法使用了数据结构小根堆来快速寻找瓶颈点算子，初始时，遍历算子列表并向小根堆中插入元素（第 1 至 11 行），元素内容为一个对象，其属性包含算子对象本身、算子当前配置下的吞吐$throughput$、算子在作业拓扑中的位置以及参照公式 5.3 计算的贡献系数θ。插入元素的过程中，小根堆会对元素进行排序，排序过程中，元素的比较规则为：优先比较算子能够支撑的输入数据速率，即$\theta \cdot throughput$，若一致则继续比较算子在拓扑图中的位置索引，即性能一致的情况下，优先选择处于上游的算子。初始化完成后，则迭代寻找作业中的瓶颈点算子，并调用函数 add_resource 逐步向瓶颈点算子添加资源，直至资源耗尽。

在 add_resource 函数中，将为传入算子的并行度加 1，并根据算子的类型分配相应的资源。若算子无需积攒数据，则可直接根据其单并行度下的 CPU 与 GPU 资源开销以及剩余资源判断是否足够分配（第 22 至 28 行）。若算子需要积攒数据，则遍历其 Batch Size 取值范围，寻找在剩余可分配资源范围内的最大吞吐配置（第 29 至 39 行）。

5.4 实验评估与分析

本小节将对本章提出的面向离线流式推理场景的资源最优化策略进行评估。首先，5.4.1 小节将对实验环境与配置进行介绍，包含软硬件环境、工作负载以及性能评估指标。接着，5.4.2 小节与 5.4.3 小节将分别对面向成本最优以及面向吞

吐最优的流式推理作业资源配置方法进行有效性评估。

5.4.1 实验环境与配置

5.4.1.1 软硬件环境

在本节实验中，使用的均为某互联网头部社交媒体平台提供的云环境，该环境可以自由对资源进行选择与配置。在本次实验中，使用的 GPU 型号为 NVIDIA V100 并配备 32GB 显存，CPU 型号为 AMD EPYC 9K84 CPU @ 2.6GHz。当资源配置完成后，云环境将自动拉起对应的资源集群，且集群间均使用 10Gbps 的以太网进行连接。

软件方面，使用的 NVIDIA CUDA 版本为 12.1.1，并使用了 Tensorflow 2.15.1 作为 AI 模型推理框架。底层计算引擎则使用了 Ray Klein v2.30，其基于开源版本的 Ray v2.30.0 进行改造，在离线场景下该引擎实质上仍然为使用 Ray Data。

5.4.1.2 工作负载

实验过程中，使用了某互联网头部社交媒体平台线上真实运行的作业，其主要流程如下：作业读取 TFRecords 文件，并送入到 AI 模型推理算子，该算子使用了 MMoE（Multi-gate Mixture-of-Experts）架构的深度推荐模型进行推理，并将推理结果输入到后处理算子中。后处理算子将对输入数据进行正则化处理，并通过 Sink 算子将结果写入到外部 Parquet 文件中。实验数据方面，同样使用某互联网头部社交媒体平台内部的真实线上数据，实验过程中将其截取为包含 1 千万条与 4 千万条数据的不同大小数据集。

5.4.1.3 性能评估指标

本小节实验中主要考虑以下两个指标：

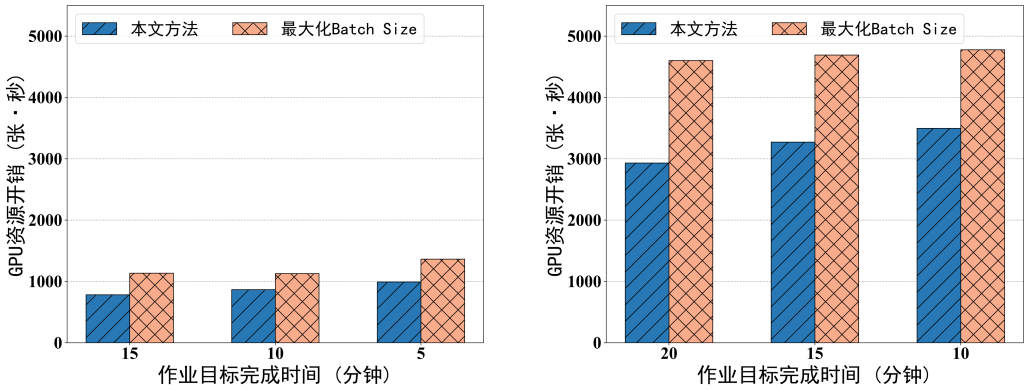
（1） 作业 GPU 资源开销：指作业占用 GPU 的数量与占用时间的乘积，反应了作业消耗 GPU 资源的情况，单位为（张 · 秒）。

（2） 作业吞吐：指作业每秒钟处理数据的条数，在实验过程中使用数据集大小除以作业完成时间进行计算，反应了作业的性能。

5.4.2 面向成本最优的资源配置方法有效性评估

本小节实验中，在不同大小数据集以及不同作业完成时间目标下对本文提出

的面向成本最优的资源配置方法进行了评估。评估过程中对比了一种在离线场景下常见基于调参经验的自动调参方法，即尽可能使用较大的 Batch Size，再进一步寻找能够满足完成时间要求的并行度配置（称其为最大化 Batch Size）。评估指标使用了作业 GPU 资源开销来对两方法进行评估。实验结果如图 5.3 所示。



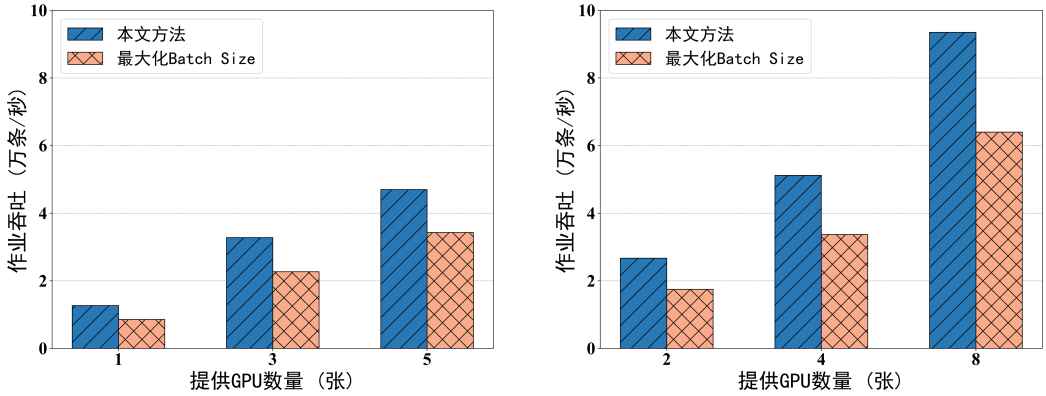
(a) 数据集大小为 1 千万条 **(b) 数据集大小为 4 千万条**

图 5.3 不同数据集与目标完成时间下作业 GPU 资源开销对比

在实验过程中，使用本文方法配置的作业均能在目标完成时间内完成。同时，采用最大化 Batch Size 策略进行配置时，使用的 GPU 资源为能稳定在目标完成时间内完成的最小资源。分析图 5.3 (a) 和 (b) 可知，本文方法给出的资源参数配置相较于最大化 Batch Size 策略可以节省最多约 30%的 GPU 资源。这是由于推理框架自身实现了显存分配与管理机制，过大的 Batch Size 可能导致频繁的显存管理开销，进而导致推理性能下降。而本文方法则结合算子性能模型给出了性能更优的显存配置，从而在相同数据集大小与目标完成时间条件下使用了更少的 GPU 资源。

5.4.3 面向吞吐最优的资源配置方法有效性评估

本小节实验同样使用了 1 千万条和 4 千万条两种大小的数据集，并且给定了不同的资源量。在资源量方面，采用的策略为 GPU 资源:CPU 资源:内存比例固定为 1:10:40，即每分配一张 GPU，则分配 10 核 CPU 以及 40GB 内存。本小节实验同样对比了 5.4.2 小节中提到的基于调参经验的自动调参方法，即尽可能使用较大的 Batch Size。评估指标方面对比了作业吞吐。实验结果如图 5.4 所示。



(a) 数据集大小为 1 千万条 **(b) 数据集大小为 4 千万条**

图 5.4 不同数据集与资源量下作业吞吐对比

分析图 5.4 (a) 和 (b) 可知，本文方法在不同数据集大小以及给定资源量下，均取得了比最大化 Batch Size 策略更高的作业吞吐，其中最高超过了约 45%。这同样是由于推理框架自身的显存管理机制导致的，而本文方法通过向作业瓶颈点迭代添加资源并选择性能最佳的配置，从而达到了更高的性能。

5.5 本章小结

在离线流式 AI 模型推理场景下，由于无需考虑作业的端到端时延，用户往往追求的是对计算资源的充分利用。本文分别针对成本最优与吞吐最优两类场景，提出了相应的资源配置方法。该方法会结合作业算子的性能模型以及作业的拓扑结构，对作业求解最优配置方案。最后对提出的资源配置方法进行了有效性评估。

第六章 系统架构与实现

本章将介绍本文提出的流式 AI 模型推理作业自动优化系统的具体实现。该系统集成了第三章、第四章以及第五章中提出的优化策略和方法。6.1 小节将介绍系统的整体架构，6.2 小节将针对架构中的各个组件进行进一步详细介绍，6.3 小节将介绍系统的使用流程。

6.1 系统架构

本文提出的流式 AI 模型推理作业自动优化系统的整体架构如图 6.1 所示。本文涉及到的工作主要集中于性能建模模块、配置求解模块以及内嵌在引擎运行时中的缓存优化策略。

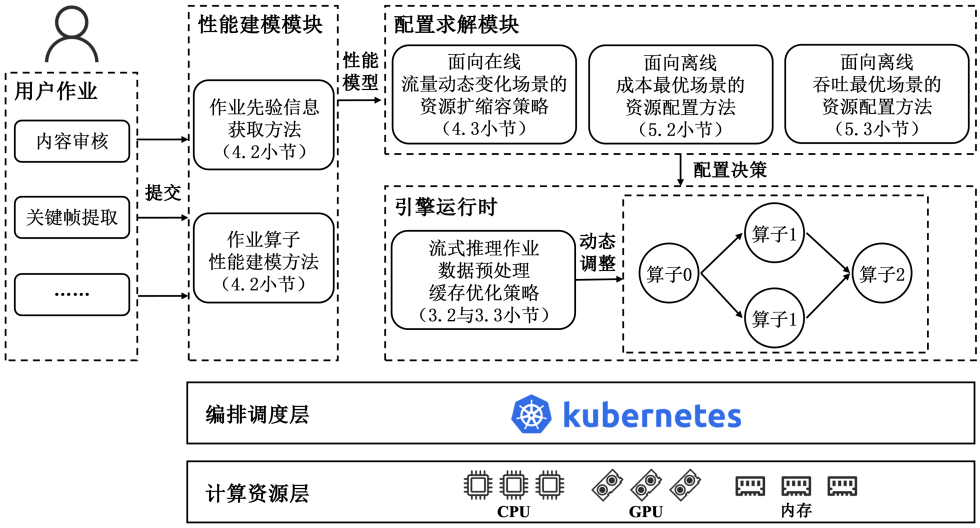


图 6.1 流式 AI 模型推理作业自动优化系统整体架构

首先，用户将编写好的各类流式 AI 模型推理作业提交后，会首先经由性能建模模块对其进行算子性能建模。建模过程与 4.2 小节中介绍的一致，首先需要获取作业的各类先验信息，再根据作业先验信息对算子进行实际运行、采集性能数据与建模。实际算子运行时需要使用各类计算资源，例如 CPU、GPU 等，则需要通过编排调度层来进行计算资源的申请与获取，这里使用的是当前云平台环境最常用的 Kubernetes 组件。

其次，当对用户作业进行完算子性能建模后，配置求解模块将根据算子性能模型以及性能目标进行配置求解。当前系统针对三类场景提出了相应的配置求解算法，分别为在线作业流量动态变化的场景、离线作业成本最优的场景以及离线

作业吞吐最优的场景，对应了本文的第 4.3 小节、5.2 小节与 5.3 小节。

最后，当配置求解模块得出作业配置后，引擎运行时将应用给出的最新配置。同时，本文第三章提出的缓存优化策略被嵌入到了引擎运行时中，在作业运行期间将持续监控流状态分布情况，并动态决策最优缓存容量。在作业运行过程中，同样需要申请各类计算资源，这里同样为与编排调度层进行交互。

6.2 组件实现与运行流程

本小节将具体介绍本文提出的流式 AI 模型推理作业自动优化系统中各个组件的具体实现与运行流程。首先，6.2.1 小节将介绍作业先验信息获取方法的实现与工作流程，接着 6.2.2 小节将介绍作业算子性能建模方法的实现与工作流程，然后 6.2.3 小节将介绍作业配置求解模块的实现与工作流程，最后 6.2.4 小节将介绍作业缓存优化策略的实现与工作流程。

6.2.1 作业先验信息获取方法

作业先验信息获取方法属于作业性能建模模块，其作用为在对作业进行正式性能建模前，获取建模所需的必要信息，也即先验信息。该方法的运行总体流程如图 6.2 所示。

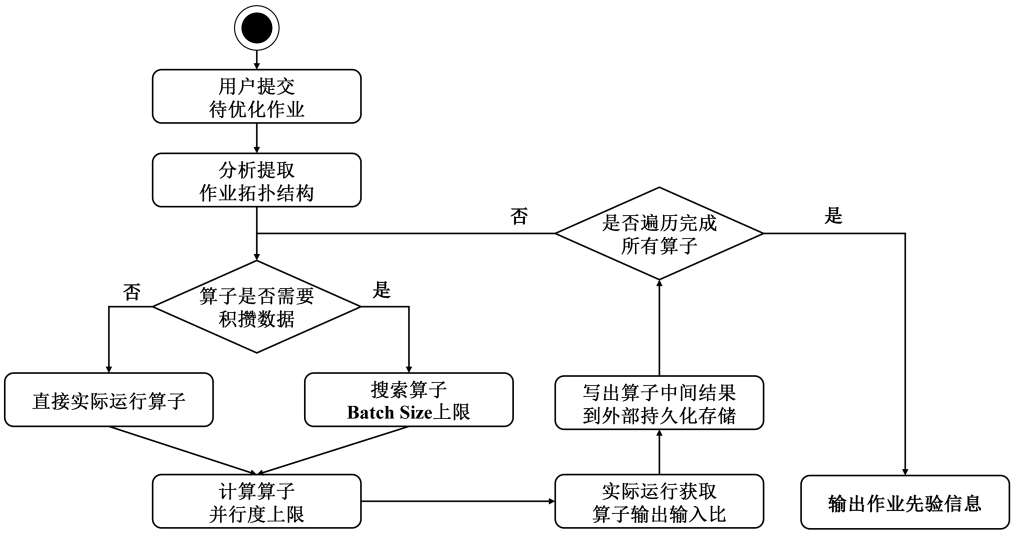


图 6.2 作业先验信息获取方法运行流程

当用户提交待优化作业后，本文方法将会对用户作业进行分析，并提取作业的拓扑结构，形成一组按作业拓扑结构组织的算子对象。每个对象中会包括该算

子的实际运行方法以及算子的各类信息，例如该算子是否需要积攒数据、运行时是否需要 GPU 等。

接着将对各个算子按拓扑排序顺序进行遍历，针对每个算子依次获取所需的先验信息：（1）若算子需要积攒数据，则需要搜索 Batch Size 上限并收集性能数据，否则直接运行获取性能数据；（2）根据算子性能数据以及优化目标初步计算并行度上限；（3）使用算子处理一批数据并统计输出输入比。

此外，当获取到所需先验信息后，还需要将算子产生的中间结果写入到外部持久化存储系统，这一步的目的是为后续待运行算子生成输入数据，从而可以做到仅运行单个算子，避免运行作业整体，节省了计算资源。

当遍历完所有算子后，会将获取到的先验信息输出，输出内容即为对用户作业进行分析后形成的一组按作业拓扑结构组织的算子对象。其中，本方法获取到的各类先验信息也会赋值到各个算子对象中。

6.2.2 作业算子性能建模方法

作业算子性能建模方法同样属于作业性能建模模块，其作用为针对用户待优化的作业构建算子级别性能模型。具体而言，该方法内部包括两个方法，分别为性能数据收集方法以及算子性能建模方法，运行流程如图 6.3 所示。

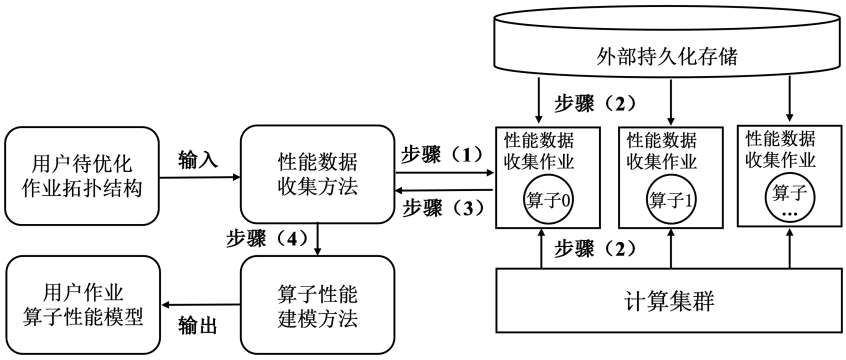


图 6.3 作业算子性能建模方法运行流程

该方法运行时主要包括以下几个步骤：

步骤（1）：当接收到获取过先验信息的用户作业拓扑结构后，性能数据收集方法会根据算子先验信息启动算子级别的性能数据收集作业。

步骤（2）：算子级别的性能数据收集作业将运行在预先申请好的计算集群中，同时在运行过程中从外部持久化存储中读取输入数据。在这一过程中，每

个单独的性能数据收集作业都只运行一个单独的算子，且多个作业直接可共同运行，可以加快性能数据的收集过程。

步骤（3）：性能数据收集方法会不断收集各个作业的性能数据，并形成各个算子的性能数据集。

步骤（4）：算子性能建模方法会根据各个算子的性能数据集，结合机器学习方法，为每个算子构建对应的性能模型，并同样赋值到作业拓扑结构中的算子对象内。当对每个算子都完成性能建模后，可以输出作业的算子性能模型。

6.2.3 作业配置求解模块

作业配置求解模块负责根据配置求解请求内容，自动选择相应的求解算法，并自动化给出对应的配置。其运行流程如图 6.4 所示。

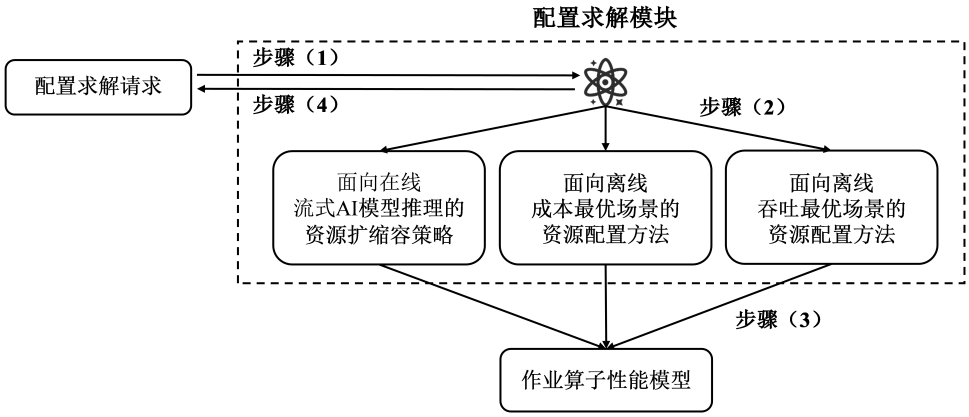


图 6.4 配置求解模块运行流程

该模块的具体运行流程如下：

步骤（1）：从配置求解请求中解析待求解的场景，以及性能目标相关参数。这一步骤的请求可能来自于新作业上线前的配置求解请求，或者已上线在线作业的动态调整请求。

步骤（2）：求解请求的场景主要包括在线流量动态变化场景、离线成本最优场景以及离线吞吐最优场景。在解析出请求场景后，会自动选择对应的配置求解算法进行求解。

步骤（3）：在配置求解算法求解过程中，需要获取算子在某配置下的性能表现，此时需要使用提前构建好的算子性能模型进行预测得出。

步骤（4）：当求解出各个算子对应的配置后，将其赋值到用户作业拓扑结构中的算子对象中并返回。

6.2.4 作业数据预处理缓存优化策略

作业数据预处理缓存优化策略部分负责根据用户性能目标，针对作业预处理部分所需的缓存进行动态调整，从而避免性能下降或资源浪费。该部分被嵌入到了引擎的运行时中，将随着作业一同运行，其运行流程如图 6.5 所示。

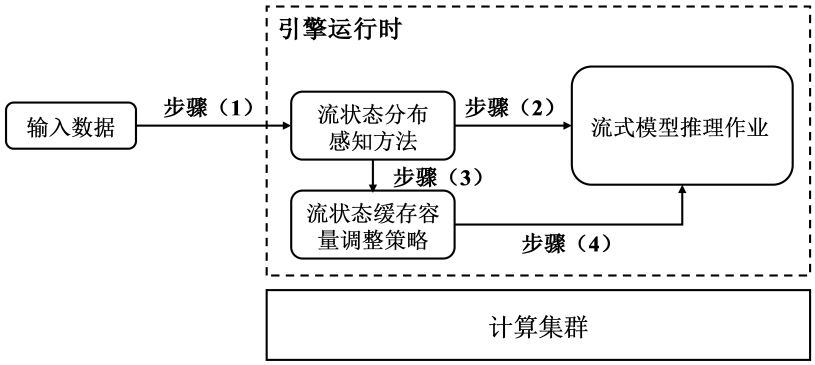


图 6.5 作业数据预处理缓存优化策略运行流程

该策略的具体运行流程如下：

步骤（1）：当外界数据输入时，会先交由本策略中的流状态分布感知方法进行记录，从而感知流状态的分布变化情况。

步骤（2）：外界数据被记录后，再交由用户作业的处理逻辑进行处理。

步骤（3）：当时间窗口结束后，流状态分布感知方法将根据当前的分布情况，决定是否需要进行流状态缓存容量调整。

步骤（4）：若需要进行调整，流状态缓存容量调整策略将基于作业处理语义以及流状态分布情况，决策新的缓存容量，并对用户作业进行调整。

6.3 系统使用流程

本小节将从用户视角介绍本系统的使用流程。本系统面向的用户为 AI 模型推理作业的编写者，在使用时，用户首先需要根据业务处理流程编写 AI 模型推理作业，一个典型的 AI 模型推理作业示例如图 6.6 所示：

```
stream_job = ctx.read_kafka(
    # 用户需配置Topic、地址等信息
    num_cpus=1,
    concurrency=4,
).map(
    DataProcess,
    fn_constructor_kwargs={'download_model_params': params},
    name="data_process",
    num_cpus=1,
    concurrency=4,
).map_batches(
    ModelInferOnline,
    fn_constructor_kwargs={'download_model_params': params},
    name="model_inference",
    num_cpus=1,
    num_gpus=1,
    concurrency=3,
    batch_size=128,
).map_batches(
    BatchPostProcessor,
    name="batch_post_process",
    num_cpus=1,
    concurrency=2,
    batch_size=100,
).write_kafka(
    # 用户需配置Topic、地址等信息
    num_cpus=1,
    concurrency=4,
)
```

图 6.6 用户编写的流式 AI 模型推理示例

用户编写好 AI 模型推理作业后，本系统提供了一键式 API，用户可以调用该 API 并传入相应参数，包括调优场景、调优目标，图 6.7 展示了对一个在线流式 AI 模型推理作业进行调优的 API 调用方法。

```
schema = profiling(stream_job, 'nearline', 10000, 2000)
```

图 6.7 一键式 API 使用示例

当调用该 API 后，用户作业以及相关参数将被打包上传至本文系统中，接着系统将对用户作业进行性能建模、配置求解等过程，并返回如图 6.8 所示的优化后的配置参数，用户即可使用该配置运行作业。

```
{
    "source": {
        "num_cpus": 0.5,
        "concurrency": 2
    },
    "data_process": {
        "num_cpus": 1,
        "concurrency": 2
    },
    "model_inference": {
        "num_cpus": 1,
        "num_gpus": 1,
        "concurrency": 3,
        "batch_size": 128
    },
    "batch_post_process": {
        "num_cpus": 1,
        "concurrency": 2,
        "batch_size": 100
    },
    "sink": {
        "num_cpus": 0.5,
        "concurrency": 2
    }
}
```

图 6.8 系统返回配置方案示例

6.4 本章小结

本章介绍了本文提出的流式 AI 模型推理作业自动优化系统的具体实现。首先，针对该系统的系统整体架构进行了详细介绍。接着，针对系统中涉及到的各个模块与组件分别介绍了其实现与运行流程。最后，从用户视角介绍了本文系统的使用流程。

第七章 总结与展望

7.1 本文工作总结

随着人工智能技术的飞速发展，越来越多的企业开始将人工智能技术应用到其业务场景中，这也直接推动了对人工智能 AI 模型推理需求的大幅提升。面对日益复杂的应用场景，以 Ray 为代表的分布式流式处理引擎开始受到关注。通过使用这类引擎，用户能够将复杂业务逻辑整合为单一的流式 AI 模型推理作业，并通过分布式并行处理显著提升推理性能。然而，使用分布式流式处理引擎运行 AI 模型推理作业仍存在若干问题。

首先，在当前主流的云平台环境下，当输入数据动态变化后，如何为数据预处理缓存快速、准确分配容量成为难题；其次，针对在线流式 AI 模型推理作业，在输入数据速率动态变化时，难以实现快速准确的资源扩缩容决策；最后，离线流式 AI 模型推理作业，由于输入数据集有限，如何实现全链路资源配置优化以最大化利用计算资源亟待解决。

针对上述挑战，本文聚焦流式 AI 模型推理场景下的计算资源优化问题，设计并实现了一套自动化优化系统。本文的主要贡献包括：

（1）针对流式 AI 模型推理数据预处理阶段缓存容量预估难的问题，本文首先提出了一种基于近似数据结构的轻量级流状态分布感知方法，能够高效实时地监测数据分布特征。同时结合基于处理语义信息的缓存容量调整策略，本文方法能够动态优化缓存分配，从而避免作业性能下降以及缓存资源浪费。

（2）针对在线流式 AI 模型推理作业高效资源扩缩容难的问题，本文首先对作业进行轻量化多阶段的算子性能建模，并结合本文提出的一种基于动态规划的配置求解算法，可以做到对作业进行高效、快速的资源扩缩容决策。

（3）针对离线流式 AI 模型推理作业最优化资源配置难的问题，分别设计了面向成本最优与吞吐最优的资源最优化配置方法。基于作业拓扑结构与作业算子性能模型，本文方法可自动化求解全局最优资源配置方案。

（4）基于以上研究，本文设计实现了一个流式 AI 模型推理作业自动优化系统。实验结果表明，在输入数据动态变化的场景下，对比静态缓存容量方法，本文的数据预处理缓存优化方法可以通过动态调整缓存容量，从而提升约 20%作业性能。同时，本文的资源扩缩容策略，对比 DS2 算法，可以在满足性能以及端

到端时延的前提下节省约 25%的 GPU 资源开销。最后，对于离线流式 AI 模型推理作业的资源最优化方法，对比基于主流调优经验的自动化调参方法，可以降低约 30%的 GPU 资源并提高约 45%的作业性能。

7.2 进一步工作

本文的未来工作考虑将从以下几个方面展开：

（1）当前工作中，基于作业处理语义信息提出了预处理缓存容量调整策略。但本文中考虑的处理语义信息为流式处理中常见的处理逻辑，例如聚合算子，如何针对用户自定义的处理逻辑进行分析成为可能的未来工作方向。例如，结合程序静态分析方法或前沿的人工智能方法对用户处理逻辑进行自动化语义分析。

（2）对于作业性能模型，当前采用的方法为将作业算子视为黑盒并进行性能数据采样以及建模，这会导致每次新作业上线前均需要重新进行建模。如何利用并泛化已经收集到的历史数据成为可能的未来工作方向。例如，对 AI 模型推理内部的计算操作进行建模并复用[71]，当新模型到来时可以根据模型结构直接分析得出性能预测，无需再次进行性能采样。

参考文献

[1] DeepSeek-AI, Guo D, Yang D, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning[J]. arXiv preprint arXiv:2501.12948, 2025.

[2] DeepSeek-AI, Liu A, Feng B, et al. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model[J]. arXiv preprint arXiv:2405.04434, 2024.

[3] DeepSeep-AI, Liu A, Feng B, et al. DeepSeek-V3 Technical Report[J]. arXiv preprint arXiv:2412.19437, 2025.

[4] OpenAI, Achiam J, Adler S, et al. GPT-4 Technical Report[J]. arXiv preprint arXiv:2303.08774, 2024.

[5] Sun X, Chen Y, Huang Y, et al. Hunyuan-Large: An Open-Source MoE Model with 52 Billion Activated Parameters by Tencent[J]. arXiv preprint arXiv:2411.02265, 2024.

[6] Qwen, Yang A, Yang B, et al. Qwen2.5 Technical Report[J]. arXiv preprint arXiv:2412.15115, 2025.

[7] Bai J, Bai S, Chu Y, et al. Qwen Technical Report[J]. arXiv preprint arXiv:2309.16609, 2023.

[8] Rombach R, Blattmann A, Lorenz D, et al. High-Resolution Image Synthesis With Latent Diffusion Models[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2022: 10684-10695.

[9] 中国算力发展指数白皮书（2023年）[EB/OL]. https://www.caict.ac.cn/kxyj/qwfb/bps/202309/t20230914_461823.htm.

[10] Trends in Training Dataset Sizes[EB/OL]. https://epoch.ai/blog/trends-in-training-dataset-sizes.

[11] Will We Run Out of Data? Limits of LLM Scaling Based on Human-Generated Data[EB/OL]. https://epoch.ai/blog/will-we-run-out-of-data-limits-of-llm-scaling-based-on-human-generated-data

[12] 人工智能发展报告（2024年）[EB/OL]. https://www.caict.ac.cn/kxyj/qwfb/bps/202412/t20241210_647283.htm.

[13] Ma J, Zhao Z, Yi X, et al. Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts[C]//Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD). 2018: 1930-1939.

[14] Li J, Li D, Xiong C, et al. BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation[C]//Proceedings of the International Conference on Machine Learning (ICML). 2022: 12888-12900.

[15] Li J, Li D, Savarese S, et al. BLIP-2: bootstrapping language-image pre-training with frozen image encoders and large language models[C]//Proceedings of the International Conference on Machine Learning (ICML). 2023: 19730-19742.

[16] Radford A, Kim J W, Hallacy C, et al. Learning Transferable Visual Models From Natural Language Supervision[J]. arXiv preprint arXiv:2103.00020, 2021.

[17] Kwon W, Li Z, Zhuang S, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention[C]//Proceedings of the Symposium on Operating Systems Principles (SOSP). 2023: 611-626.

[18] Zheng L, Yin L, Xie Z, et al. SGLang: Efficient Execution of Structured Language Model Programs[J]. Advances in Neural Information Processing Systems, 2024, 37: 62557-62583.

[19] LMDeploy [EB/OL]. https://lmdeploy.readthedocs.io/en/latest.

[20] TensorRT-LLM[EB/OL]. https://docs.nvidia.com/tensorrt-llm/index.html.

[21] Yu G I, Jeong J S, Kim G W, et al. Orca: A Distributed Serving System for Transformer-Based Generative Models[C]// Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2022: 521-538.

[22] Agrawal A, Kedia N, Panwar A, et al. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve[C]// Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2024: 117-134.

[23] NVIDIA Triton Inference Server[EB/OL]. https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/index.html.

[24] Paszke A, Gross S, Massa F, et al. PyTorch: an imperative style, high-performance deep learning library[C]//Proceedings of the International Conference on Neural Information

Processing Systems (NIPS). 2019: 8026-8037.
[25] Abadi M, Barham P, Chen J, et al. TensorFlow: a system for large-scale machine learning[C]//Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2016: 265-283.
[26] NVIDIA TensorRT[EB/OL]. https://docs.nvidia.com/deeplearning/tensorrt/latest/index.html.
[27] Moritz P, Nishihara R, Wang S, et al. Ray: a distributed framework for emerging AI applications[C]//Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2018: 561-577.
[28] Offline Batch Inference | Ray, Apache Spark & SageMaker[EB/OL]. https://www.anyscale.com/blog/offline-batch-inference-comparing-ray-apache-spark-and-sagemaker.
[29] FLIP-423: Disaggregated State Storage and Management[EB/OL]. https://cwiki.apache.org/confluence/pages/viewpage.action?pageId=293046855.
[30] Robinson B, Power R, Cameron M. A sensitive Twitter earthquake detector[C]//Proceedings of the International Conference on World Wide Web (WWW). 2013: 999-1002.
[31] Fragkoulis M, Carbone P, Kalavri V, et al. A survey on the evolution of stream processing systems[J]. The VLDB Journal (VLDBJ), 2024, 33(2): 507-541.
[32] Rastegar S H, Abbasfar A, Shah-Mansouri V. Rule Caching in SDN-Enabled Base Stations Supporting Massive IoT Devices With Bursty Traffic[J]. IEEE Internet of Things Journal (IOTJ), 2020, 7(9): 8917-8931.
[33] Xu D, Liu X, Vasilakos A V. Traffic-Aware Resource Provisioning for Distributed Clouds[J]. IEEE Cloud Computing (CLOUD), 2015, 2(1): 30-39.
[34] Islam S, Venugopal S, Liu A. Evaluating the impact of fine-scale burstiness on cloud elasticity[C]//Proceedings of the ACM Symposium on Cloud Computing (SoCC). 2015: 250-261.
[35] Somashekar G, Tandon K, Kini A, et al. OPPerTune: Post-Deployment Configuration Tuning of Services Made Easy[C]//Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI). 2024: 1101-1120.
[36] Karthikeyan A, Natarajan N, Somashekar G, et al. SelfTune: Tuning Cluster Managers[C]// Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI). 2023: 1097-1114.
[37] Wu Y, Tan K L. ChronoStream: Elastic stateful stream computation in the cloud[C]// Proceedings of the IEEE International Conference on Data Engineering (ICDE). 2015: 723-734.
[38] Song W W, Um T, Elnikety S, et al. Sponge: Fast Reactive Scaling for Stream Processing with Serverless Frameworks[C]//Proceedings of the USENIX Annual Technical Conference (ATC). 2023: 301-314.
[39] Mao Y, Chen Z, Zhang Y, et al. StreamOps: Cloud-Native Runtime Management for Streaming Services in ByteDance[C]//Proceedings of the VLDB Endowment (VLDB). 2023, 16(12): 3501-3514.
[40] Wang Y, Liu Z. A sneak peek at RisingWave: a cloud-native streaming database[C]//Proceedings of the ACM International Conference on Distributed and Event-Based Systems (DEBS). 2022: 190-193.
[41] O'Neil P, Cheng E, Gawlick D, et al. The log-structured merge-tree (LSM-tree)[J]. Acta Inf., 1996, 33(4): 351-385.
[42] Xu L, Saxena D, Yadwadkar N J, et al. Dirigo: Self-scaling Stateful Actors For Serverless Real-time Data Processing[J]. arXiv preprint arXiv:2308.03615, 2023.
[43] Floratou A, Agrawal A, Graham B, et al. Dhalion: self-regulating stream processing in heron[C]//Proceedings of the VLDB Endowment (VLDB). 2017, 10(12): 1825-1836.
[44] Kalavri V, Liagouris J, Hoffmann M, et al. Three steps is all you need: fast, accurate, automatic scaling decisions for distributed streaming dataflows[C]//13th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2018: 783-798.
[45] Mei Y, Cheng L, Talwar V, et al. Turbine: Facebook's Service Management Platform for Stream Processing[C]//Proceedings of the IEEE International Conference on Data Engineering (ICDE). 2020: 1591-1602.
[46] She Z, Mao Y, Xiang H, et al. StreamSwitch: Fulfilling Latency Service-Layer Agreement for Stateful Streaming[C]//Proceedings of the IEEE Conference on Computer Communications

(INFOCOM). 2023: 1-10.
[47] Lian J, Zhang X, Shao Y, et al. ContTune: Continuous Tuning by Conservative Bayesian Optimization for Distributed Stream Data Processing Systems[C]//Proceedings of the VLDB Endowment (VLDB). 2023, 16(13): 4282-4295.
[48] Akiba T, Sano S, Yanase T, et al. Optuna: A Next-generation Hyperparameter Optimization Framework[C]//Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD). 2019: 2623-2631.
[49] Bergstra J, Bardenet R, Bengio Y, et al. Algorithms for hyper-parameter optimization[C]//Proceedings of the International Conference on Neural Information Processing Systems (NIPS). 2011: 2546-2554.
[50] Hansen N, Ostermeier A. Completely Derandomized Self-Adaptation in Evolution Strategies[J]. Evolutionary Computation, 2001, 9(2): 159-195.
[51] Wang L, Yang L, Yu Y, et al. Morphling: Fast, Near-Optimal Auto-Configuration for Cloud-Native Model Serving[C]//Proceedings of the ACM Symposium on Cloud Computing (SoCC). 2021: 639-653.
[52] Model Analyzer — NVIDIA Triton Inference Server[EB/OL]. https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/perf_benchmark/model_analyzer.html.
[53] Agrawal A, Kedia N, Mohan J, et al. VIDUR: A LARGE-SCALE SIMULATION FRAMEWORK FOR LLM INFERENCE[C]//Proceedings of Machine Learning and Systems (MLSys). 2024, 6: 351-366.
[54] Amazon S3 - Cloud Object Storage[EB/OL]. https://aws.amazon.com/s3.
[55] 阿里云对象存储 OSS[EB/OL]. https://www.aliyun.com/product/oss.
[56] Apache Flink[EB/OL]. https://flink.apache.org.
[57] Kubernetes[EB/OL]. https://kubernetes.io.
[58] Apache Flink Kubernetes Operator[EB/OL]. //nightlies.apache.org/flink/flink-kubernetes-operator-docs-release-1.11.
[59] Ray on Kubernetes[EB/OL]. https://docs.ray.io/en/latest/cluster/kubernetes/index.html.
[60] Apache Flink 2.0.0: A new Era of Real-Time Data Processing[EB/OL]. https://flink.apache.org/2025/03/24/apache-flink-2.0.0-a-new-era-of-real-time-data-processing.
[61] FLIP-427: ForSt - Disaggregated state Store[EB/OL]. https://cwiki.apache.org/confluence/display/FLINK/FLIP-427%3A+ForSt+-+Disaggregated+state+Store.
[62] 阿里云 GPU 云服务器[EB/OL]. https://www.aliyun.com/product/ecs/gpu.
[63] qGPU 概述[EB/OL]. https://www.tencentcloud.com/zh/document/product/457/42973.
[64] GPU 容器共享技术 cGPU[EB/OL]. https://help.aliyun.com/zh/egs/what-is-cgpu.
[65] NVIDIA Multi-Process Service[EB/OL]. https://docs.nvidia.com/deploy/mps/index.html.
[66] [FLINK-3660] Measure latency of elements and expose it as a metric[EB/OL]. https://issues.apache.org/jira/browse/FLINK-3660.
[67] Dosovitskiy A, Beyer L, Kolesnikov A, et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale[C]//Proceedings of the International Conference on Learning Representations (ICLR). 2021: 1-21.
[68] He K, Zhang X, Ren S, et al. Deep Residual Learning for Image Recognition[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016: 770-778.
[69] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding[J]. arXiv preprint arXiv:1810.04805, 2019.
[70] Valiant L G. A bridging model for parallel computation[J]. Commun. ACM, 1990, 33(8): 103-111.
[71] Xie Z, Emani M, Yu X, et al. Centimani: Enabling Fast AI Accelerator Selection for DNN Training with a Novel Performance Predictor[C]//Proceedings of the International USENIX Annual Technical Conference (ATC). 2024: 1203-1221.
[72] Rong G, Kai Z, Zhihao X, et al. Fluid: Dataset Abstraction and Elastic Acceleration for Cloud-native Deep Learning Training Jobs[C]// Proceedings of the IEEE International Conference on Data Engineering (ICDE). 2022: 2182-2195.

致谢

行文至此，论文已接近尾声，我在南京大学的三年读研时间也接近了尾声。三年来，真切体会到了什么是做科学研究，也感受到了自己从单纯的被动接受知识到主动探索问题的转变。在这里要感谢学校的培养、导师的指导以及家人朋友的无私支持与帮助。

首先要感谢我的导师顾荣老师。顾老师的言传身教深深感染了我，教会了我该如何做科研。顾老师严谨的治学态度、渊博的学识和敏锐的学术洞察力，不仅为我的科研道路指明了方向，更让我深刻理解了科研的真谛。同时我也要感谢戴海鹏老师，戴老师勤奋、严谨的工作态度是我一直以来的学习的目标。

同窗情谊是求学路上珍贵的礼物。我要感谢我的同学、舍友，是你们在我求学路上给了我无私的帮助。在科研遇到困难时，是你们的陪伴给了我力量，也是和你们的讨论给了我新的思路，那些关于学术观点的激烈辩论和相互鼓励，都将成为我珍藏的记忆。

最后，我要感谢我的父母。漫漫求学路，是你们用无条件的爱与支持为我筑起最坚实的后盾。或许你们并不清楚我的研究，不明白一个个学术名词，但你们一直以我为骄傲，只愿往后的时间，能够好好回报你们如山海般的恩情。

附录

研究生期间发表论文

[1] **Xiaozheng Zhang**, Xu Wang, Jing Shang, Zhiwen Xiao, Rong Gu. Efficient Serverless Stream Processing based on High-level Programming and Parallelism AutoTuni. **(IEEE HPCC, CCF-C, 2024)**. **[第一作者]**

研究生期间参与科研项目

[1] 基于 Fluid 与 Ray 的分布式 AI 模型推理系统性能优化研究，项目参与者. **[项目负责人为指导老师]**

[2] 江苏省研究生科研创新计划：流处理系统动态调整能力与云原生化的研究，项目负责人.

研究生期间获奖情况

[1] 华为“难题揭榜”火花奖：PB 级数据实时计算引擎，主要参与者.